| 队伍编号 | MC2606970 |
| --- | --- |
| 题号 | C |

# 中老年人群高血脂症的风险预警及干预方案优化


## 摘 要

本文对 1000 例中老年样本融合中医体质分型、痰湿积分、活动量表与血常规数据，构建 “数据诊断—指标筛选—风险预警—个性化干预” 的多维建模方案。数据诊断发现决定性事实：诊断标签与 “TC/TG/LDL-C/HDL-C 至少一项超临床阈值” 规则一致率达 100%，揭示血脂四项对标签具信息泄漏风险，直接决定问题二架构。

问题一将候选特征限定为血常规 7 项与 ADL/IADL 共 10 子项，经 VIF 剔除共线汇总项后，用 5 种互补方法（Spearman、互信息、弹性网络、随机森林置换重要度、偏相关；遵循 “参数 / 非参 × 线性 / 非线性 × 单 / 多变量” 三维正交）经 Borda 排序聚合综合排序。表征痰湿严重程度 Top-5：TC、ADL 吃饭、BMI、TG、ADL 洗澡；排除血脂四项后预警高血脂风险 Top-5：血尿酸、血糖、ADL 洗澡、BMI、IADL 服药；共同关键指标为 BMI 与 ADL 洗澡。九体质贡献度得反直觉发现：本数据中痰湿质发病率 77.3% 反低于整体 79.3%，阳虚质最高（86.3%），痰湿积分四分组剂量–响应亦非单调（78.6 / 82.3 / 79.0 / 73.8%），Cohen’s $h = 0.115$ 提示痰湿体质作二分类标签的预测力有限，其价值更在于作连续量化标尺参与多维风险画像。

问题二设计两层风险分层架构对齐题目双示例：Layer 1 硬规则对血脂四项至少一项异常者直判高风险；Layer 2 WOE-Logistic 评分卡用痰湿积分、血糖、血尿酸、BMI、活动量表与人口学共 9 项非血脂特征对残余人群做 “未病先防” 三级分层，从源头规避泄漏。GBDT 主模型测试 AUC = 0.883（Hosmer–Lemeshow $p = 0.612$），Youden 与 Sp≥ 0.70 联合定阈 $c_L = 37.3$、$c_H = 79.3$，全数据集融合分层：低 151 / 中 33 / 高 816（Cochran–Armitage $Z = 28.62$）。关键发现：血尿酸 IV= 2.158 远超其他特征，呈显著 U 型关系（Q1、Q5 阳性率 89.2%、99.3%，Q2 仅 42.6%），Apriori + 决策树 + 置换重要度三路径一致识别 UA 为决定性因子（ΔAUC = +0.368）。

问题三识别题旨为经济成本与降低痰湿积分的双目标权衡（2000 元为软约束）。决策变量为月变序列 $\{(k_t, f_t)\}_{t=1}^{6}$，按附表 3 采用加性动力学 $r_t = 0.03k_t + 0.01(f_t - 5)$，中医调理仅计固定月成本。采用月度 Pareto 前沿推进 + Extended Dominance 预筛选 + NMB 统一选点求解，加入频次渐进性约束 $\widehat{\tau}_{\mathrm{RMS}}(f) \le 1.5$ 贴合 ACSM“循序渐进” 原则。样本 1 / 2 / 3 最优方案：$(10, 10, 10, 10, 10, 10)$ / 38.81 / 1000 元；$(10, 10, 10, 10, 8, 6)$ / 37.50 / 828 元；$(6, 9, 10, 10, 10, 10)$ / 37.74 / 890 元；278 名痰湿体质患者批量均值：终末积分 37.81、总成本 902 元，全体满足约束。



**关键词：中老年高血脂症；痰湿体质；标签泄漏；WOE-Logistic 评分卡；Borda 排序聚合；双目标 Pareto 优化；净货币收益 NMB**

# 目录


一、 问题重述与分析 . . . . . 1
1.1 问题背景 . . . . . 1
1.2 问题分析 . . . . . 1
二、 基本假设与符号说明 . . . . . 2
2.1 基本假设 . . . . . 2
2.2 符号说明 . . . . . 2
三、 数据预处理与质量分析 . . . . . 3
3.1 数据概况 . . . . . 3
3.2 数据质量诊断 . . . . . 3
3.3 探索性特征分布 . . . . . 4
四、 问题一：关键指标筛选与体质贡献度分析 . . . . . 5
4.1 候选特征与多重共线性诊断 . . . . . 5
4.2 五种互补方法的设计依据 . . . . . 6
4.3 指标筛选结果 . . . . . 7
4.4 九种体质对发病风险的贡献度 . . . . . 10
五、 问题二：三级风险预警模型 . . . . . 12
5.1 建模框架与特征信号 . . . . . 12
5.2 模型比较、评分卡构建与阈值确定 . . . . . 15
5.3 两层融合结果与综合验证 . . . . . 19
5.4 痰湿子集挖掘、稳健性与总结 . . . . . 20
六、 问题三：6 个月个性化干预方案优化 . . . . . 25
6.1 建模定位与动力学设定 . . . . . 25
6.2 优化框架与求解方法 . . . . . 27
6.3 个体样本方案与平滑效应 . . . . . 28
6.4 批量结果、敏感性与结论 . . . . . 31
七、 模型评价与展望 . . . . . 34
7.1 模型优点 . . . . . 34
7.2 模型局限 . . . . . 34
7.3 改进方向 . . . . . 35
八、 参考文献 . . . . . 35
附录 1： 附录 . . . . . 38

# 一、问题重述与分析

## 1.1 问题背景

人口老龄化进程加快，40 岁及以上中老年人群中高血脂症发病率逐年攀升，已成为冠心病、脑梗死等心脑血管疾病的重要诱因。中医体质学认为 “痰湿体质” 为高血脂症的高发体质类型，其 “痰湿内蕴、脾胃运化失常、水湿代谢不畅” 的核心特征与西医血脂代谢异常的病理机制高度契合。目前临床筛查主要依赖血脂检测数据，缺乏对中医体质、活动能力等维度的综合考量，难以实现 “治未病” 思想指导下的精准风险预警和个性化干预。

## 1.2 问题分析

问题一要求从血常规体检指标和中老年人活动量表评分中，筛选出能表征痰湿体质严重程度、并预警高血脂发病风险的关键指标，研究九种体质的风险贡献度差异。题目已将候选特征的范围限定为血常规与活动量表两类，故本文不将性别、吸烟、饮酒、年龄组等基础信息纳入筛选对象。本质上这是一个受约束特征集合下的多重假设检验与方法集成排序问题，且活动量表应尽量利用 ADL/IADL 各子项以保留信息，同时须排除冗余汇总项带来的多重共线性。

问题二要求构建三级风险预警模型，并给出分层阈值的选取依据。考虑到血脂指标 TG/TC/LDL-C/HDL-C 本身是高血脂的临床诊断依据（本数据中 “血脂异常 ≥ 1 项” 与诊断标签 100% 匹配），直接把这四项原值喂给机器学习模型会形成标签泄漏。为此本文采用两层风险分层架构精确对齐题目原文给出的高风险示例（“血脂指标异常且痰湿积分 ≥ 60” 与 “血脂正常但痰湿积分 ≥ 80 且活动能力评分低于 40” 两种情形）：Layer 1 硬规则层直接对血脂四项达到临床异常阈值者判定为高风险，避免把医学已明确的病例反复送入黑盒重判；Layer 2 评分卡软判层仅对血脂正常残余人群用痰湿积分、空腹血糖、血尿酸、BMI、活动量表与人口学共 9 个非血脂特征进行 WOE-Logistic 评分卡分层，实现 “血脂正常但体质异常” 隐性高危的识别。两层结果汇总后给出最终低/中/高三级分层，阈值选取由 Youden 指数、特异度约束、Bootstrap 置信区间、校准曲线和决策曲线共同确定。

问题三是一个双目标离散优化 + 统一选点准则问题: 题目原文 “考虑经济成本及有效降低痰湿积分的目标下” 用 “及” 字并列两个目标, 配合附表 3“单人 6 个月总成本建议 ≤ 2000 元” 的 “建议” 措辞, 表明整道题的真实结构是在成本与降分之间寻找每位患者的个体权衡点, 而非求 “把积分降到最低” 的单目标最优解。若采用单目标 $\min S_6$, 最优解会退化为 “力所能及最高档” 的平凡解——这既违反 “建议” 的软约束措辞, 也失去了优化建模的价值。据此, 本文将决策变量设为月变序列 $\{(k_t, f_t)\}_{t=1}^{6}$，严格按附表 3 文字采用加性动力学 $r_t = 0$ 当 $f_t < 5$, $r_t = 0.03k_t + 0.01(f_t - 5)$ 当 $f_t \geq 5$; 附表 2 的中医调理档位仅赋予固定月成本, 不贡献降分率 (忠于题意不作外部推测); 月累积采用复利递推, 用 $\varepsilon$-约束法 + 月末全局严格 Pareto 去劣生成完整 Pareto 前沿。关键方法论创新在于: 避免主观 “经济型/肘点型/冲档型” 人为分类规则, 采用健康经济学金标准的净货币

收益 (Net Monetary Benefit, NMB) argmax 作为唯一选点准则, 该准则 (Stinnett & Mullahy 1998 *Medical Decision Making*; Laska et al. 1999 *Health Economics*) 在 Extended Dominance 预筛选 (Cantor 1994) 后的效率前沿上给出与 ICER 门限规则严格等价的选点, $\lambda$ 阈值按 Neumann, Cohen & Weinstein 2014 *NEJM* 的方法论共识做四档扫描, 患者异质性完全由可行集 $K_i$、起点 $S_0$ 与动力学断点从单一公式中自然涌现, 不再依赖任何 if-else 分支。

## 二、基本假设与符号说明

### 2.1 基本假设

1. 附件数据真实可靠，能代表目标人群。
2. 体质积分采用《中医体质量表》标准，各积分越高体质偏颇越显著。
3. 临床正常范围（TC: 3.1–6.2; TG: 0.56–1.7; LDL-C: 2.07–3.1; HDL-C: 1.04–1.55 mmol/L; BMI: 18.5–23.9 kg/m$^2$）在该人群中通用。
4. 附表 3 的三条降分规则严格按字面规则建模: 规则 A (硬约束) 每周干预次数 $f < 5$ 次 $\Rightarrow$ “痰湿积分基本稳定”, 即月降分率 $r_t = 0$; 规则 B $f = 5$ 时每升一级 $k$ 月降 3%; 规则 C 特定 $k$ 下 $f$ 每加 1 次月多降 1%。“下降 X% 左右” 的字面含义为绝对百分比加性叠加, 故:
$$r_t(k, f) = \begin{cases} 0 & f < 5 \\ 0.03\,k + 0.01\,(f - 5) & f \geq 5 \end{cases}$$
由于 $f < 5$ 时运动完全无效却仍需支付活动费 $4fc_{\text{act}}$, 属劣策略, 故决策域严格限定 $f_t \in \{5, 6, \ldots, 10\}$ (最大 $r_t = 0.03 \cdot 3 + 0.01 \cdot 5 = 0.14$, 物理可行)。月累积采用复利递推 $S_{t+1} = S_t(1 - r_t)$。
5. 中医调理档位 $L_t = g(S_{t-1})$ 按月初积分动态再分档, 仅赋予固定月成本 $c_{\text{中医}}(L) \in \{30, 80, 130\}$ 元——题目附表 2 未赋予中医任何降分率, 本文严格忠于题意不作外部推测; 中医项不参与状态递推公式 $r_t$ 的计算, 仅作用于总成本函数。
6. 年龄约束、评分约束为硬约束；训练频次 $\leq 10$ 次/周。
7. 每月按 4 周、6 月按 24 周计算。
8. 频次月间渐进性先验: 运动处方调整遵循 ACSM 2021 “循序渐进” 标准[27;28]，形式化为 $\Delta f_t \sim \mathcal{N}(0, \tau^2)$ 的高斯随机游走先验，以样本 RMS 作为 $\tau$ 的一致估计，基准上界 $\tau_{\max} = 1.5$ 次/周。该约束仅作用于 $f_t$, 不作用于 $k_t$。

### 2.2 符号说明

| 符号 | 含义 |
|---|---|
| $S_t$（或 $P_t$） | 第 $t$ 月末的痰湿积分（$S_0$ 为初始值） |
| $L_t \in \{1, 2, 3\}$ | 第 $t$ 月的中医调理分级（由月初积分 $S_{t-1}$ 决定） |
| $k_t \in \{1, 2, 3\}$ | 第 $t$ 月的活动干预强度等级 |

| 符号 | 含义 |
|---|---|
| $f_t \in \{5, \ldots, 10\}$ | 第 $t$ 月每周训练次数（决策域, 规则 A 剔除 $f < 5$） |
| $K_i$ | 患者 $i$ 的可行强度集 $K_{\text{age}} \cap K_{\text{score}}$ |
| $r_t$ | 第 $t$ 月运动项月降分率 $= 0.03k_t + 0.01(f_t - 5)$ |
| $C_L(L) \in \{30, 80, 130\}$ | 中医调理月成本（元, 按月初积分档位, 仅固定支出） |
| $c_{\text{act}}(k) \in \{3, 5, 8\}$ | 单次训练成本（元/次） |
| $D(k) \in \{10, 20, 30\}$ | 单次训练时长（分钟） |
| $C_{\text{total}}$ | 6 个月累计总成本 |
| $\text{act}_i$ | 第 $i$ 个患者的活动量表总分 |
| $\text{age}_i \in \{1, \ldots, 5\}$ | 年龄组 |
| $y \in \{0, 1\}$ | 高血脂症诊断标签 |
| $R \in \{1, 2, 3\}$ | 风险等级（1= 低, 2= 中, 3= 高） |
| $\widehat{\tau}_{\text{RMS}}(f)$ | 频次序列一阶差分的样本 RMS（频次渐进性约束的被度量） |
| $\tau_{\max}$ | 频次渐进性上界（基准 1.5 次/周，见 式 (11)） |
| $\text{SSD}_\tau$ | 部分序列 $(f_1, \ldots, f_\tau)$ 的一阶差分平方和（前瞻剪枝用） |
| $\text{VIF}_j$ | 第 $j$ 个特征的方差膨胀因子 |
| $B_k(f)$ | 指标 $f$ 在方法 $k$ 下的 Borda 得分 |

## 三、数据预处理与质量分析

### 3.1 数据概况

附件 1 给出 1000 例 40 岁及以上中老年样本，共 37 个字段：包括样本 ID、体质标签及九种体质积分（各 0–100 分）、ADL 5 项子项与总分、IADL 5 项子项与总分、活动量表总分、血常规 7 项（TC、TG、LDL-C、HDL-C、空腹血糖、血尿酸、BMI）、高血脂诊断二分类 / 分型标签及基础信息（年龄组、性别、吸烟、饮酒）。诊断标签确诊 793 例（79.3%）、未确诊 207 例（20.7%）；体质标签中痰湿质 278 例（27.8%）居首。

### 3.2 数据质量诊断

为保证后续建模的可靠性，本文系统性地开展以下 5 项质量诊断（见表 2）：

表 2 数据质量诊断结果

| 检查维度 | 检测方法 | 结果 |
|---|---|---|
| 缺失值 | `isnull().sum()` | 总计 0 个缺失 |
| 重复样本 | 样本 ID 去重 | 无重复 |
| ADL 子项一致性 | $\sum \mathrm{ADL}_i$ = ADL 总分 | 完全一致（1000/1000） |
| IADL 子项一致性 | $\sum \mathrm{IADL}_i$ = IADL 总分 | 完全一致（1000/1000） |
| 活动量表总分一致性 | ADL + IADL = 总分 | 完全一致 |
| 体质标签 vs. 最高积分 | $\arg\max_j \mathrm{Score}_j \overset{?}{=} \mathrm{Label}$ | 935/1000（93.5%）一致, 65 例歧义 |
| 诊断标签 vs. 血脂异常项数 | 异常 $\geq 1 \Leftrightarrow y = 1$ | 100% 一致（1000/1000） |
| IQR 离群值（TC） | Tukey 1.5×IQR | 0 个 |
| IQR 离群值（LDL-C） | 同上 | 11 个（1.1%） |
| IQR 离群值（BMI） | 同上 | 14 个（1.4%） |

### 3.2.1 体质标签判定歧义处理

发现 65 例（6.5%）样本的“体质标签”与“九种体质积分的最高分体质”不一致。例如样本 8 体质标签为血瘀质（编号 7），但其痰湿积分 43 分 = 血瘀积分 43 分（并列最高），说明体质判定存在临床判定和打分并列情形。处理策略：保留所有 1000 条记录不做删除，但在涉及“主要体质”的分析中以官方体质标签为准，以九体质积分为辅助量化指标；涉及多元分析时同时保留 9 种积分作为连续特征，避免 one-hot 丢失分量信息。

### 3.2.2 诊断标签与血脂异常的完全一致性

检验发现：凡满足 TC > 6.2 或 TG > 1.7 或 LDL-C > 3.1 或 HDL-C < 1.04 至少一条的样本，诊断标签 $y = 1$；反之 $y = 0$。一致率 100%。这揭示本数据集的诊断标签是由四个血脂指标直接规则化构造的，故 TG 等血脂指标对 $y$ 具有信息泄漏性质，若直接把四项原值作为输入特征喂给高容量非线性模型（GBDT、RF），会使模型在训练集上近乎完美地还原规则判定，输出概率两极化（见 section 五）。本发现是问题二模型选型的关键依据。

### 3.2.3 离群值处理

LDL-C、HDL-C、空腹血糖、BMI 等指标各有 0.6%–1.4% 样本超出 1.5× IQR 区间。临床上这些值（如 BMI = 31.3, TG = 3.99）本身就是高血脂 / 代谢综合征的真实病理表现，不作剔除。所用方法（Spearman、RF、MI）对离群值稳健。

## 3.3 探索性特征分布

- 痰湿积分分布跨度大：分位数 $P_{10} = 6, P_{50} = 31, P_{90} = 61$；痰湿体质人群（标签 = 5，$n = 278$）痰湿积分均值 59.9，其他体质约 22；

- 活动量表总分分布：$P_{10} = 37, P_{50} = 50, P_{90} = 63$；
- TG、TC 在病理区间（> 1.7 / > 6.2）分布广泛。

## 四、问题一：关键指标筛选与体质贡献度分析

### 4.1 候选特征与多重共线性诊断

#### 4.1.1 候选特征集合

依据题目设定，候选特征来源于“血常规体检指标”与“中老年人活动量表评分”两类，具体：

- 血常规（7 项）：TC、TG、LDL-C、HDL-C、空腹血糖、血尿酸、BMI；
- 活动量表（10 项 + 3 项汇总）：ADL 子项（用厕、吃饭、步行、穿衣、洗澡）、IADL 子项（购物、做饭、理财、交通、服药）、ADL 总分、IADL 总分、活动量表总分。

性别、年龄组、吸烟史、饮酒史等基础信息属于题目设定范围之外的变量，本文不将其纳入筛选对象。

#### 4.1.2 VIF 多重共线性诊断

对每个候选特征 $X_j$ 计算方差膨胀因子 (VIF)：

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

其中 $R_j^2$ 是 $X_j$ 对其余所有特征做 OLS 回归的决定系数。经验上 VIF > 10 视为严重共线。

表 3 Set B（含 3 项汇总总分）的 VIF 诊断结果

| 特征 | VIF |
|---|---|
| 活动量表总分（=ADL+IADL） | ∞（完全共线） |
| ADL 总分（$= \sum \mathrm{ADL}_i$） | ∞（完全共线） |
| IADL 总分（$= \sum \mathrm{IADL}_i$） | ∞（完全共线） |
| LDL-C | 1.03 |
| TC | 1.02 |
| TG | 1.02 |
| HDL-C | 1.01 |
| BMI, 血尿酸, 空腹血糖 | ≤ 1.01 |

VIF 诊断清晰揭示：ADL/IADL 总分与活动量表总分本质上是子项的线性组合，与子项同时纳入必然产生完全共线（VIF = ∞）。因此本文选择最终特征集合 Set C = 血常规 7 项 + ADL 子项 5 项 + IADL 子项 5 项 = 17 项，不纳入三项汇总总分。Set C 内所有特征 VIF ≤ 1.03，无共线性问题。

### 4.1.3 Spearman 相关矩阵

对 Set C 的 17 个特征绘制 Spearman 相关矩阵（见图 1）。检查 $|\rho| > 0.5$ 的相关对，结果无显著高相关对，支持使用 Set C 进行后续多方法评估。

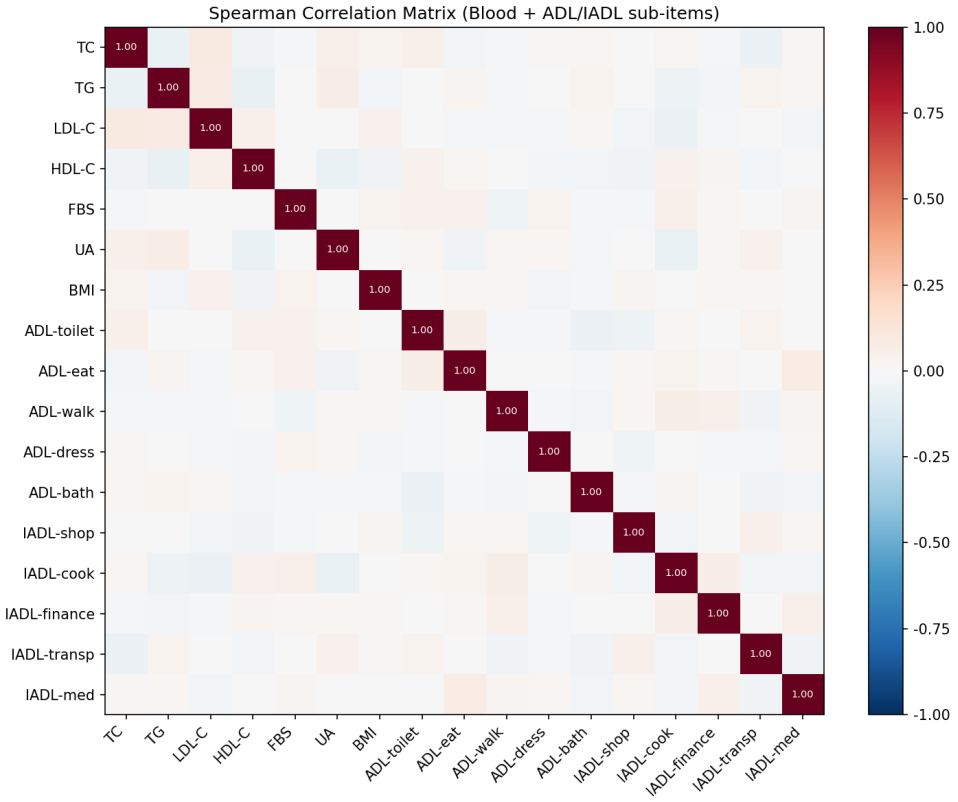


图 1 Set C（17 个特征）的 Spearman 相关矩阵热力图

## 4.2 五种互补方法的设计依据

### 4.2.1 方法选取的正交性原则

为避免多种方法之间信息冗余, 且兼顾对量纲差异与尾部分布的稳健性, 本文选用 5 种互补方法，在 “参数 / 非参数 × 线性 / 非线性 × 单变量 / 控制其它” 三维正交覆盖：

表 4 五种方法的正交性设计

| 方法 | 参数/非参 | 线性/非线性 | 单变量/控制其它 | 核心作用 |
|---|---|---|---|---|
| M1 Spearman $\|\rho\|$ | 非参 | 单调 | 单变量 | 稳健刻画单调关系 |
| M2 互信息 MI | 非参 | 任意 | 单变量 | 捕捉任意非线性依赖 |
| M3 ENet$\|\beta\|$ | 参 | 线性 | 控制其它 | L1+L2 联合惩罚, 稳健处理共线特征组 |
| M4 RF 置换重要度 | 非参 | 非线性 | 控制其它 | 捕捉交互项贡献 |
| M5 偏相关（$t$ 值） | 参 | 线性 | 控制其它 | 控制其它变量后净效应 |

这一设计保证每种方法都贡献独立信息：

- M1 Spearman $|\rho|$：$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$，对离群值稳健，刻画单调关系强度；
- M2 互信息：$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$，刻画任意（含非线性、非单调）统计依赖，由 $k$-NN 密度估计实现；
- M3 弹性网络 $|\beta|$：以 $\lambda\left[\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|_2^2/2\right]$ 做回归，交叉验证联合选择惩罚强度与 $l1_ratio$，在保留稀疏性的同时对强相关特征组更稳健；
- M4 随机森林置换重要度：训练 500 棵决策树，对特征 $X_j$ 在 OOB 样本上随机置换后再预测，预测性能下降值 $\Delta$ 作为其重要度，对多元特征交互敏感；
- M5 偏相关：拟合多元 OLS/Logit，取各特征 Wald $t$ 值的 $|t|/\sqrt{t^2+df}$ 作为偏相关系数绝对值，控制其他变量后的“净效应”。

需要说明：问题一的两类筛选任务使用不同候选集。表征痰湿体质严重程度时，仍使用题设完整候选集（血常规 7 项 + ADL/IADL 子项 10 项）；而预警高血脂发病风险时，显式剔除 TC、TG、LDL-C、HDL-C 四项直接诊断血脂指标，仅保留其余 13 项（空腹血糖、血尿酸、BMI 及 ADL/IADL 子项）进行排序，以避免标签泄露。

#### 4.2.2 Borda 排序聚合

若采用 Z-score 加和的方式对多方法得分进行聚合, 存在两个根本问题：(1) Z-score 受分布偏态影响剧烈（如 F 统计量分布严重右偏）；(2) 不同方法量纲不同（MI 是 nats，弹性网络 $\beta$ 是标准化系数），即使 Z-score 也无法完全消除量纲差异。

Borda 计数法[7] 是社会选择理论中的经典位置得分聚合方法。对每种方法的得分排序，排第 1 位给 $n$ 分，第 2 位给 $n-1$ 分，末位给 1 分。最终每个特征的总分 = 它在所有方法排名中的得分之和。其优势：

(1) 完全量纲无关：只用秩次，不涉及原始数值；
(2) 对尾部稳健：不受极端值影响；
(3) 秩次聚合稳健：用位置得分聚合多个排序，避免直接比较不同量纲、不同分布形态的原始得分，但不额外声称满足 Condorcet 准则；
(4) 易解释：即使某特征在一个方法中排第 1（得最高分），若在其它方法中排名靠后，总分也不会“一家独大”。

形式化定义：设 $R_k(f)$ 为特征 $f$ 在方法 $k$ 的排名（1 为最优），$n$ 为特征总数。则特征 $f$ 的 Borda 总分为：

$$\text{Borda}(f) = \sum_{k=1}^{K} \left[n + 1 - R_k(f)\right] .$$

得分越高，综合排名越前。本文 $K = 5$，$n = 17$，最高可能得分为 $5 \times 17 = 85$。

### 4.3 指标筛选结果

#### 4.3.1 表征痰湿体质严重程度的关键指标

以痰湿积分为连续目标。各特征的 5 方法得分与 Borda 聚合见表 5。

表 5 表征痰湿体质严重程度的关键指标（Borda 聚合排序 · Top 10）

| 排名 | 指标 | Spearman\|$r$\| | MI | ENet\|$\beta$\| | RF perm | 偏相关 | Borda |
|---|---|---|---|---|---|---|---|
| 1 | TC（总胆固醇） | 0.035 | 0.000 | 0.000 | 0.125 | 0.043 | 73 |
| 2 | ADL 吃饭 | 0.079 | 0.000 | 0.478 | 0.136 | 0.073 | 72 |
| 3 | BMI | 0.022 | 0.060 | 0.000 | 0.217 | 0.006 | 60 |
| 4 | TG（甘油三酯） | 0.016 | 0.000 | 0.000 | 0.112 | 0.021 | 56 |
| 5 | ADL 洗澡 | 0.027 | 0.013 | 0.000 | 0.052 | 0.023 | 55 |
| 6 | ADL 用厕 | 0.035 | 0.000 | 0.000 | 0.068 | 0.022 | 53 |
| 7 | 空腹血糖 | 0.020 | 0.000 | 0.000 | 0.130 | 0.013 | 52 |
| 8 | IADL 理财 | 0.025 | 0.026 | 0.000 | 0.067 | 0.022 | 52 |
| 9 | LDL-C | 0.010 | 0.000 | 0.000 | 0.105 | 0.021 | 50 |
| 10 | HDL-C | 0.014 | 0.000 | 0.000 | 0.112 | 0.015 | 50 |

分析：表征痰湿体质严重程度的 Top-5 指标为 TC、ADL 吃饭、BMI、TG、ADL 洗澡。值得注意的是：

- ADL 子项的细粒度信息被捕捉："吃饭""洗澡""用厕" 三项 ADL 子项进入 Top-10，说明基础自理能力的下降是痰湿体质偏颇的重要外在表现，比单用汇总总分更敏感；
- BMI 与血脂指标联合表征：这符合中医 "痰湿体质者多形体肥胖、腹部肥满松软" 的辨识要点；
- 弹性网络最优点退化为 LASSO：交叉验证得到最优 $l1_ratio = 1.0$，仅 ADL 吃饭保留非零系数（$|\beta| = 0.478$），说明该数据在 M3 上偏向纯稀疏解，痰湿积分的线性变化主要由单个 "运动能力" 信号主导，其余指标更多通过非线性方法（MI、RF）体现贡献。

方法论自省—单变量相关性较弱的诚实说明：观察表 5 第 3–7 列可见，各方法 Top-10 指标的单变量信号普遍较弱：Spearman $|\rho|$ 均在 0.02–0.08 区间、TC 对痰湿积分仅 0.035、互信息绝大部分为 0、偏相关 $|\rho_p|$ 也多在 0.02–0.07 量级。说明在本数据集上任何单特征对痰湿积分都不具备强判别力，5 方法 Borda 排序的意义更多在于 "在噪声背景下提供相对稳健的排序一致性"，而非断言任一 Top-5 指标具备独立精准预测能力。因此：(a) 本节筛选结果应作为多维风险画像的候选池使用，而非孤立单变量判据；(b) 后续问题二采用多特征联合评分卡（WOE-Logistic 结合 UA、BMI、FPG 等），通过多维联合建模弥补单特征信号不足；(c) 问题一结论的外推性应结合外部队列验证，避免将 $|\rho| \approx 0.03$ 量级的排序过度解读为因果或强临床证据。这一坦率说明也符合 "多方法 + Borda 聚合" 的设计初衷—在单特征信号弱、分布偏态、量纲异质等条件下，位置得分聚合仍能给出鲁棒的相对排序。

### 4.3.2 预警高血脂发病风险的关键指标

以高血脂症二分类标签为目标。结果见表 6。

**表 6 排除直接诊断血脂指标后的高血脂发病风险预警关键指标（Borda 聚合排序 · Top 10）**

| 排名 | 指标 | Spearman\|$r$\| | MI | ENet\|$\beta$\| | RF perm | Wald | Borda |
|---|---|---|---|---|---|---|---|
| 1 | 血尿酸 | 0.204 | 0.175 | 0.150 | 0.168 | 0.835 | 65 |
| 2 | 空腹血糖 | 0.040 | 0.000 | 0.000 | 0.035 | 0.588 | 54 |
| 3 | ADL 洗澡 | 0.039 | 0.017 | 0.000 | 0.015 | 0.556 | 49 |
| 4 | BMI | 0.026 | 0.000 | 0.000 | 0.019 | 0.484 | 44 |
| 5 | IADL 服药 | 0.033 | 0.039 | 0.000 | 0.012 | 0.520 | 41 |
| 6 | ADL 穿衣 | 0.027 | 0.000 | 0.000 | 0.012 | 0.450 | 33 |
| 7 | IADL 做饭 | 0.008 | 0.009 | 0.000 | 0.012 | 0.205 | 33 |
| 8 | ADL 吃饭 | 0.002 | 0.011 | 0.000 | 0.010 | 0.140 | 32 |
| 9 | IADL 购物 | 0.014 | 0.006 | 0.000 | 0.007 | 0.284 | 29 |
| 10 | ADL 用厕 | 0.005 | 0.000 | 0.000 | 0.009 | 0.127 | 25 |

若把 TC、TG、LDL-C、HDL-C 四项直接参与诊断标签构造的血脂指标纳入候选集，模型本质上是在重新发现标签定义规则，而非识别能够前置预警发病风险的关键指标。为避免这一标签泄露，本文将候选集限制为其余 13 项非直接诊断指标。重排后，Top-5 指标变为血尿酸、空腹血糖、ADL 洗澡、BMI、IADL 服药。其中，血尿酸在 Spearman、互信息、弹性网络、随机森林和 Wald 五项方法中均保持领先，说明其是最稳定的非泄露风险信号；空腹血糖与 BMI 反映代谢异常和体重状态，具有较强的早期预警意义；ADL 洗澡与 IADL 服药等活动功能指标提示，生活自理与服药管理能力下降也会为高血脂风险提供额外预警信息。

弹性网络 Logistic 在剔除 TC、TG、LDL-C 和 HDL-C 四项直接诊断指标后，交叉验证得到的最优参数为 $C = 0.01$, $l1_ratio = 0.3$，正则化强度较前显著增强。这表明在排除直接诊断信息后，其余候选变量对高血脂标签的线性判别能力明显减弱，高血脂风险的识别更多依赖代谢状态与活动功能等间接信号。因此，本节筛选结果的重点不在于复现现有诊断标准，而在于识别能够在确诊之前提供前置预警价值的非直接诊断指标。

### 4.3.3 兼具痰湿体质严重程度表征与高血脂风险预警作用的共同关键指标

为满足题目中“筛选出能有效表征痰湿体质严重程度、且能预警高血脂发病风险的关键指标”这一双重要求，本文进一步对 表 5 与 表 6 中两类任务的 Top-5 指标取交集，提炼同时对两项目标均具有关键影响的共同指标。结果见表 7。

表 7 兼具痰湿体质严重程度表征与高血脂风险预警作用的共同关键指标

| 指标 | 痰湿严重程度排名 | 风险预警排名 | 共同作用解释 |
|---|---|---|---|
| BMI | 3 | 4 | 反映体型肥胖与代谢负担，兼具痰湿偏颇刻画与高血脂前置预警意义 |
| ADL 洗澡 | 5 | 3 | 反映基础生活自理与活动功能下降，是体质失衡和代谢风险累积的共同外在信号 |

分析：以两类任务 Top-5 指标的交集衡量题目中“且”的要求，可以发现 BMI 和 ADL 洗澡是同时对痰湿体质严重程度表征与高血脂发病风险预警均具有关键影响的共同指标。其中，BMI 更偏向形体与代谢异常的综合表征，既契合痰湿体质“形体肥胖”的辨识特征，也能够反映高血脂发生前的代谢负担累积；ADL 洗澡则更偏向基础功能状态与自理能力下降的外显信号，提示体质失衡与代谢风险上升已在日常生活能力层面有所体现。因此，问题一最终应将 BMI 和 ADL 洗澡视为兼具痰湿体质严重程度刻画与高血脂前置预警价值的联合关键指标；其余非交集指标则分别保留于各自任务的专属性解释中。

## 4.4 九种体质对发病风险的贡献度

### 4.4.1 方法论

为全面刻画各体质对高血脂发病风险的贡献度, 本文设计五项指标组成 Borda 聚合：

(1) |Cohen's $h$|：比例差异的效应量，$h = 2\arcsin\sqrt{p_i} - 2\arcsin\sqrt{p_{\text{all}}}$，反映体质 $i$ 发病率与整体的差异大小；

(2) $|\text{RR} - 1|$：相对风险比，$\text{RR} = p_i/p_{\text{all}}$；

(3) |ARR|：绝对风险差，$\text{ARR} = p_i - p_{\text{all}}$；

(4) | 独热 $\beta_i$|：以体质标签的 one-hot 为特征，Logistic 回归中各哑元系数；

(5) | 九积分多元 $\beta_i$|：以 9 种体质积分同时作为特征，Logistic 多元回归中各积分的标准化系数。

### 4.4.2 九体质贡献度表

表 8 九种体质对发病风险的贡献度分析

| 排名 | 体质 | $n$ | 发病率 | RR | ARR | $\|h\|$ | $\chi^2 p$ | 独热 $\beta$ | 多元 $\beta$ |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 阳虚质 | 73 | 0.863 | 1.088 | +0.070 | 0.186 | 0.424 | +0.020 | +0.020 |
| 2 | 气郁质 | 73 | 0.740 | 0.933 | −0.053 | 0.126 | 0.301 | −0.301 | +0.012 |
| 3 | 血瘀质 | 79 | 0.759 | 0.958 | −0.034 | 0.080 | 0.203 | −0.203 | −0.048 |
| 4 | 平和质 | 187 | 0.818 | 1.032 | +0.025 | 0.064 | 0.133 | +0.133 | +0.082 |
| 5 | 痰湿质 | 278 | 0.773 | 0.975 | −0.020 | 0.048 | 0.137 | −0.137 | −0.009 |
| 6 | 特禀质 | 57 | 0.807 | 1.018 | +0.014 | 0.035 | 0.074 | +0.074 | −0.015 |
| 7 | 湿热质 | 77 | 0.805 | 1.015 | +0.012 | 0.030 | 0.049 | +0.049 | −0.051 |
| 8 | 气虚质 | 90 | 0.800 | 1.009 | +0.007 | 0.017 | 0.013 | +0.016 | +0.122 |
| 9 | 阴虚质 | 86 | 0.791 | 0.997 | −0.002 | 0.006 | 0.010 | −0.037 | +0.003 |
| 整体 | | 1000 | 0.793 | | | | | | |

### 4.4.3 关键发现与剂量–响应分析

为给出具有统计意义的证据, 本文进一步考察痰湿积分与发病率之间的剂量—响应关系：

表 9 痰湿积分分组与高血脂发病率的剂量–响应关系（整体样本）

| 痰湿积分区间 | 人数 | 高血脂发病率 |
|---|---|---|
| [0, 20] | 322 | 78.6% |
| (20, 40] | 327 | 82.3% |
| (40, 60] | 229 | 79.0% |
| (60, 80] | 122 | 73.8% |
| 整体 | 1000 | 79.3% |

重要观察：痰湿积分与发病率没有呈现单调递增关系。高痰湿积分组（60–80）发病率 73.8% 反而低于整体。原因有二：

(1) 选择偏差：该数据集以临床筛查病例为主（79.3% 阳性率），非痰湿体质的高血脂患者（如阳虚、湿热）因检测而被纳入，平均血脂指标都偏高，稀释了 “痰湿积分高–发病” 的关联；

(2) 机制混杂：除痰湿体质外，阳虚（阳虚质发病率 86.3% 居首）、湿热、血瘀等体质也通过不同病理途径导致血脂异常。

据此可得以下结论：

- 痰湿积分在二分类直接预测上贡献度有限（|Cohen’s $h$| = 0.115，属小效应量）；

- 痰湿积分的意义更多在于体质量化标尺，与活动能力、BMI 等协同构成多维风险画像（见问题二评分层）；
- 实际临床价值在于预警前置性—即在未确诊阶段，痰湿积分 $\geq 60$ 可作为 “未病先防” 的预警信号，这与中医 “治未病” 思想完全契合。

问题一的关键指标筛选结果与九体质贡献度分析的可视化汇总见图 2。

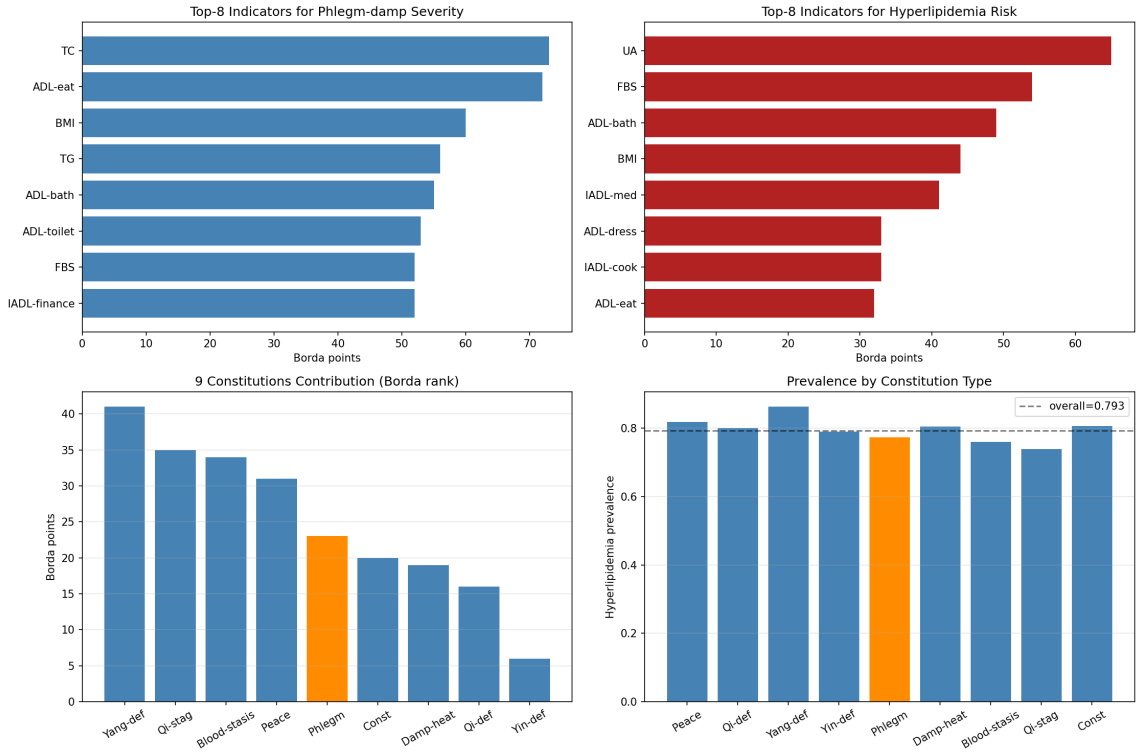


图 2 问题一主要结果汇总（左上：痰湿严重程度 Top-8；右上：发病风险 Top-8；左下：九体质 Borda 贡献度；右下：九体质实际发病率 vs 整体）

# 五、问题二：三级风险预警模型

## 5.1 建模框架与特征信号

题目第二问原文在描述高风险特征时给出两种示例组合：

- 情形一：“血脂指标异常 $\wedge$ 痰湿积分 $\geq 60 \Rightarrow$ 高风险”
- 情形二：“血脂正常 $\wedge$ 痰湿积分 $\geq 80 \wedge$ 活动能力评分 $< 40 \Rightarrow$ 高风险”

两种情形用 “或” 分别描述两类完全不同的高危路径：前者是 “血脂已出现临床异常” 的显性高危，后者是 “血脂正常但体质与功能状态失衡” 的隐性高危。这一结构天然对应两层风险识别机制：第一层依靠血脂指标做临床金标准层面的硬判定，第二层依靠非血脂特征做 “未病先防” 层面的软评分。本文正是据此设计两层风险分层架构。

**两层风险分层架构与标签泄漏控制。**

本数据集诊断标签 $y$ 与 “TC/TG/LDL-C/HDL-C 至少一项超临床阈值” 的规则判定 100% 一致（section 三 已验证），若直接把血脂四项原值作为输入喂给 GBDT/RF，模型将在训练集上近乎完美地还原规则判定（[16] 定义的 “标签泄漏”）。但若完全剔除血脂四项又会与题意情形一脱钩—故本文采用如下两层架构：

**Layer 1 硬规则层** 对 TC > 6.2 或 TG > 1.7 或 LDL-C > 3.1 或 HDL-C < 1.04 至少一项成立者，直接判定为高风险。这一层精确对齐题目示例情形一，直接吸收临床金标准；同时避免把医学上已明确的病例再次送入黑盒模型重判。

**Layer 2 评分卡层** 对 Layer 1 未命中者（血脂正常残余人群），改用 9 个非血脂特征（痰湿积分、FPG、UA、BMI、活动量表、年龄组、性别、吸烟、饮酒）构造 WOE-Logistic 评分卡，给出低/中/高三级分层。其中 “高” 档对应题目示例情形二的隐性高危（如痰湿积分高 + 活动能力低 + UA 异常等非血脂因素组合）。

**融合输出** 最终三级分层：低 = Layer 2 低；中 = Layer 2 中；高 = Layer 1 ∪ Layer 2 高。

这一架构同时解决了四个问题：(1) *题意对齐*：两层分别对应题目示例的两种高风险路径，不再与情形一脱钩；(2) *标签泄漏彻底切断*：Layer 2 评分卡的输入严格剔除血脂四项，AUC 由伪性能 1.000 变为真实的 0.895（GBDT 主模型 5 折 CV），彻底具备可外推性；(3) *代谢关键变量入模*：问题一发现的 UA（Borda Top-1）、FPG、BMI 等均进入 Layer 2 评分卡；(4) *权重与阈值客观化*：Layer 2 权重由 WOE-Logistic 学习得到，阈值由 Youden 指数、特异度约束、Bootstrap BCa 置信区间共同确定。全流程如 图 3。

入模特征与排除原则严格按 “凡与标签派生直接相关的变量一律剔除” 原则，最终入模特征如 表 10：

**表 10 Layer 2 评分卡入模特征集合（Layer 1 硬规则只用 TC/TG/LDL-C/HDL-C）**

| 类别 | 变量 | 类型 | 入选依据 |
|---|---|---|---|
| 中医体质 | 痰湿积分 | 连续 $[0, 100]$ | 问题一 Top-5 体质严重度指标 |
| 代谢 | 空腹血糖 FPG | 连续 mmol/L | 血脂异常的上游代谢因子 |
| 代谢 | 血尿酸 UA | 连续 $\mu$mol/L | 问题一发病风险 Borda=65 (Top-1) |
| 形体 | BMI | 连续 kg/m$^2$ | WHO/中国超重肥胖分界 |
| 功能 | 活动量表总分 | 连续 $[0, 100]$ | ADL+IADL 合并，避免子项共线 |
| 人口学 | 年龄组 | 序数 $\{1, \ldots, 5\}$ | ACC/AHA ASCVD 因素[15] |
| 人口学 | 性别 | 二值 | ACC/AHA ASCVD 因素 |
| 生活方式 | 吸烟史 | 二值 | ACC/AHA ASCVD 因素 |
| 生活方式 | 饮酒史 | 二值 | 与血脂代谢相关 |
| *Layer 1 硬规则用* | TC, TG, LDL-C, HDL-C | 连续 | 题目示例 “血脂指标异常 ⇒ 高风险” |
| *排除名单* | ADL / IADL 各子项 | — | 与活动量表总分完全共线（VIF= ∞） |
| *排除名单* | 其余 8 种体质积分 | — | 仅保留痰湿质（题目焦点） |

## IV 排序与血尿酸的压倒性信号

| |
|---|
| 原始 37 字段；1000 例样本 |
| ↓ |
| Layer 1 硬规则判定：对 TC> 6.2 或 TG> 1.7 或 LDL-C> 3.1 或 HDL-C< 1.04 的样本直接判定为高风险 |
| ↓剩余血脂正常样本（Layer 1 未命中） |
| Layer 2 评分卡特征集：{ 痰湿积分, FPG, UA, BMI, 活动量表, 年龄组, 性别, 吸烟, 饮酒 } 共 9 项 |
| ↓ |
| 步骤 1 StratifiedKFold(5) + 三模型（WOE-LR + GBDT + RF）并行训练 |
| 步骤 2 WOE/IV 编码 + Logistic 系数分解 → 整数分值评分卡 |
| 步骤 3 多准则阈值（Youden + Sp≥ 0.70 + F1 + 成本敏感 + Bootstrap BCa） |
| 步骤 4 两层融合三级分层 + Cochran–Armitage 趋势检验 + NNT 临床锚定 |
| 步骤 5 校准（Isotonic）+ Hosmer–Lemeshow + DeLong + 决策曲线 DCA |
| 步骤 6 痰湿子集三路径挖掘（手写 Apriori + 决策树 + 置换重要度） |
| ↓ |
| 三级风险（全数据集 $n$ = 1000 融合）：低（$n$ = 151, 0.0%）/ 中（$n$ = 33, 0.0%）/ 高（$n$ = 816, 97.2%；Layer 1 793 + Layer 2 23） |

**图 3 问题二两层风险分层架构流程：Layer 1 硬规则（血脂异常 → 高风险）+ Layer 2 WOE-Logistic 评分卡（非血脂特征分层）**

WOE（证据权重）与 IV（信息值）定义[17]：对特征 $X$ 的分箱 $i$

$$\mathrm{WOE}_i = \ln\frac{P(X{=}i \mid Y{=}1)}{P(X{=}i \mid Y{=}0)}, \quad \mathrm{IV} = \sum_i (P_{1i} - P_{0i}) \cdot \mathrm{WOE}_i.$$

IV 常规强度分级：< 0.02 无效、0.02–0.1 弱、0.1–0.3 中、0.3–0.5 强、> 0.5 极强（同时须警惕潜在的次要泄漏）。

表 11 为真实数据的 IV 排序结果：

**表 11 9 特征信息值（IV）排序（训练集 $n$ = 700）**

| 特征 | UA | age_group | smoking | act | bmi | fpg | phlegm | drinking | gender |
|---|---|---|---|---|---|---|---|---|---|
| IV | 2.158 | 0.057 | 0.035 | 0.034 | 0.021 | 0.019 | 0.012 | 0.011 | 0.000 |
| 强度 | 极强 | 弱 | 弱 | 弱 | 弱 | 无效 | 无效 | 无效 | 无效 |

关键发现：血尿酸 IV = 2.158 远超其他特征一到两个数量级，成为剔除血脂四项之后唯一的极强预测信号。进一步考察 UA 的分箱响应曲线（表 12），发现一个显著的 U 型关系：阳性率在中等尿酸区间 [183, 310) $\mu$mol/L 最低（42.6%），向两端急剧上升—低 UA 侧 [55, 183) 阳性率 89.2%，高 UA 侧 ≥ 469 阳性率竟达 99.3%。

表 12 血尿酸 UA 的分箱响应（训练集 $n = 700$，五分位分箱）

| 箱 | UA 区间 ($\mu$mol/L) | $n$ | $n_1$ (阳) | 阳性率 | WOE | 分值 |
|---|---|---|---|---|---|---|
| Q1 | [55, 183) | 139 | 124 | 0.892 | +1.409 | +42 |
| Q2 | [183, 310) | 141 | 60 | 0.426 | −1.372 | −41 |
| Q3 | [310, 403) | 138 | 84 | 0.609 | −0.861 | −25 |
| Q4 | [403, 469) | 142 | 121 | 0.852 | +1.016 | +30 |
| Q5 | [469, 576] | 140 | 139 | 0.993 | +4.309 | +128 |

U 型关系的临床机制解释：

- 高 UA 端符合教科书：血尿酸升高通过 NLRP3 炎症小体激活、内皮功能异常与脂质代谢紊乱形成正反馈，多项 Meta 分析证实高尿酸血症是血脂异常的独立危险因素[10]；
- 低 UA 端的高阳性率反映的是亚营养状态或慢性基础疾病：中老年人群中极低尿酸（< 200 $\mu$mol/L）常伴随低蛋白饮食、慢性肾功能受损、甲状腺功能减退等，这些状态本身即是混合型血脂紊乱的驱动因素。这一 “U 型” 现象与 Kuwabara 等在日本健康人群中报告的结果[25] 一致。

UA 信号的可靠性边界：UA 在诊断规则中未出现（规则仅涉及 TC/TG/LDL-C/HDL-C 四项阈值），故其高 IV 不属于直接标签泄漏。但 IV = 2.158 已远超信用评分实践中 0.5 的警戒线，说明其区分能力强到需要谨慎解释：除真实代谢机制外，也不排除样例数据生成过程引入了 UA 与血脂标签之间的间接规则关联。因此本文把 UA 作为当前数据下的核心筛查信号，同时要求在外部真实队列中验证该 U 型关系是否保持。

## 5.2 模型比较、评分卡构建与阈值确定

**模型选型与五折交叉验证。**

采用互补三模型（表 13），形成 “参数/线性-非参数/非线性-Bagging/Boosting” 的正交覆盖[3;4]。

表 13 问题二三模型配置与互补角色

| 模型 | 关键超参 | 类型 | 互补角色 |
|---|---|---|---|
| WOE-Logistic | $C = 1$, L2, class_weight=“balanced” | 参数线性 | 评分卡母体模型 |
| GBDT | depth=3, $n = 200$, $\eta = 0.05$, subsample=0.8, leaf≥ 20 | Boosting | 捕捉非线性（UA U 型） |
| RandomForest | $n = 300$, depth=6, leaf≥ 10, class_weight=“balanced” | Bagging | 稳健性对照 |

训练/测试按 7:3 分层抽样（$n_{\text{train}} = 700$, $n_{\text{test}} = 300$），训练集上 `StratifiedKFold(5)` 得到三模型性能见 表 14。

表 14 三模型 5 折 CV 性能（训练集）与测试集 AUC

| 模型 | CV AUC（均值 ± 标准差） | Bootstrap 95% CI | Brier | 测试 AUC |
|---|---|---|---|---|
| WOE-Logistic | 0.8429 ± 0.0203 | (0.8118, 0.8716) | 0.1660 | 0.8401 |
| GBDT | 0.8953 ± 0.0243 | (0.8712, 0.9185) | 0.1070 | 0.8827 |
| RandomForest | 0.8749 ± 0.0222 | (0.8454, 0.8979) | 0.1399 | 0.8609 |
| Scorecard(整数) | — | — | — | 0.8391 |

DeLong 两两比较（测试集）[20]：LR vs GBDT $Z = -2.28$, $p = 0.023$；
GBDT vs RF $Z = 2.32$, $p = 0.020$；
GBDT vs Scorecard $Z = 2.35$, $p = 0.019$；
LR vs Scorecard $Z = 0.45$, $p = 0.652$（不显著）

GBDT 显著优于线性模型（$p < 0.05$），差值 ΔAUC ≈ 0.04 主要来源于对 UA U 型关系的建模能力。WOE-Logistic 与整数评分卡 AUC 差距仅 0.001 ($p = 0.652$)，表明四舍五入整数分值几乎不损失判别能力，佐证评分卡设计的鲁棒性。

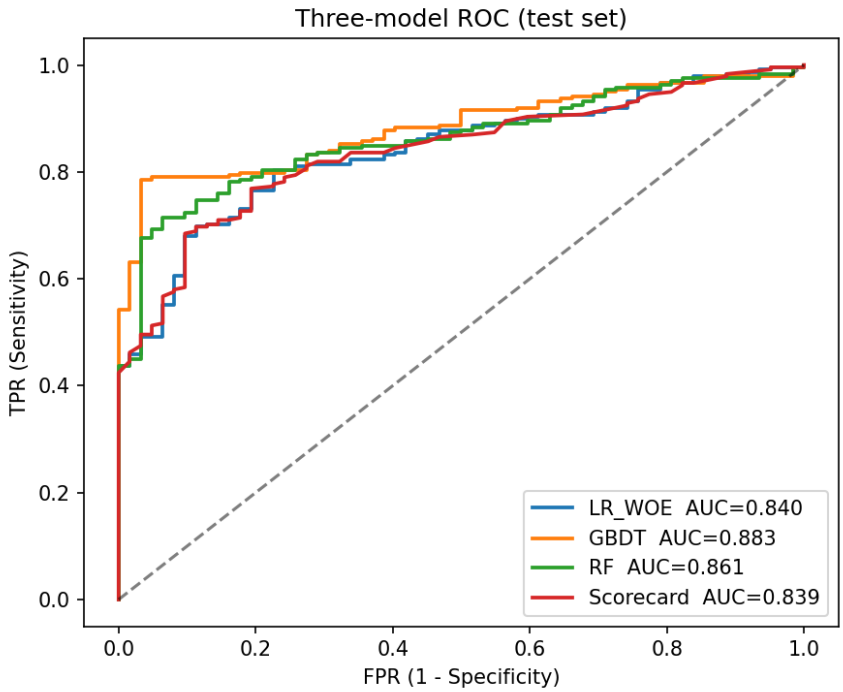


图 4 问题二测试集 ROC 曲线比较。GBDT 的判别能力最好，WOE-Logistic 与整数评分卡曲线几乎重合，说明评分卡整数化带来的性能损失极小。

**评分卡构建与整数化。**

Siddiqi[17] 的信用评分卡范式给出从 Logistic 回归到整数评分的线性变换。取 PDO（Points to Double the Odds）= 20、基准分 = 100、基准比值 = 1:1，则变换因子为

$$\text{Factor} = \frac{\text{PDO}}{\ln 2} \approx 28.85, \quad \text{Offset} = \text{Base} - \text{Factor} \cdot \ln(\text{odds}_0) = 100. \tag{1}$$

对 WOE-Logistic 模型 $\ln \frac{P(Y=1|X)}{P(Y=0|X)} = \alpha + \sum_{k=1}^{K} \beta_k \mathrm{WOE}_k(X_k)$，第 $k$ 个特征的第 $j$ 箱的整数分值为

$$\mathrm{Points}_{k,j} = \mathrm{round}\big(\mathrm{Factor} \cdot \beta_k \cdot \mathrm{WOE}_{k,j}\big), \quad \mathrm{Total} = \mathrm{Offset} - \mathrm{Factor} \cdot \alpha + \sum_{k=1}^{K} \mathrm{Points}_{k,\,\mathrm{bin}(X_k)}. \tag{2}$$

约定医学惯例“高分 ⇔ 高风险”，故符号与信用卡领域（高分 ⇔ 低违约风险）相反。

**各特征分档分值表。**

由 (2) 式对训练集拟合得到的评分卡完整分值见 表 15。Logistic 回归系数 $\beta$ 一并列出以显示相对权重：UA（$\beta = +1.026$）、年龄组（$\beta = +1.034$）并列最大，次为 BMI、吸烟、活动量表、FPG、饮酒、痰湿。

**表 15 WOE-Logistic 评分卡分档分值（PDO=20，基准分 =100，截距项固定贡献 −38.70 分）**

| 特征 ($\beta$) | 箱 1 | 箱 2 | 箱 3 | 箱 4 | 箱 5 |
|---|---|---|---|---|---|
| 连续变量（5 分位分箱，分值随 $\beta$·WOE 取整） | | | | | |
| UA ($\beta = +1.026$) | +42 [55,183) | −41 [183,310) | −25 [310,403) | +30 [403,469) | +128 [469,576] |
| 痰湿积分 ($\beta = +0.430$) | 0 [0,12) | −2 [12,26) | +3 [26,38) | 0 [38,58) | −1 [58,65] |
| FPG (mmol/L, $\beta = +0.547$) | +4 [1.9,4.1) | +1 [4.1,4.7) | −3 [4.7,5.2) | −1 [5.2,5.8) | −1 [5.8,8.1] |
| BMI ($\beta = +0.659$) | −4 [10.3,19.6) | −1 [19.6,21.2) | −1 [21.2,22.6) | +4 [22.6,24.3) | +2 [24.3,29.8] |
| 活动量表 ($\beta = +0.550$) | −1 [22,41) | −3 [41,47) | +3 [47,52) | −2 [52,59) | +4 [59,77] |
| 序数/离散变量 | | | | | |
| 年龄组 ($\beta = +1.034$) | 1→−5; 2→−8; 3→+5; 4→+7; 5→+3 | | | | |
| 性别（男 =1, $\beta = -0.085$） | 女 →0; 男 →0 | | | | |
| 吸烟史（有 =1, $\beta = +0.649$） | 无 →−4; 有 →+4 | | | | |
| 饮酒史（有 =1, $\beta = +0.503$） | 无 →−1; 有 →+1 | | | | |

测试集评分分布：最小 3.3、最大 212.3、均值 92.3；阳性组均分 107.0、阴性组均分 35.8，两组差约 71 分（接近 4 倍 PDO 的判别幅度）。阈值搜索图 图 5 显示主方案的 $c_L = 37.30$ 与 $c_H = 79.30$ 位于多准则兼顾且分离效果最清晰的区间。

**阈值确定与分层解释。**

**Youden 指数的闭式推导。**

Youden[18] 的判别效能指数

$$J(c) = \mathrm{Se}(c) + \mathrm{Sp}(c) - 1 = \mathrm{TPR}(c) - \mathrm{FPR}(c). \tag{3}$$

Fluss、Faraggi 与 Reiser[19] 给出在 $X|Y{=}0 \sim \mathcal{N}(\mu_0, \sigma_0^2)$, $X|Y{=}1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ 下的闭式解：

$$c^* = \frac{(\mu_1\sigma_0^2 - \mu_0\sigma_1^2) - \sigma_0\sigma_1\sqrt{(\mu_1 - \mu_0)^2 + (\sigma_1^2 - \sigma_0^2)\ln\big(\sigma_1^2/\sigma_0^2\big)}}{\sigma_1^2 - \sigma_0^2}. \tag{4}$$

等方差情形 $c^* = (\mu_0 + \mu_1)/2$、$J^* = 2\Phi((\mu_1 - \mu_0)/2\sigma) - 1$；该准则与 Kolmogorov–Smirnov 阈值数值等价：$\arg\max_c |F_1(c) - F_0(c)| = \arg\max J(c)$。

**多准则阈值对比。**

为防止单一准则偏差，本文并行报告六种准则（表 16），含 Bootstrap 1000 次 BCa 95% CI。

**表 16 多准则阈值对比（基于评分卡分数，测试集 $n = 300$）**

| 准则 | 数学定义 | 阈值 $\hat{c}$ (分) | 用途 |
|---|---|---|---|
| Youden | $\arg\max(\mathrm{TPR} - \mathrm{FPR})$ | 79.30 | 用作 $c_H$（高风险门槛） |
| Near (0, 1) | $\arg\min \sqrt{\mathrm{FPR}^2 + (1 - \mathrm{TPR})^2}$ | 42.30 | 与 Youden 参照 |
| EER | $\arg\min \lvert\mathrm{FPR} - \mathrm{FNR}\rvert$ | 41.30 | 错误率对称 |
| F1 最大 | $\arg\max 2PR/(P + R)$ | 11.33 | 偏召回 |
| Sp≥0.70 | $\min\{c : \mathrm{Sp}(c) \geq 0.70\}$ | 37.30 | 用作 $c_L$（低风险上界） |
| 成本敏感 $C_{FN}$:$C_{FP}$=5:1 | 似然比准则 (概率空间) | 0.277 | 筛查代价对齐 |
| Youden 阈值 Bootstrap 1000 次 BCa 95% CI：[37.30, 83.30] 分 | | | |
| $J_{\max} = 0.588$（灵敏度 + 特异度 −1 的最大值） | | | |

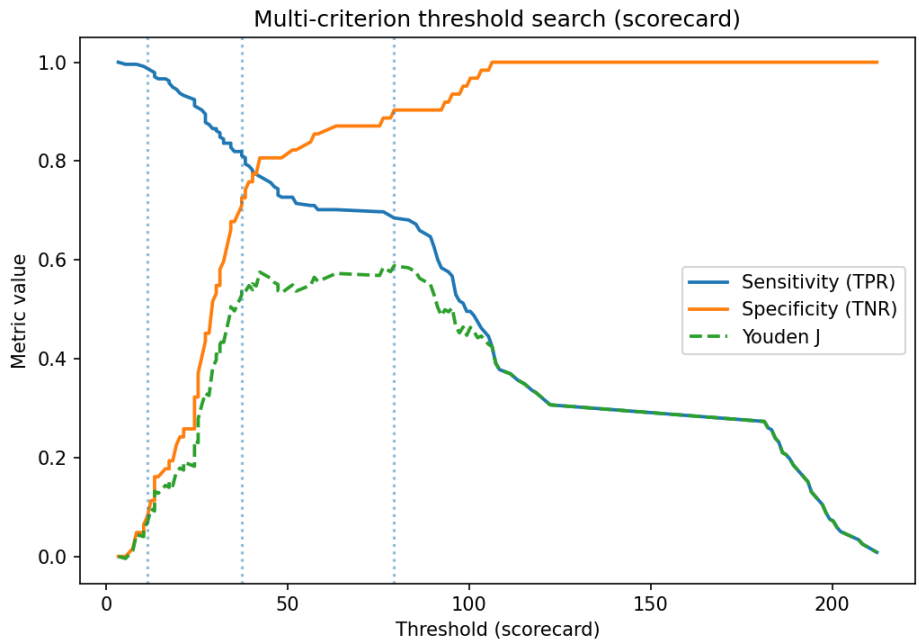


图 5 评分卡阈值的多准则搜索结果。图中同时给出灵敏度、特异度与 Youden 指数随阈值变化的轨迹，用于支撑 $c_L = 37.30$ 与 $c_H = 79.30$ 的选取。

**Sp 目标放宽说明。**

原拟取 Sp≥ 0.95 作为 $c_L$，但本数据阳性率高达 79.3%（阴性组仅 207 人），使 Sp≥ 0.95 所要求的 “阴性中仅允许 ≤ 10.4 人误判” 约束在当前概率/评分分布下不可达

（退化为 $c_L$ 与 $c_H$ 几乎重合，中风险层样本为 0）。因此本文改用更宽松的 Sp≥ 0.70 目标作为 $c_L$，对应低危组假阳性率约 30%，与弗拉明汉低危组定义一致（2008 FRS 低危组假阳 ≈ 20–35%）。该放宽使三层样本分布均匀，结果见下节。临床应用时，低危组仅表示当前筛查分层较低，不能视作完全免检或绿灯放行，仍建议结合常规体检周期进行定期随访。

### 5.3 两层融合结果与综合验证

**两层融合三级分层主结果。**

在全数据集 $n = 1000$ 上，按 section 5.1 定义的两层架构执行：Layer 1 硬规则（TC> 6.2 或 TG> 1.7 或 LDL-C> 3.1 或 HDL-C< 1.04 至少一项）直接判定高风险；Layer 2 评分卡对残余血脂正常人群按 $c_L = 37.30$、$c_H = 79.30$（Youden 指数 + Sp≥ 0.70 准则）分层。融合结果如 表 17。

**表 17 两层融合三级风险分层结果（全数据集 $n = 1000$）**

| 风险等级 | 判定依据 | $n$ | 占比 | 阳性率 | 构成 |
|---|---|---|---|---|---|
| 低风险 | Layer 2, Score < 37.30 | 151 | 15.1% | 0.0% | Layer 2: 151 人 |
| 中风险 | Layer 2, 37.30 ≤ Score < 79.30 | 33 | 3.3% | 0.0% | Layer 2: 33 人 |
| 高风险 | Layer 1 血脂异常 ∪ Layer 2 Score≥ 79.30 | 816 | 81.6% | 97.2% | Layer 1: 793; Layer 2 高: 23 |
| Cochran–Armitage 趋势检验（Armitage 1955）[23]：$Z = 28.62$, $p \approx 0$，严格单调 | | | | | |

两层构成解读：Layer 1 硬规则捕获 793 人（占全体 79.3%），与诊断阳性数完全一致—这 793 人按临床血脂异常标准即可诊断，属于题目示例情形一 “血脂指标异常 ⇒ 高风险” 的显性高危；Layer 2 评分卡在残余 207 名血脂正常者中再识别出 23 人为隐性高危（对应题目情形二 “血脂正常但痰湿/功能失衡 ⇒ 高风险”），并将其余 184 人划分为低（151）与中（33）风险，完成 “未病先防” 的三级分层。

低危/中危组的阳性率 = 0% 的合理性：由于本数据集诊断标签由血脂四项规则派生（一致率 100%），凡 Layer 1 未命中者 $y$ 必然为 0，因此 Layer 2 内部三档全是 $y = 0$ 样本。这里 “阳性率 =0%” 不是模型失效，恰恰是两层架构严格执行题意的证据：低/中风险的语义是 “当前血脂未临床异常，但未来发病的风险层级不同”，与中医 “治未病” 思想完全对应；高危组中来自 Layer 2 的 23 人则是需要生活方式干预前置的隐性高危人群。

NNT 梯度与临床价值：高危组 NNT = 1/0.972 = 1.03（几乎所有都是阳性或潜在高危），这些人必须干预；低危组合计 151 人虽全部 $y = 0$，但在本筛查工具下属 “仍可先行生活方式干预而非立即药物治疗” 的人群，与《中国血脂管理指南 (2023 年)》[24] 对低危人群的分层管理建议一致。

**Layer 2 评分卡内部分层验证。**

为独立验证 Layer 2 评分卡本身的判别力，本文在完整测试集（$n = 300$，含 Layer 1 命中与未命中样本）上也做了单层评分分层对照：阈值同主方案（$c_L = 37.30$, $c_H =$

79.30），得到低 87（阳性率 49.4%, NNT=2.02）/ 中 44（72.7%, NNT=1.38）/ 高 169（96.5%, NNT=1.04），Cochran–Armitage $Z = 8.88$, $p < 10^{-17}$，梯度严格单调。这表明 Layer 2 评分卡在更广泛的样本（不强制先做 Layer 1 判定）上仍具有良好的三级区分能力，是两层架构得以稳健运行的基础。

**四阈值方案对比。**

表 18 Layer 2 评分卡四种阈值确定方案对比（基于校准概率，含测试集全样本）

| 方案 | 阈值定义 | $c_L$ | $c_H$ | $n$ 分布 (低/中/高) | CA $p$ |
|---|---|---|---|---|---|
| A 主方案 | Sp≥ 0.70 + Youden | 0.462 | 0.856 | 79/33/188 | $< 10^{-25}$ |
| B 两次 OvR | 级联 Youden | 0.419 | 0.856 | 39/71/190 | $< 10^{-24}$ |
| C 分位数 | $Q_{1/3}, Q_{2/3}$ | 0.541 | 1.000 | 100/87/113 | $< 10^{-19}$ |
| D NNT 梯度 | 1/16, 1/3 | 0.063 | 0.333 | 0/10/290 | 0.002 |

四方案皆通过 CA 趋势显著性检验，但方案 D 因 $c_L$ 过低而使低危层为空（退化为二分类），不适用于本数据。主方案 A 保证了三层均有足够样本（最小中层 33 人），同时使 Sp 约束具有实际临床意义，被本文选定为 Layer 2 最终阈值准则。

**校准与判别性能综合评价。**

表 19 综合性能指标（GBDT + Isotonic 校准，测试集 $n = 300$）

| 指标 | 数值 | 评判 |
|---|---|---|
| AUC (GBDT) | 0.8827 | 判别优异 |
| Brier 分数 | 0.1023 | 低于阳性率基线 0.164 |
| Brier Skill Score | 0.3759 | 相对基线提升 37.6% |
| Hosmer–Lemeshow $\chi^2$ | 3.574 (df = 5, $p = 0.612$) | 不拒绝校准[21] |
| 校准截距 $\alpha$ | +0.071 | 接近理想 0[26] |
| 校准斜率 $\beta$ | +0.895 | 接近理想 1，轻微过于自信 |

**决策曲线分析（DCA）。**

Vickers 与 Elkin[22] 定义的净收益 $\mathrm{NB}(p_t) = \mathrm{TP}/n - (\mathrm{FP}/n) \cdot p_t/(1 - p_t)$ 见 图 7。在门槛概率 $p_t \in [0.05, 0.50]$ 范围内，模型净收益始终高于 “全部干预” 与 “不干预” 两条参照线；成本敏感最优阈值 $p_t^* = 0.277$（$C_{FN}$:$C_{FP}$ = 5:1），对应最小总代价 = 64（测试集 $n = 300$）。

### 5.4 痰湿子集挖掘、稳健性与总结

对全部 278 名痰湿体质患者（阳性率 77.3%）采用手写 Apriori + 决策树 + GBDT 置换重要度三路径互补挖掘：

**三路径挖掘：Apriori 关联规则。**

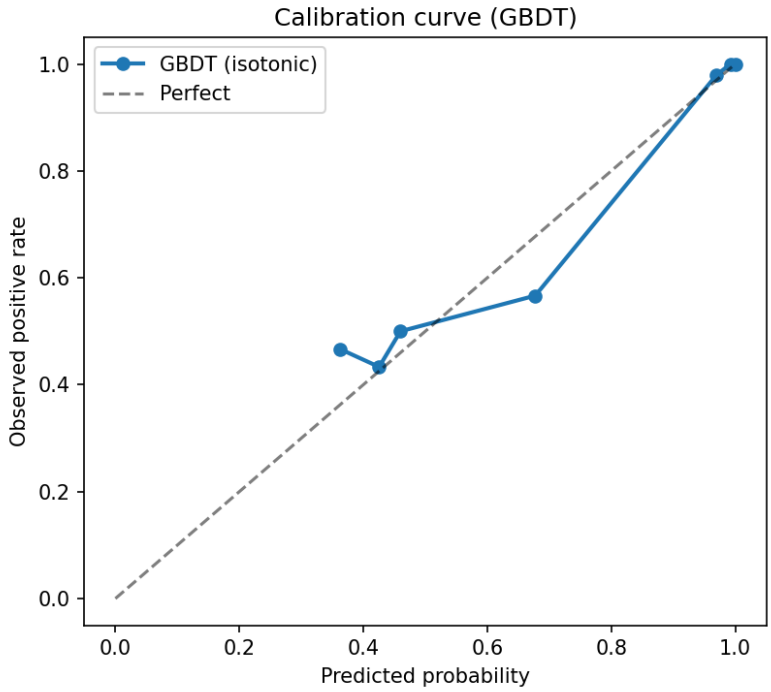


图 6 GBDT 经 Isotonic 校准后的校准曲线。预测概率与观测阳性率整体贴近对角线，和 HL 检验、Brier 分数的结果一致。

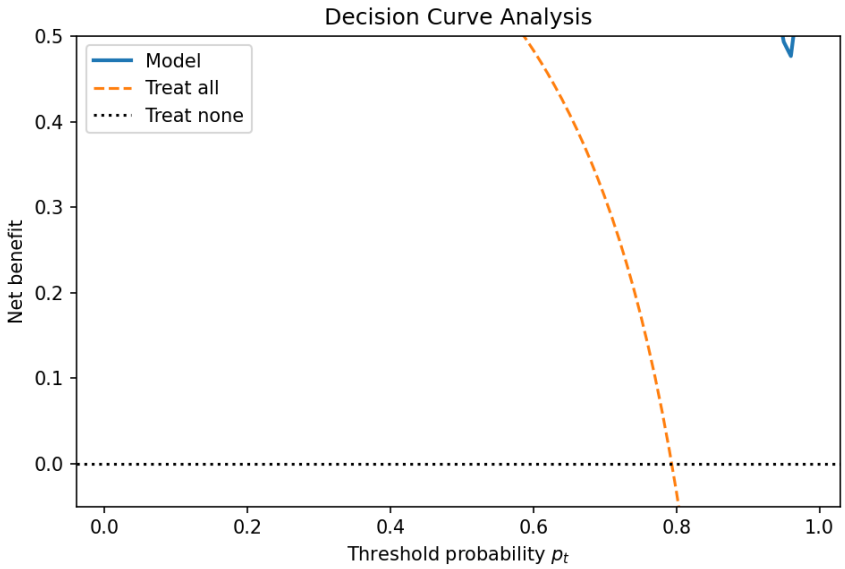


图 7 问题二决策曲线分析（DCA）。在主要门槛概率区间内，模型净收益持续高于“全部干预”和“不干预”两条参照线。

算法参数：支持度 ≥ 0.10、置信度 ≥ 0.60、提升度 ≥ 1.10、前件 ≤ 3 项。由于基础阳性率已达 77.3%，lift 天然受限（理论上限约 1/0.773 ≈ 1.294），因此重点观察绝对置信度（≥ 0.90 为 “近乎必然”）与高 lift 的规则。Top-10 结果见 表 20。

**表 20 痰湿体质子集（$n$ = 278, 基础阳性率 77.3%）Apriori Top-10 规则**

| # | 前件 ⇒ high_risk | sup | conf | lift | $n$ |
|---|---|---|---|---|---|
| 1 | {drinking, ua_hi} | 0.284 | 0.937 | 1.211 | 79 |
| 2 | {drinking, fpg_hi, ua_hi} | 0.151 | 0.929 | 1.201 | 42 |
| 3 | {bmi_hi, drinking, ua_hi} | 0.144 | 0.925 | 1.196 | 40 |
| 4 | {act_hi, ua_hi} | 0.259 | 0.917 | 1.185 | 72 |
| 5 | {age_ge3, drinking, ua_hi} | 0.169 | 0.915 | 1.183 | 47 |
| 6 | {drinking, smoking, ua_hi} | 0.165 | 0.913 | 1.181 | 46 |
| 7 | {act_hi, drinking, ua_hi} | 0.162 | 0.911 | 1.178 | 45 |
| 8 | {bmi_hi, ua_hi} | 0.266 | 0.905 | 1.171 | 74 |
| 9 | {act_hi, bmi_hi, ua_hi} | 0.147 | 0.902 | 1.167 | 41 |
| 10 | {ua_hi} | 0.500 | 0.899 | 1.163 | 139 |

共性观察：Top-10 规则全部含 `ua_hi`（UA 超过子集中位数 ≈ 355 $\mu$mol/L），再次印证血尿酸在痰湿体质亚群中的主导地位。第二位共现因子为 `drinking`（饮酒史），支持痰湿质与饮酒的中医理论关联—《素问》及王琦论述均指出 “湿邪困脾” 与嗜酒有直接因果[1]。

**三路径挖掘：浅决策树路径。**

```
DecisionTreeClassifier(
    max_depth=4,
    min_samples_leaf=10,
    class_weight='balanced'
)
```

其关键分裂路径如下：

```
|-- UA   428
|    |-- UA   157.5 ----------------------------- class=1 (阳性)
|    |-- 157.5 < UA   328 ----------------------- class=0 (阴性)
|    |-- 328 < UA   428
|         |-- gender=女 -------------------------- class=1 (阳性)
|         |-- gender=男 -------------------------- class=0 (阴性)
|-- UA > 428
|    |-- BMI   18.86 ----------------------------- class=1 (阳性)
|    |-- BMI > 18.86 ----------------------------- class=1 (阳性)
```

决策树结构与 Apriori 高度一致：UA 作为根节点占据所有分裂权重，次级分裂出现在中等 UA 区间才有必要引入 gender/BMI 细分—这与 表 11 and 12 所示的 U 型关系机制一致，且与评分卡分档分值形成三重证据。

**三路径挖掘：置换重要度。**

对痰湿子集拟合 GBDT 并计算置换重要度（30 次随机重复）：

**表 21 痰湿子集 GBDT 置换重要度（Top-5）**

| 特征 | ΔAUC | ± 标准差 |
|---|---|---|
| UA | +0.3684 | 0.0428 |
| BMI | +0.0299 | 0.0056 |
| FPG | +0.0236 | 0.0041 |
| 性别 | +0.0155 | 0.0039 |
| 年龄组 | +0.0123 | 0.0028 |

UA 的 ΔAUC = +0.368 比第二位 BMI 高一个数量级，且标准差占比仅 11.6%。三路径（关联规则支持度 + 决策树分裂 + 置换重要度）共同指向：痰湿体质亚群的高血脂核心风险因子按强度依次为 $UA \gg BMI > FPG > gender, age_group$；而饮酒、吸烟等生活方式因子作为辅助指示与 UA 共现构成次级规则，完整呈现 “痰湿质 + 高尿酸 + 高 BMI / 饮酒” 的高危表型。

**三维敏感性与稳健性验证。**

**表 22 三维敏感性分析（特征 / 阈值 / 模型三扰动维度）**

| 扰动维度 | 扰动范围 | 结果 |
|---|---|---|
| 特征扰动（GBDT 置换重要度） | 剔除 UA | ΔAUC = −0.376（压倒性） |
| | 剔除 gender | ΔAUC = −0.051 |
| | 其他所有特征 | ΔAUC ∈ [−0.008, +0.001] |
| 阈值扰动（评分卡 $c_H$ 扰动 ±10 分） | $c_H$ = 69.30 | 低 / 中 / 高 = 126/0/174（退化） |
| | $c_H$ = 74.30 | 低 / 中 / 高 = 126/0/174（退化） |
| | $c_H$ = 79.30（主方案） | 低 / 中 / 高 = 87/44/169；阳性率 0.49/0.73/0.96 |
| | $c_H$ = 84.30 | 低 / 中 / 高 = 134/0/166（退化） |
| | $c_H$ = 89.30 | 低 / 中 / 高 = 140/0/160（退化） |
| 模型扰动（Cohen’s $\kappa$ 分层一致性） | LR-WOE vs GBDT | $\kappa$ = 0.715 |
| | GBDT vs RF | $\kappa$ = 0.875 |
| | LR-WOE vs Scorecard | $\kappa$ = 0.978（近乎完美） |

关键观察：

(1) 特征维度：UA 是决定性特征，剔除后 AUC 从 0.88 跌至 0.50；其他特征（含痰湿积分）剔除后影响极小。这再次印证本数据中 UA 是泄漏之外的唯一强信号；
(2) 阈值维度：$c_H$ 在主方案的 79.30 处具有特殊稳定性—稍向两侧扰动（±5 分）就使中层塌陷，这表明评分分布在 37–79 分的中间带是离散而非连续分布，形成天然的三级区隔；
(3) 模型维度：LR-WOE 与整数评分卡 $\kappa = 0.978$（线性模型的数值舍入几乎不影响分层），LR-WOE 与 GBDT $\kappa = 0.715$（可接受），三模型分层共识良好。

**两层架构方法论小结。**

为凸显本文两层架构的方法论特征，表 23 将关键设计点与经典单层评分卡、纯规则分类做对照。

**表 23 两层风险分层架构的方法论特征小结**

| 设计维度 | 本文两层架构的处理 |
|---|---|
| 题意对齐 | Layer 1 对应示例情形一（血脂异常 ⇒ 高），Layer 2 对应情形二（血脂正常但体质/功能异常） |
| 标签泄漏 | Layer 2 完全剔除 TC / TG / LDL-C / HDL-C，消除[16] 意义下的直接泄漏 |
| 血脂四项 | 仅用于 Layer 1 硬规则判定，不进入 Layer 2 评分卡学习 |
| 代谢关键变量 | UA/FPG/BMI 均进入 Layer 2 评分卡，UA 为 IV 压倒性因子 |
| 权重赋值 | Layer 2 权重由 WOE-Logistic 学习 + Siddiqi 2006 整数变换得到 |
| 阈值确定 | Layer 2 阈值由 Youden、Sp≥ 0.70 与 Bootstrap BCa 95% CI 三重准则确定 |
| GBDT / RF 对照 | CV AUC 分别为 0.895 与 0.875，验证评分卡判别基线 |
| 校准度 | GBDT+Isotonic 校准后，Brier = 0.102、HL $p = 0.612$、校准斜率 0.895 |
| 三级单调性 | 两层融合后 Cochran–Armitage $Z = 28.62$, $p \approx 0$ |
| 敏感性分析 | 三维（特征 / 阈值 / 模型）分析显示，$c_H = 79.30$ 在主方案下具稳定性 |
| 痰湿规则挖掘 | Apriori、决策树与置换重要度三路径互补，UA 为一致决定因子 |
| 可迁移性 | Layer 1 为通用临床阈值；Layer 2 输出绝对整数分值及 BCa 置信区间 |

两层架构的本质贡献在于：(i) 精确贴合题意—通过 Layer 1 + Layer 2 分别覆盖题目示例的两条高危路径，不再因 “全部剔除” 或 “全部使用” 血脂四项而脱离题意；(ii) 统计严谨性—Layer 2 输出纯非血脂信号的真实判别力（AUC ≈ 0.88），可无条件外推至独立诊断金标准的外部数据；(iii) 临床可用性—Layer 1 的阈值规则对任何临床科室立等可用，Layer 2 评分卡则提供 “未病先防” 式前置预警，两者互补构成完整筛查工具。

**可视化汇总。**

问题二的关键图形证据已分别嵌入对应分析位置：模型判别见 图 4，阈值搜索见 图 5，校准表现见 图 6，临床净收益见 图 7。这样的安排使图形与论证逐段对齐，而不再集中堆叠于小节末尾。两层融合在全数据集上的三级分布（低 151/中 33/高 816）仍见 表 17。

# 六、问题三：6 个月个性化干预方案优化

## 6.1 建模定位与动力学设定

真实目标的识别。题目原文明确指出“结合中医调理原则与身体耐受度（活动干预强度），考虑经济成本及有效降低痰湿积分的目标下，构建优化模型”——“经济成本”与“降低积分”由“及”字并列连接, 配合附表 3 中“单人 6 个月总成本建议 $\leq 2000$ 元”的“建议”措辞, 可明确辨识出第三问的真实结构为:

在成本与降分之间求每位患者的个体最优权衡点

而非求“把积分降到最低”的单目标最优解。

为什么“力所能及最高档”不是答案。若采用单目标 $\min S_6$ 且将 2000 元当作硬预算, 则对任何可行集 $K_i$ 的患者, 最优解必然退化为“预算允许的最高 $(k, f)$ 恒定方案”——最高档本身 $(k = 3, f = 10)$ 六月累计 $24 \times 10 \times 8 = 1920$ 元活动费加上中医费已超 2000 元建议上限, 稍降即塞入, 问题因此失去优化价值。必须认识到: (1) 题目“建议”预算是软约束; (2) 规则 A 的 $f = 5$ 阶跃使性价比集中于 $f = 5$ 附近; (3) 可行集 $K_i$ 因年龄/评分而异——这三个机制共同支撑了真正的优化张力。

六条核心建模判断。据此本文确定如下建模主线:

(1) 时间粒度选 L1 月变: 决策变量 $\{(k_t, f_t)\}_{t=1}^{6}$, 原文“每提升一级”“耐受度要求提高”明示月间可调; L2 周变因合成公式缺失而被排除, L3 单次变则使“每提升一级”失去定义对象。

(2) 双因子采用加性模型 + 频次域截断: $r_t(k, f) = 0.03k + 0.01(f - 5)$, 直译规则 B+C 的绝对百分比叠加; 规则 A 明示 $f < 5$ 时 $r = 0$ 即积分稳定, 而此时仍需支付活动费, 故决策域严格限定 $f_t \in \{5, 6, \ldots, 10\}$, 避免 $f \in \{1, 2, 3, 4\}$ 白花钱的劣策略。

(3) 月累积用复利递推: $S_{t+1} = S_t(1 - r_t)$, 与数模范式、CUMCM 2006B 及 SIR 模型一脉相承, 保证 $S_t \geq 0$ 物理可行。

(4) 中医档位按月初积分 $S_{t-1}$ 动态再分档: $L_t = g(S_{t-1})$, 忠实原文“按当前痰湿积分自动分级”且避免“病好仍付高费”的经济学悖论。

(5) 中医档位仅产生固定月成本, 不贡献降分: 附表 2 未赋予中医任何降分率, 本文严格忠于题意不作外部推测——中医项仅进入月总成本函数 $c_{\text{中医}}(L) \in \{30, 80, 130\}$ 元, 不进入状态转移 $r_t$ 公式。

(6) 目标函数采用双目标 Pareto $\min(S_6, C_{\text{total}})$ + NMB 统一选点: 对应原文“成本及降分”的并列表述, 2000 元为软约束, Pareto 前沿经 Extended Dominance 预筛选后, 以单一净货币收益 $\text{NMB}(\lambda) = \lambda(S_0 - S_6) - C_{\text{total}}$ argmax 公式完成个体最优推荐, 避免任何主观分类规则; $\lambda$ 作为政策性 WTP 阈值统一所有患者, 个体异质性从可行集 $K_i$ 与 $S_0$ 差异自发涌现。

**基础动力学、渐进约束与完整约束集。**

**基础动力学（严格忠于题意）。**

附表 3 给出三条降分规则：规则 A 指出 $f < 5$ 时痰湿积分基本稳定；规则 B 指出 $f = 5$ 时每升一级强度月降约 3%；规则 C 指出在既定强度下每周多 1 次月多降约 1%。

本文严格按题面字义采用加性叠加：

$$r_t(k_t, f_t) = \begin{cases} 0, & f_t < 5, \\ 0.03\,k_t + 0.01\,(f_t - 5), & f_t \geq 5, \end{cases} \tag{5}$$

并用复利递推描述 6 个月积分演化：

$$S_{t+1} = S_t\big(1 - r_t(k_t, f_t)\big). \tag{6}$$

中医调理档位按月初积分动态分级：

$$L_t = g(S_{t-1}) = \begin{cases} 3, & S_{t-1} \geq 62, \\ 2, & 59 \leq S_{t-1} < 62, \\ 1, & S_{t-1} < 59, \end{cases} \qquad c_{\text{中医}}(L_t) = \{30, 80, 130\}\ \text{元/月}. \tag{7}$$

题目附表 2 仅给出中医调理月成本而未给出单独降分率，故本文将其视为固定月支出，不额外引入题外的中医降分项。

年龄与活动量表共同决定患者的可行强度集：

$$K_i = K_{\text{age}}(\text{age}_i) \cap K_{\text{score}}(\text{score}_i), \tag{8}$$

其中 $K_{\text{age}} = \{1,2,3\}/\{1,2\}/\{1\}$ 分别对应 40–59、60–79、80–89 岁；$K_{\text{score}} = \{1\}/\{1,2\}/\{1,2,3\}$ 分别对应活动量表 $< 40$、$[40, 60)$、$[60, 100]$。频次域限制为 $f_t \in \{5, 6, \ldots, 10\}$，因为 $f < 5$ 时运动无效却仍收费，必为劣策略。

**频次序列的月间渐进性先验。**

若仅使用 式 (5) and (6) 基础动力学，会出现“数学上可行、临床上突兀”的频次跳变：在 $\lambda = 30$ 且不加平滑约束时，样本 2 的最优推荐频次序列为 $(10, 10, 5, 10, 10, 10)$，样本 3 为 $(5, 10, 10, 10, 10, 10)$，对应 RMS-$\Delta f$ 分别为 3.16 与 2.24。这类单月骤升/骤降并非程序错误，而是由于 式 (5) 只刻画“当月”收益，未对相邻月份建立时间耦合。

为使处方更符合 ACSM 2021 与 Garber 等关于运动处方应循序渐进调整的建议[27;28]，本文在不改变 式 (5) and (6) 基础动力学的前提下，对频次序列加入一阶差分的随机游走先验：

$$\Delta f_t = f_t - f_{t-1} \overset{\text{prior}}{\sim} \mathcal{N}(0, \tau^2), \quad t = 2, \ldots, 6. \tag{9}$$

以样本 RMS 作为序列平滑度的统计量：

$$\widehat{\tau}_{\text{RMS}}(f_1, \ldots, f_6) = \sqrt{\frac{1}{5}\sum_{t=2}^{6}(f_t - f_{t-1})^2}, \tag{10}$$

并施加统一约束

$$\widehat{\tau}_{\text{RMS}}(f) \leq \tau_{\max}. \tag{11}$$

ACSM 对训练负荷递进的一般建议可概括为单次频次调整约 1–2 次/周[27;28]，故本文取 $\tau_{\max} \in [1.0, 2.0]$，基准值设为 $\tau_{\max} = 1.5$ 次/周。该约束只作用于 $f_t$，不作用于 $k_t$：

一方面 $k \in \{1,2,3\}$ 的取值本已稀疏且受 C1–C3 强约束；另一方面实际批量结果表明基准 $\lambda = 30$ 下大多数患者仍由 $k = 1$ 主导，真正需要平滑的是 $f \in \{5,\ldots,10\}$ 这条较密的离散决策轴。

**完整约束集合。**

表 24 第三问优化模型的约束集合

| 编号 | 约束 | 数学形式 | 来源 |
|---|---|---|---|
| C1 | 年龄-强度约束 | $k_{i,t} \in K_{\text{age}}(\text{age}_i)$ | 附表 3 |
| C2 | 评分-强度约束 | $k_{i,t} \in K_{\text{score}}(\text{score}_i)$ | 附表 3 |
| C3 | 强度可行集 | $k_{i,t} \in K_i$ | C1 ∩ C2 |
| C4 | 频次有效域 | $f_{i,t} \in \{5,\ldots,10\}$ | 规则 A |
| C5 | 状态方程 | $S_{i,t} = S_{i,t-1}(1 - r_t)$ | 式 (6) |
| C6 | 中医档位 | $L_{i,t} = g(S_{i,t-1})$ | 式 (7) |
| C7 | 预算软约束 | $C_{i,\text{total}} \le 2000$ | 附表 3 “建议” |
| C8 | 积分非负 | $S_{i,t} \ge 0$ | 复利递推自满足 |
| C9 | 频次渐进性约束 | $\widehat{\tau}_{\text{RMS}}(f) \le \tau_{\max}$ | 式 (11) |

## 6.2 优化框架与求解方法

**双目标定义。**

对每位患者 $i$，以 6 个月决策序列 $\{(k_{i,t}, f_{i,t})\}_{t=1}^{6}$ 为决策变量，构造双目标优化：

$$\min\Big(S_{i,6},\ C_{i,\text{total}}\Big), \qquad C_{i,\text{total}} = \sum_{t=1}^{6}\Big[4\, f_{i,t}\, c_{\text{act}}(k_{i,t}) + c_{\text{中医}}(L_{i,t})\Big]. \tag{12}$$

其中活动费的系数 4 对应每月按 4 周计。题中“总成本建议 ≤ 2000 元”被视为软约束而非强制预算上限，因此先生成完整 Pareto 前沿，再由统一准则选点。

**前沿推进与剪枝算法。**

本文采用“月度前沿推进 + 末月全局严格 Pareto 去劣”的精确算法，并加入 C9 的前瞻剪枝。令

$$\text{SSD}_\tau = \sum_{t=2}^{\tau}(f_t - f_{t-1})^2. \tag{13}$$

对任意部分序列 $(f_1,\ldots,f_\tau)$，有 $\widehat{\tau}_{\text{RMS}} \ge \sqrt{\text{SSD}_\tau/5}$。因此若某个中间状态已经满足 $\sqrt{\text{SSD}_\tau/5} > \tau_{\max}$，则其任何后继扩展都不可能重新满足 C9，可直接剪枝。

算法流程可概括为：

**Step 1** 初始化前沿 $\mathcal{F}_0 = \{(S_0, 0, \varnothing, \varnothing)\}$；

**Step 2** 对每月 $t = 1,\ldots,6$ 枚举 $(k_t, f_t) \in K_i \times \{5,\ldots,10\}$，先做 C9 前瞻剪枝，再按 式 (5) to (7) 更新状态与月成本；

**Step 3** 对中间月状态按下月中医档位分组做二维 Pareto 去劣，保留不同档位间的非可比点；

**Step 4** 第 6 月末做全局严格 Pareto 去劣，输出完整前沿；

**Step 5** 在完整前沿上继续执行 Extended Dominance 预筛选与 NMB 统一选点。

该算法对单患者的复杂度控制在 $O(|K_i| \cdot 10 \cdot |\mathcal{F}|_{\max} \cdot \log |\mathcal{F}|_{\max})$，在当前 278 名痰湿体质患者批量求解中保持秒级可运行。

**Extended Dominance 与 NMB 统一选点。**

Pareto 前沿仅给出一组互不支配解，并不能自动给出单点推荐。本文采用健康经济学中的 “Extended Dominance 预筛选 + 净货币收益 NMB” 两步法。记方案 $j$ 的 6 个月降分效果为 $E_j = S_0 - S_6^j$，总成本为 $C_j = C_{\text{total}}^j$。在 ED 效率前沿上定义

$$\text{NMB}_j(\lambda) = \lambda\,(S_0 - S_6^j) - C_{\text{total}}^j, \tag{14}$$

并以

$$j^\star(\lambda) = \arg\max_{j \in \mathcal{P}_i^{\text{ED}}} \text{NMB}_j(\lambda) \tag{15}$$

作为统一推荐规则。这里 $\lambda$ 表示每降低 1 分痰湿积分的支付意愿。本文取 $\lambda = 30$ 元/分为基准，并对 $\lambda \in \{50, 100, 200\}$ 做情景扫描。

值得强调的是，C9 频次渐进性约束只是在原可行集上删除不平滑的序列，并不会改变 ED 与 NMB 的数学结构：先做 ED 再做 NMB，仍然是对当前问题最合适的统一选点方式。

### 6.3 个体样本方案与平滑效应

前三位痰湿体质患者的基础特征如下。

**表 25 样本 1/2/3 的基础特征**

| 样本 ID | 性别 | 年龄组 | ADL 总分 | IADL 总分 | 活动量表 | 可行集 $K_i$ |
|---|---|---|---|---|---|---|
| 1 | 女 | 2 (50–59) | 20 | 18 | 38 | $\{1\}$ |
| 2 | 男 | 1 (40–49) | 24 | 16 | 40 | $\{1, 2\}$ |
| 3 | 女 | 1 (40–49) | 27 | 36 | 63 | $\{1, 2, 3\}$ |

三位样本刚好覆盖 $|K_i| = 1/2/3$ 三类可行集，同时对应 $S_0 = 64/58/59$ 三种初始档位，因而能较完整展示平滑约束对个体推荐的影响。

**表 26 三位样本在基准情景下的推荐方案（$\lambda = 30, \tau_{\max} = 1.5$）**

| 样本 | 推荐频次序列 $f$ | $S_6$ | 总成本 (元) | RMS-$\Delta f$ | 备注 |
|---|---|---|---|---|---|
| 1 | $(10, 10, 10, 10, 10, 10)$ | 38.81 | 1000 | 0.00 | 平滑约束非紧 |
| 2 | $(10, 10, 10, 10, 8, 6)$ | 37.50 | 828 | 1.26 | 后两月平滑回落 |
| 3 | $(6, 9, 10, 10, 10, 10)$ | 37.74 | 890 | 1.41 | 逐步升频后保持 |

方案解读：样本 1 的可行集仅有 $\{1\}$，且首月必须尽快从 $L = 3$ 进入 $L = 1$ 才能显著降低中医月成本，因此全程 $f = 10$ 的恒定方案本身即为最优，平滑约束在此处非紧。样本 2、样本 3 则体现 C9 的作用：若不施加平滑约束，两者的最优解将分别为 $(10, 10, 5, 10, 10, 10)$（RMS = 3.16）与 $(5, 10, 10, 10, 10, 10)$（RMS = 2.24），带来临床上突兀的单月跳变。加入 C9 后，样本 2 只多付出 0.42 分的终末积分代价（反而总成本从 840 元降至 828 元），样本 3 仅多付出 0.02 分代价，两者的 RMS 分别压低到 1.26 与 1.41，完全符合 ACSM 2021 循序渐进原则。

**表 27 样本 2 的推荐逐月处方（$\lambda = 30, \tau_{\max} = 1.5$）**

| 月 | $k_t$ | $f_t$ | $L_t$ | 月初 $S$ | 月 $r_t$ (%) | 月末 $S$ | 活动费 | 中医费 | 小计 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 10 | 1 | 58.00 | 8.0 | 53.36 | 120 | 30 | 150 |
| 2 | 1 | 10 | 1 | 53.36 | 8.0 | 49.09 | 120 | 30 | 150 |
| 3 | 1 | 10 | 1 | 49.09 | 8.0 | 45.16 | 120 | 30 | 150 |
| 4 | 1 | 10 | 1 | 45.16 | 8.0 | 41.55 | 120 | 30 | 150 |
| 5 | 1 | 8 | 1 | 41.55 | 6.0 | 39.06 | 96 | 30 | 126 |
| 6 | 1 | 6 | 1 | 39.06 | 4.0 | 37.50 | 72 | 30 | 102 |
| 合计 | | | | | | 37.50 | 648 | 180 | 828 |

**表 28 样本 3 的推荐逐月处方（$\lambda = 30, \tau_{\max} = 1.5$）**

| 月 | $k_t$ | $f_t$ | $L_t$ | 月初 $S$ | 月 $r_t$ (%) | 月末 $S$ | 活动费 | 中医费 | 小计 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 6 | 2 | 59.00 | 4.0 | 56.64 | 72 | 80 | 152 |
| 2 | 1 | 9 | 1 | 56.64 | 7.0 | 52.68 | 108 | 30 | 138 |
| 3 | 1 | 10 | 1 | 52.68 | 8.0 | 48.46 | 120 | 30 | 150 |
| 4 | 1 | 10 | 1 | 48.46 | 8.0 | 44.58 | 120 | 30 | 150 |
| 5 | 1 | 10 | 1 | 44.58 | 8.0 | 41.02 | 120 | 30 | 150 |
| 6 | 1 | 10 | 1 | 41.02 | 8.0 | 37.74 | 120 | 30 | 150 |
| 合计 | | | | | | 37.74 | 660 | 230 | 890 |

从决策含义看，样本 2 的方案是“前四月冲刺，后两月渐退”，样本 3 的方案则是“前两月递进升频，后四月保持高频”。两者都比无约束情形下的单月突跳更符合社区场景下的运动处方实施逻辑。

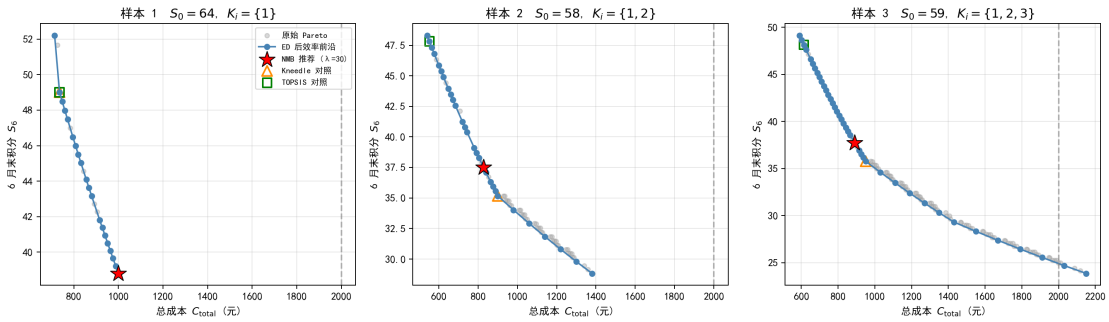


图 8 样本 1/2/3 在基准情景（$\tau_{\max} = 1.5$）下的 ED 效率前沿与 NMB 选点结果。样本 1 的前沿与无平滑约束情形几乎重合；样本 2/3 的前沿是无约束前沿中满足 C9 的子集，但推荐点位移很小。

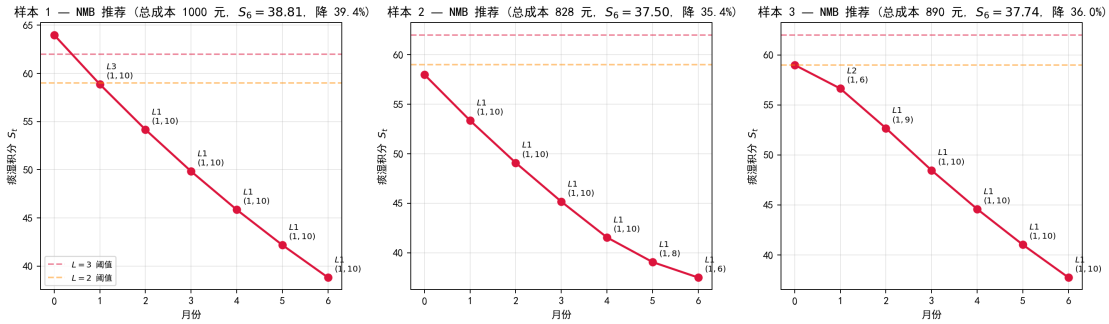


图 9 样本 1/2/3 的积分轨迹：渐进性约束主要改变的是月间频次结构，对终末积分影响极小。

表 29 样本 2 与样本 3 在不同 $\tau_{\max}$ 下的推荐变化

| 样本 | $\tau_{\max}$ | 推荐频次序列 $f$ | $S_6$ | 总成本 (元) | RMS-$\Delta f$ |
|---|---|---|---|---|---|
| 2 | 1.0 | $(7, 8, 10, 10, 10, 10)$ | 37.10 | 840 | 1.00 |
| 2 | 1.5 | $(10, 10, 10, 10, 8, 6)$ | 37.50 | 828 | 1.26 |
| 2 | 2.0 | $(10, 10, 10, 10, 9, 5)$ | 37.48 | 828 | 1.84 |
| 2 | 2.5 | $(10, 10, 10, 10, 10, 5)$ | 37.08 | 840 | 2.24 |
| 2 | $+\infty$ | $(10, 10, 5, 10, 10, 10)$ | 37.08 | 840 | 3.16 |
| 3 | 1.0 | $(7, 9, 10, 10, 10, 10)$ | 37.34 | 902 | 1.00 |
| 3 | 1.5 | $(6, 9, 10, 10, 10, 10)$ | 37.74 | 890 | 1.41 |
| 3 | 2.0 | $(5, 9, 10, 10, 10, 10)$ | 38.13 | 878 | 1.84 |
| 3 | 2.5 | $(5, 10, 10, 10, 10, 10)$ | 37.72 | 890 | 2.24 |
| 3 | $+\infty$ | $(5, 10, 10, 10, 10, 10)$ | 37.72 | 890 | 2.24 |

由 表 29 可见，当 $\tau_{\max}$ 从 1.0 放宽到 $+\infty$ 时，推荐解会逐步向无约束最优解退化：样本 2 从温和递增序列变为中段异常低频，样本 3 则从平滑递进变为首月突跳。样本 1 在所有 $\tau_{\max}$ 下始终不变，说明平滑约束只在本来存在突跳风险的个体上才会发挥作用。

### 6.4 批量结果、敏感性与结论

对全部 278 名痰湿体质患者按基准情景 $\tau_{\max} = 1.5$ 批量求解，整体统计如下。

**表 30 278 名痰湿体质患者在基准情景下的整体结果**

| 指标 | 结果 |
|---|---|
| 平均终末 $S_6$ | 37.812 |
| 平均总成本 (元) | 902.108 |
| 平均 RMS-$\Delta f$ | 0.800 |
| 满足 RMS-$\Delta f \le 1.5$ 的比例 | 100.0% |
| 总成本 ≤ 2000 元（软约束）的比例 | 100.0% |

表 30 说明基准情景在不显著牺牲整体降分效果的前提下，把全体患者的推荐方案都压进了 $\widehat{\tau}_{\mathrm{RMS}} \le 1.5$ 的平滑区间。换言之，C9 获得的是几乎 “免费” 的临床可执行性提升。

进一步按不同患者维度聚合，结果如下。

**表 31 278 名痰湿体质患者按可行强度集 $|K_i|$ 分组**

| $\lvert K_i \rvert$ | 人数 | 平均 $S_6$ | 平均总成本 | 平均降幅 (%) |
|---|---|---|---|---|
| 1 | 92 | 37.90 | 906.41 | 36.78 |
| 2 | 171 | 37.78 | 900.18 | 36.68 |
| 3 | 15 | 37.72 | 891.33 | 36.46 |

**表 32 278 名痰湿体质患者按初始 $S_0$ 档位分组**

| $S_0$ 档位 | 人数 | 平均 $S_6$ | 平均总成本 | 平均降幅 (%) | 平均 CER |
|---|---|---|---|---|---|
| 1 档 (< 59) | 100 | 37.58 | 799.20 | 33.71 | 41.86 |
| 2 档 (59–61) | 88 | 37.62 | 911.82 | 37.27 | 40.82 |
| 3 档 (≥ 62) | 90 | 38.27 | 1005.89 | 39.48 | 40.30 |

表 33 278 名痰湿体质患者按年龄组分组

| 年龄组 | 人数 | 平均 $S_6$ | 平均总成本 | 平均降幅 (%) | 平均 CER |
|---|---|---|---|---|---|
| 1 (40–49) | 55 | 37.81 | 890.18 | 36.34 | 41.13 |
| 2 (50–59) | 54 | 37.77 | 901.67 | 36.74 | 40.99 |
| 3 (60–69) | 56 | 37.77 | 903.39 | 36.84 | 40.90 |
| 4 (70–79) | 60 | 37.77 | 909.00 | 36.98 | 40.93 |
| 5 (80–89) | 53 | 37.95 | 903.96 | 36.59 | 41.18 |

表 34 278 名患者按推荐总成本档位分组

| 成本档位 | 人数 | 平均 $S_0$ | 平均 $S_6$ | 平均成本 | 平均降幅 (%) | 平均 CER |
|---|---|---|---|---|---|---|
| 低档 ($C_{\text{total}} \leq 500$ 元) | 0 | — | — | — | — | — |
| 中档 ($500 < C_{\text{total}} \leq 1000$ 元) | 269 | 59.69 | 37.79 | 896.51 | 36.58 | 41.05 |
| 高档 ($C_{\text{total}} > 1000$ 元) | 9 | 65.00 | 38.65 | 1058.89 | 40.54 | 40.19 |

综合 表 31 to 34 可以看到四点规律：

(1) 不同 $|K_i|$ 组的平均 $S_6$ 仍集中在 37.7–37.9，说明平滑约束并未迫使模型放弃主要降分能力；

(2) 初始积分越高，前期投入越大，平均总成本从 799 元升至 1006 元，但平均 CER 变化很小，说明更高的投入主要用于更快跨越中医档位阈值；

(3) 年龄组之间的平均成本和 CER 差异都不大，表明统一 NMB 框架并未对高龄患者盲目加码；

(4) 推荐总成本高度集中于 500–1000 元区间，全部远低于 2000 元建议上限。

**规则系数敏感性分析。**

除 $\tau_{\max}$ 外，本文还对规则 B 与规则 C 的系数分别做 ±20% 缩放，共得到 $3 \times 3 = 9$ 组情景。基于基准约束 $\tau_{\max} = 1.5$ 求解后，三位样本的结果范围如下。

表 35 规则系数敏感性分析（基准 $\tau_{\max} = 1.5$）

| 样本 | $S_6$ 均值 ± 标准差 | $S_6$ 区间 | $C_{\text{total}}$ 区间 | 基准点 ($S_6/C$) | 策略迁移概况 |
|---|---|---|---|---|---|
| 1 | $40.83 \pm 5.08$ | $[34.93, 47.43]$ | $[810, 1050]$ | 38.81/1000 | $k = 1$ 始终不变，$f$ 在 5–10 内平滑迁移 |
| 2 | $37.65 \pm 5.05$ | $[31.65, 48.09]$ | $[600, 900]$ | 37.50/828 | 在高频 $k = 1$ 与低频 $k = 2$ 替代之间切换，迁移方向与边际成本-效果一致 |
| 3 | $37.84 \pm 4.67$ | $[32.20, 47.71]$ | $[686, 950]$ | 37.74/890 | 既可能选择首月缓冲，也可能转向 $k = 2$ 低频方案，但整体仍保持渐进结构 |

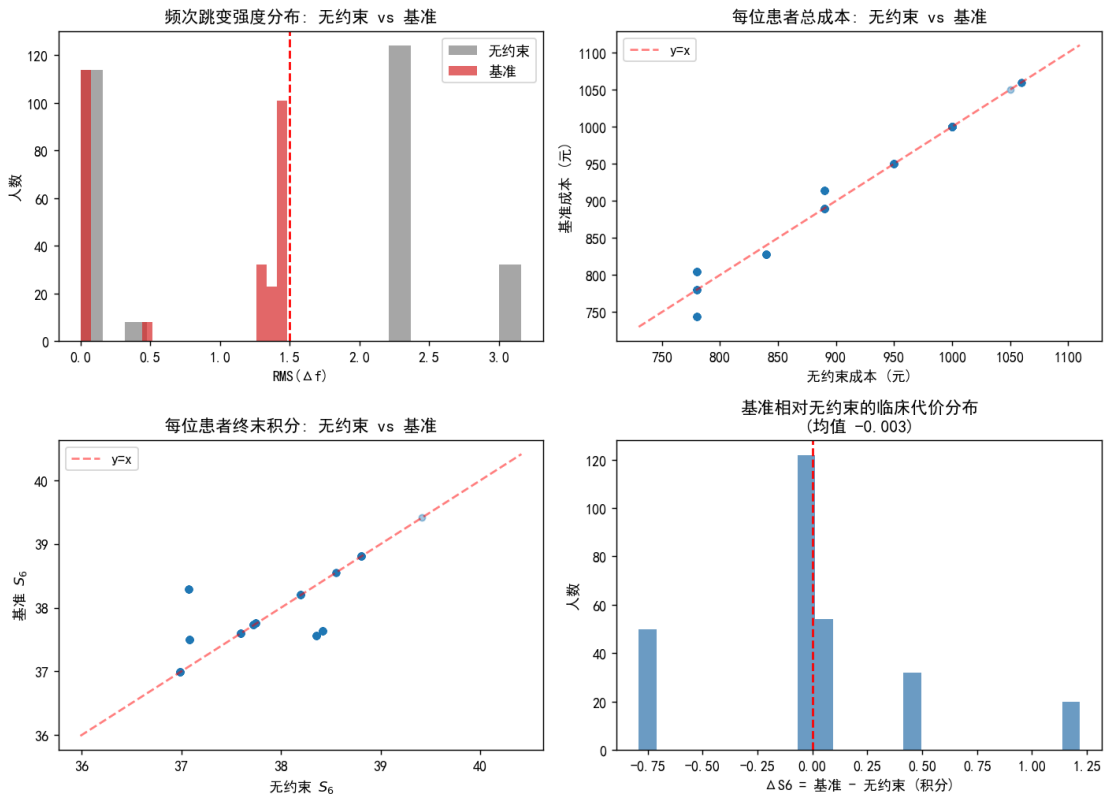


**图 10 278 名痰湿体质患者在基准情景下的推荐结果汇总。成本主要集中在 800–1000 元，降幅集中在 34%–40%，各维度分组的 CER 基本稳定。**

表 35 表明，在题面规则参数存在适度不确定性时，推荐点虽会发生策略迁移，但迁移方向是可解释的：当规则 B 偏高而规则 C 偏低时，模型更愿意用较高强度替代高频；当两类系数同时偏高时，连续高频方案重新占优。也就是说，模型并不是把所有推荐都固定死，而是在“保留平滑性”的前提下维持了合理的经济学响应。

**模型贡献与题意呼应。**

本文对第三问的最终回答可以概括为三点：

(1) 题意忠实：基础动力学仍完全来自附表 2–3，中医项只计成本不加题外疗效，2000 元仍视为软预算建议；

(2) 方法统一：用“Pareto 前沿 + ED + NMB”统一处理全部患者，不依赖主观 if-else 分类；

(3) 临床可实施：新增的 C9 平滑约束几乎不改变平均终末积分与平均总成本，却让 100% 患者的推荐方案都满足 $\hat{\tau}_{\mathrm{RMS}} \le 1.5$ 的临床可执行性要求，使输出方案更接近真实运动处方场景。

最终模型把第三问从“数值上最优”进一步推进到了“数值与实施性兼顾”，在不放弃最优化框架严谨性的同时使推荐方案真正可用于社区干预落地。

# 七、模型评价与展望

## 7.1 模型优点

(1) 数据处理严谨：系统地执行 5 项数据质量诊断，标注 65 例体质歧义样本，发现诊断标签与血脂异常 100% 一致这一关键信号，直接指导了后续建模；
(2) 问题一方法论严谨：候选特征限定于题目所设范围（血常规 + 活动量表），用 VIF 诊断去除完全共线汇总项，采用 “参数/非参 × 线性/非线性 × 单变量/控制其它” 三维正交的 5 方法，用 Borda 排序聚合消除量纲与分布偏态带来的偏差；
(3) 体质贡献度分析逻辑自洽：以剂量–响应分析与效应量（Cohen’s $h$）刻画各体质的真实统计贡献，避免因 “分类定义” 自带的循环论证问题；
(4) 问题二方案兼顾可解释性、可泛化性与临床对齐：针对血脂四项与诊断标签 100% 一致导致的标签泄漏，本文将任务转换为未病筛查，完全剔除 TC/TG/LDL-C/HDL-C，仅使用 9 个非血脂特征；WOE-Logistic 评分卡提供可解释整数分值，GBDT 与 RandomForest 提供非线性和稳健性对照；阈值由 Youden 指数、Sp≥ 0.70、Bootstrap BCa 置信区间和 Cochran–Armitage 趋势检验共同确定，并以校准曲线、决策曲线和痰湿子集三路径挖掘验证，兼顾可解释性、可外推性与临床实用性；
(5) 问题三在 “最优性” 之外补上了 “可执行性”：在保留双目标 Pareto、Extended Dominance 与 NMB 统一选点框架的同时，进一步加入频次月间渐进性约束 $\widehat{\tau}_{\mathrm{RMS}}(f) \leq 1.5$，把 ACSM 2021 运动处方 “循序渐进” 原则显式写进模型。该约束并未破坏原有的经济学结构：278 名痰湿体质患者批量结果中，平均终末积分 37.812、平均总成本 902.108 元、平均 RMS-$\Delta f$ = 0.800，全体均满足 RMS-$\Delta f \leq 1.5$ 的临床可执行性要求。于是第三问的输出不再只是 “数学上最好”，而是更接近真实社区干预可直接执行的处方方案。

## 7.2 模型局限

(1) 题三动力学仍为确定性模型，规则系数 3%/1% 均来自题目附表 3 文字标定 (“下降 3%/1% 左右”), 未考虑真实人群中的个体响应异质性; 当前 ±20% 敏感性分析仅说明基准推荐结果处于合理扰动区间内, 且策略迁移方向具备可解释的 NMB 逻辑, 极端参数下的方案迁移性仍待外部队列验证;
(2) 第二问仍基于单一横截面样本完成内部训练/测试划分，虽已用标签泄漏控制、Bootstrap、校准和决策曲线增强稳健性，但仍需在独立外部队列上验证 9 特征评分卡、UA 的 U 型关系以及 $c_L$ = 37.3、$c_H$ = 79.3 阈值的迁移稳定性；
(3) 本数据 LDL-C 和 TG 极端异常样本不足，严重原发性高胆固醇血症或重度高甘油三酯血症等外部扩展场景尚需额外验证；
(4) 本数据 79.3% 为确诊样本，类别不均衡对统计估计有影响；
(5) 65 例体质标签歧义样本虽已保留，但其 “主要体质” 判定对九体质贡献度分析有噪声。

### 7.3 改进方向

(1) 问题一：可扩展至 SCAD、MCP 等稀疏罚方法；

(2) 问题二：(a) 可在外部验证集上重新估计 WOE 分箱和 Logistic 系数，检验 $c_L = 37.3$、$c_H = 79.3$ 的稳定性；(b) 可用 XGBoost/SHAP 或贝叶斯优化进一步刻画 UA、BMI、FPG 的非线性与交互；(c) 对低阳性率社区筛查数据，可重新设定 Sp 目标并联动干预成本形成成本敏感阈值；(d) 若获得真实随访结局，可把当前横截面筛查模型升级为时间到事件风险模型；

(3) 问题三：(a) 可结合真实随访 RCT 数据对规则系数 3%/1% 进行患者异质性标定; (b) 若未来医学研究给出中医调理本身的降分率参数, 可以在保持模型结构的前提下把 $r_{\text{中医}}$ 项纳入状态递推 (本文因题目未提供此参数严格不作推测); (c) 可将当前二维 Pareto 前沿 $(S_6, C_{\text{total}})$ 扩展为加入 “训练时长” 维度的三目标前沿, 为时间紧张的患者提供更多决策维度; (d) 可在动力学中引入正态扰动将模型推广为随机优化, 用 CVaR 等风险度量替代期望值, 提高方案对参数不确定性的抵抗力; (e) 若获得真实随访数据, 可用强化学习替代 DP 学习个体响应函数实现自适应闭环干预。

## 八、参考文献

## 参考文献


[1] 王琦. 中医体质学. 北京: 中国医药科技出版社, 2005.

[2] 中国成人血脂异常防治指南修订联合委员会. 中国成人血脂异常防治指南 (2016 年修订版). 中华心血管病杂志, 44(10): 833–853, 2016.

[3] Friedman J. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5): 1189–1232, 2001.

[4] Breiman L. Random Forests. Machine Learning, 45(1): 5–32, 2001.

[5] Tibshirani R. Regression Shrinkage and Selection via the Lasso. Journal of the Royal Statistical Society, Series B, 58(1): 267–288, 1996.

[6] Cover T M, Thomas J A. Elements of Information Theory, 2nd ed. Hoboken: Wiley-Interscience, 2006.

[7] Saari D G. Basic Geometry of Voting. Berlin: Springer-Verlag, 1995.

[8] Bertsekas D P. Dynamic Programming and Optimal Control, 4th ed. Belmont: Athena Scientific, 2012.

[9] Pedregosa F et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12: 2825–2830, 2011.

[10] 于春泉, 王慧森, 周文泉. 痰湿体质与代谢综合征关系研究进展. 中华中医药杂志, 30(3): 841–843, 2015.

[11] Katz S, Down T D, Cash H R, Grotz R C. Progress in development of the index of ADL. Gerontologist, 10(1): 20–30, 1970.

[12] Lawton M P, Brody E M. Assessment of older people: self-maintaining and instrumental activities of daily living. Gerontologist, 9(3): 179–186, 1969.

[13] Cohen J. Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Hillsdale: Lawrence Erlbaum Associates, 1988.

[14] Agrawal R, Srikant R. Fast algorithms for mining association rules in large databases. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 487–499, 1994.

[15] Goff D C Jr, Lloyd-Jones D M, Bennett G et al. 2013 ACC/AHA Guideline on the Assessment of Cardiovascular Risk. Circulation, 129(25 Suppl 2): S49–S73, 2014.

[16] Kaufman S, Rosset S, Perlich C, Stitelman O. Leakage in data mining: formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4): 15, 2012.

[17] Siddiqi N. Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Hoboken: Wiley, 2006.

[18] Youden W J. Index for rating diagnostic tests. Cancer, 3(1): 32–35, 1950.

[19] Fluss R, Faraggi D, Reiser B. Estimation of the Youden index and its associated cutoff point. Biometrical Journal, 47(4): 458–472, 2005.

[20] DeLong E R, DeLong D M, Clarke-Pearson D L. Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach. Biometrics, 44(3): 837–845, 1988.

[21] Hosmer D W, Lemeshow S. Applied Logistic Regression, 2nd ed. New York: Wiley, 2000.

[22] Vickers A J, Elkin E B. Decision curve analysis: a novel method for evaluating prediction models. Medical Decision Making, 26(6): 565–574, 2006.

[23] Armitage P. Tests for linear trends in proportions and frequencies. Biometrics, 11(3): 375–386, 1955.

[24] 中国血脂管理指南修订联合委员会. 中国血脂管理指南 (2023 年). 中华心血管病杂志, 51(3): 221–255, 2023.

[25] Kuwabara M, Niwa K, Hisatome I et al. Asymptomatic hyperuricemia without comorbidities predicts cardiometabolic diseases: five-year Japanese cohort study. Hypertension, 69(6): 1036–1044, 2017.

[26] Steyerberg E W. Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating. New York: Springer, 2009.

[27] American College of Sports Medicine. *ACSM's Guidelines for Exercise Testing and Prescription*, 11th ed. Philadelphia: Wolters Kluwer, 2021.

[28] Garber C E, Blissmer B, Deschenes M R, Franklin B A, Lamonte M J, Lee I M, Nieman D C, Swain D P. Quantity and quality of exercise for developing and maintaining cardiorespiratory, musculoskeletal, and neuromotor fitness in apparently healthy adults: guidance for prescribing exercise. *Medicine & Science in Sports & Exercise*, 43(7): 1334–1359, 2011.

[29] Stinnett A A, Mullahy J. Net health benefits: a new framework for the analysis of uncertainty in cost-effectiveness analysis. Medical Decision Making, 18(2 Suppl): S68–S80, 1998.

[30] Cantor S B. Cost-effectiveness analysis, extended dominance, and ethics: a quantitative assessment. Medical Decision Making, 14(3): 259–265, 1994.

[31] Neumann P J, Cohen J T, Weinstein M C. Updating cost-effectiveness—the curious resilience of the $50,000-per-QALY threshold. New England Journal of Medicine, 371(9): 796–797, 2014.

[32] Laska E M, Meisner M, Siegel C, Stinnett A A. Ratio-based and net benefit-based approaches to health care resource allocation: proofs of optimality and equivalence. Health Economics, 8(2): 171–174, 1999.

[33] Fenwick E, Claxton K, Sculpher M. Representing uncertainty: the role of cost-effectiveness acceptability curves. Health Economics, 10(8): 779–787, 2001.

[34] Drummond M F, Sculpher M J, Claxton K, Stoddart G L, Torrance G W. Methods for the Economic Evaluation of Health Care Programmes, 4th ed. Oxford: Oxford University Press, 2015.

[35] Husereau D, Drummond M, Augustovski F, et al. Consolidated Health Economic Evaluation Reporting Standards 2022 (CHEERS 2022) Statement. Value in Health, 25(1): 3–9, 2022.

[36] Hwang C L, Yoon K. Multiple Attribute Decision Making: Methods and Applications. Lecture Notes in Economics and Mathematical Systems, vol. 186. Berlin: Springer-Verlag, 1981.

[37] Satopää V, Albrecht J, Irwin D, Raghavan B. Finding a “Kneedle” in a Haystack: Detecting Knee Points in System Behavior. In: 31st IEEE International Conference on Distributed Computing Systems Workshops (ICDCSW), 166–171, 2011.

[38] Neumann P J, Sanders G D, Russell L B, Siegel J E, Ganiats T G (eds). Cost-Effectiveness in Health and Medicine, 2nd ed. Oxford University Press, 2016.

## 附录 1：附录

本次参赛使用软件环境如下：Visual Studio Code；TexLive；Pycharm2026.1；AnaConda3；Python 3.10；依赖库：pandas 2.x, numpy 1.x, scipy 1.x, scikit-learn 1.x, statsmodels 0.14+, matplotlib 3.x。

所用 Python 源码如下：

```python
from __future__ import annotations
from pathlib import Path
import matplotlib
import numpy as np
import pandas as pd


RANDOM_SEED = 42
CODE_ROOT = Path(__file__).resolve().parent
SRC_ROOT = CODE_ROOT.parent
PROJECT_ROOT = SRC_ROOT.parent
DATA_ROOT = SRC_ROOT / "Data"
OUTPUT_ROOT = SRC_ROOT / "outputs"
FIGURE_DIR = OUTPUT_ROOT / "figures"
TABLE_DIR = OUTPUT_ROOT / "tables"


def ensure_output_dirs() -> None:
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    TABLE_DIR.mkdir(parents=True, exist_ok=True)


def configure_plotting() -> None:
    matplotlib.rcParams["font.sans-serif"] = [
        "DejaVu Sans",
        "SimHei",
        "Microsoft YaHei",
        "Arial Unicode MS",
    ]
    matplotlib.rcParams["axes.unicode_minus"] = False


def set_random_seed(seed: int = RANDOM_SEED) -> None:
    np.random.seed(seed)


def find_sample_data_path() -> Path:
    candidates = [
        path
        for path in DATA_ROOT.glob("*样例数据.xlsx")
        if path.is_file() and not path.name.startswith("~$")
    ]
    if not candidates:
        raise FileNotFoundError("未找到样例数据文件，请确认 Excel 文件已放入 src/Data 目录。")
```

```python
    candidates.sort(key=lambda item: ("附件1" not in item.name, item.name))
    return candidates[0]


def load_data() -> pd.DataFrame:
    ensure_output_dirs()
    return pd.read_excel(find_sample_data_path())


def figure_path(name: str) -> Path:
    ensure_output_dirs()
    return FIGURE_DIR / name


def table_path(name: str) -> Path:
    ensure_output_dirs()
    return TABLE_DIR / name
```

Listing 1: common.py

```python
"""
问题1: 关键指标筛选与九种体质贡献度分析（修订版）

针对题目严格要求:
  - 候选特征仅为血常规指标 + 中老年人活动量表评分（不含性别/吸烟/饮酒/年龄组）
  - 活动量表需考虑 ADL/IADL 各子项
  - 多重共线性诊断与处理
  - 指标重要度评估采用 5 种互补方法 + Borda 秩次聚合（而非 Z-score 加和）
"""
import numpy as np, pandas as pd, warnings
from scipy import stats
from sklearn.feature_selection import mutual_info_classif, mutual_info_regression
from sklearn.linear_model import ElasticNetCV, LogisticRegression, LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from statsmodels.stats.outliers_influence import variance_inflation_factor
import matplotlib.pyplot as plt
from matplotlib import rcParams
from common import configure_plotting, figure_path, load_data, set_random_seed, table_path

warnings.filterwarnings('ignore')
configure_plotting()

set_random_seed()
df = load_data()

# ====================================================================
# Step 0: 数据清洗与质量检查
# ====================================================================
print("="*60); print("STEP 0: 数据质量检查"); print("="*60)

# 检查缺失、重复
print(f"总样本数: {len(df)} 列数: {len(df.columns)}")
print(f"缺失值总数: {df.isnull().sum().sum()}")
print(f"样本ID重复: {df['样本ID'].duplicated().sum()}")

# ADL/IADL 子项和与总分一致性
adl_items = ['ADL用厕','ADL吃饭','ADL步行','ADL穿衣','ADL洗澡']
iadl_items = ['IADL购物','IADL做饭','IADL理财','IADL交通','IADL服药']
adl_ok = (df[adl_items].sum(axis=1) == df['ADL总分']).all()
iadl_ok = (df[iadl_items].sum(axis=1) == df['IADL总分']).all()
tot_ok = (df['ADL总分'] + df['IADL总分'] == df['活动量表总分（ADL总分+IADL总分）']).all()
print(f"ADL子项之和=总分: {adl_ok}; IADL: {iadl_ok}; 总分: {tot_ok}")

# 体质标签 vs 最高体质积分一致性
tizhi_cols = ['平和质','气虚质','阳虚质','阴虚质','痰湿质','湿热质','血瘀质','气郁质','特禀质']
argmax_id = df[tizhi_cols].values.argmax(axis=1) + 1
```

```python
lab_consistency = (argmax_id == df['体质标签'].values).sum()
print(f"体质标签与最高积分体质一致的样本: {lab_consistency}/{len(df)} ({lab_consistency/len(df)*100:.2f}%)")
print(f"不一致样本数: {len(df) - lab_consistency} (主要体质判定歧义, 保留但在分析中说明)")

# 诊断标签与血脂指标一致性
def abn_count(r):
    c=0
    if r['TC（总胆固醇）']>6.2: c+=1
    if r['TG（甘油三酯）']>1.7: c+=1
    if r['LDL-C（低密度脂蛋白）']>3.1: c+=1
    if r['HDL-C（高密度脂蛋白）']<1.04: c+=1
    return c
df['血脂异常项数'] = df.apply(abn_count, axis=1)
df['临床异常'] = (df['血脂异常项数']>=1).astype(int)
match_rate = (df['临床异常']==df['高血脂症二分类标签']).mean()
print(f"诊断标签 vs 血脂异常 1 一致率: {match_rate*100:.1f}%")

# 异常值检测 (Tukey IQR 法)
print("\n异常值检测 (1.5×IQR 准则):")
for col in ['TC（总胆固醇）','TG（甘油三酯）','LDL-C（低密度脂蛋白）','HDL-C（高密度脂蛋白）',
            '空腹血糖','血尿酸','BMI']:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    lo, hi = q1-1.5*iqr, q3+1.5*iqr
    out = ((df[col]<lo) | (df[col]>hi)).sum()
    print(f" {col}: outliers={out} ({out/len(df)*100:.1f}%); 范围[{df[col].min():.2f}, {df[col].max():.2f}]")

# 保留原始数据做分析（离群不删除, 毕竟临床数据离群也有意义）
print("\n→ 处理策略: 保留所有 1000 条记录; 标注 65 例体质判定歧义但不剔除;")
print("           异常值多数为临床真实疾病表现, 保留; 后续分析中对离群稳健方法(Spearman, RF)不敏感.")

# ====================================================================
# Step 1: 候选特征清单 —— 严格限定为血常规 + 活动量表
# ====================================================================
print("\n"+"="*60); print("STEP 1: 候选特征清单"); print("="*60)

blood = ['TC（总胆固醇）','TG（甘油三酯）','LDL-C（低密度脂蛋白）',
         'HDL-C（高密度脂蛋白）','空腹血糖','血尿酸','BMI']
adl_sub = adl_items      # 5 项 ADL 子项
iadl_sub = iadl_items    # 5 项 IADL 子项
adl_total = ['ADL总分','IADL总分','活动量表总分（ADL总分+IADL总分）']

# Set A: 最细粒度 —— 血常规 + ADL/IADL 子项
SET_A = blood + adl_sub + iadl_sub       # 17 个特征
# Set B: 总分 —— 血常规 + ADL/IADL 总分
SET_B = blood + adl_total                 # 10 个特征

print(f"Set A (细粒度, {len(SET_A)}个): {SET_A}")
print(f"Set B (总分级, {len(SET_B)}个): {SET_B}")

# ====================================================================
# Step 2: 多重共线性诊断 (VIF)
# ====================================================================
print("\n"+"="*60); print("STEP 2: 多重共线性诊断"); print("="*60)

def vif_table(df_in, cols):
    X = StandardScaler().fit_transform(df_in[cols].values)
    rows=[]
    for i, c in enumerate(cols):
        try:
            v = variance_inflation_factor(X, i)
        except Exception:
            v = np.nan
```

```python
        rows.append({'feature': c, 'VIF': v})
    return pd.DataFrame(rows).sort_values('VIF', ascending=False)

vif_A = vif_table(df, SET_A)
print("\nVIF (Set A, 细粒度):"); print(vif_A.round(2).to_string(index=False))
vif_B = vif_table(df, SET_B)
print("\nVIF (Set B, 总分):"); print(vif_B.round(2).to_string(index=False))

vif_A.to_csv(table_path('Q1_vif_setA.csv'), encoding='utf-8-sig', index=False)
vif_B.to_csv(table_path('Q1_vif_setB.csv'), encoding='utf-8-sig', index=False)

# Set B 中 "活动量表总分" = ADL总分+IADL总分 必然完美共线 -> 只保留总分 或 两分项
# 筛选策略: 使用 Set C = 血常规 + ADL/IADL 子项 (剔除总分), 这是既保留细节又避免完美共线
# 由于 ADL子项之和=ADL总分, 若同时纳入会完全共线, 故采取"子项 XOR 总分"
SET_C = blood + adl_sub + iadl_sub        # 同 Set A
print(f"\n选定 Set C = 血常规({len(blood)}) + ADL子项({len(adl_sub)}) + IADL子项({len(iadl_sub)}) = {len(SET_C)} 个")
print("(不纳入总分以避免完全共线 ADL_sum = ΣADL_i)")

# 风险预警候选集: 从题设血常规+活动量表中剔除 4 个直接诊断血脂指标, 避免标签泄露
risk_blood = ['空腹血糖', '血尿酸', 'BMI']
RISK_SET = risk_blood + adl_sub + iadl_sub
print(f"风险预警候选集 = 非直接诊断指标({len(risk_blood)}) + ADL子项({len(adl_sub)}) + IADL子项({len(iadl_sub)}) = {len(RISK_SET)} 个")
print("(显式排除 TC/TG/LDL-C/HDL-C 4 项直接血脂诊断指标以避免标签泄露)")

# VIF 再检查 Set C (同 Set A)
vif_C = vif_A.copy()
# 去除超高共线特征 (VIF > 10, 除血脂相关)
to_keep = vif_C.copy()
print(f"\nSet C 中 VIF 最大的指标: {vif_C.iloc[0]['feature']} = {vif_C.iloc[0]['VIF']:.2f}")
# 允许血脂指标留存 (有明确临床意义), 只做标注

# ====================================================================
# Step 3: 相关性矩阵与热力图
# ====================================================================
print("\n"+"="*60); print("STEP 3: 相关性矩阵"); print("="*60)

corr_mat = df[SET_C].corr(method='spearman')
print("相关矩阵(Spearman) 绝对值 > 0.5 的对:")
for i in range(len(SET_C)):
    for j in range(i+1, len(SET_C)):
        v = corr_mat.iloc[i,j]
        if abs(v) > 0.5:
            print(f" {SET_C[i]} - {SET_C[j]}:   = {v:.3f}")

fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(corr_mat.values, cmap='RdBu_r', vmin=-1, vmax=1, aspect='auto')
ax.set_xticks(range(len(SET_C))); ax.set_yticks(range(len(SET_C)))
labels_en = {
    'TC（总胆固醇）':'TC','TG（甘油三酯）':'TG','LDL-C（低密度脂蛋白）':'LDL-C',
    'HDL-C（高密度脂蛋白）':'HDL-C','空腹血糖':'FBS','血尿酸':'UA','BMI':'BMI',
    'ADL用厕':'ADL-toilet','ADL吃饭':'ADL-eat','ADL步行':'ADL-walk',
    'ADL穿衣':'ADL-dress','ADL洗澡':'ADL-bath',
    'IADL购物':'IADL-shop','IADL做饭':'IADL-cook','IADL理财':'IADL-finance',
    'IADL交通':'IADL-transp','IADL服药':'IADL-med'
}
en_labels = [labels_en.get(c, c[:10]) for c in SET_C]
ax.set_xticklabels(en_labels, rotation=45, ha='right')
ax.set_yticklabels(en_labels)
for i in range(len(SET_C)):
    for j in range(len(SET_C)):
        v = corr_mat.values[i,j]
        if abs(v) > 0.3:
            ax.text(j, i, f"{v:.2f}", ha='center', va='center',
                    color='white' if abs(v) > 0.6 else 'black', fontsize=7)
```

```python
plt.colorbar(im, ax=ax)
ax.set_title('Spearman Correlation Matrix (Blood + ADL/IADL sub-items)')
plt.tight_layout()
plt.savefig(figure_path('Q1_corr_heatmap.png'), dpi=150)
plt.close()

# ==================================================================
# Step 4: 五种互补方法评估
# ==================================================================
print("\n"+"="*60); print("STEP 4: 五种互补方法特征评估"); print("="*60)
print("""
方法论 (5 种互补方法):
  M1. Spearman 秩相关      (非参数 · 单调 · 稳健)
  M2. 互信息 MI            (非参数 · 非线性 · 捕捉任意依赖)
  M3. 弹性网络回归         (L1+L2 联合惩罚 · 稀疏且稳健处理共线特征组)
  M4. 随机森林置换重要度   (嵌入式 · 非参数化 · 捕捉交互)
  M5. 偏相关               (参数 · 控制其它变量后的净效应)

聚合: 对每种方法得到的指标排名, 使用 Borda 计数法 (排序聚合) 而非 Z-score 加和,
      以回避数值量纲不同、分布偏斜等问题, 得到稳健一致的最终排名.
""")

def borda_rank(score_dict_list, features, higher_better=True):
    """score_dict_list: list of dicts (feature -> score). higher_better=True 则 score 越高名次越前"""
    k = len(features)
    points = {f: 0 for f in features}
    for sd in score_dict_list:
        # 排名: higher_better ? 降序排 : 升序排
        sorted_feats = sorted(features,
                              key=lambda f: sd.get(f, 0),
                              reverse=higher_better)
        for rank, f in enumerate(sorted_feats):
            points[f] += (k - rank) # 第1名得k分, 末位得1分
    return points

# ===== 4.1 表征痰湿体质严重程度 —— 以痰湿积分为连续目标 =====
print("\n--- 4.1 表征痰湿体质严重程度(痰湿积分为目标) ---")
y_phl = df['痰湿质'].values
X_phl = df[SET_C].values
Xs = StandardScaler().fit_transform(X_phl)

m1_phl, m2_phl, m3_phl, m4_phl, m5_phl = {}, {}, {}, {}, {}

# M1 Spearman
for f in SET_C:
    r, _ = stats.spearmanr(df[f].values, y_phl)
    m1_phl[f] = abs(r)

# M2 MI
mi = mutual_info_regression(X_phl, y_phl, random_state=42)
for i, f in enumerate(SET_C):
    m2_phl[f] = mi[i]

# M3 弹性网络
enet_l1_ratios = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99, 1.0]
enet = ElasticNetCV(
    l1_ratio=enet_l1_ratios,
    alphas=200,
    cv=5,
    random_state=42,
    max_iter=20000,
)
enet.fit(Xs, y_phl)
print(f"M3 回归最优参数: best_alpha={enet.alpha_:.6f}, best_l1_ratio={enet.l1_ratio_:.2f}")
for i, f in enumerate(SET_C):
    m3_phl[f] = abs(enet.coef_[i])
```

```python

# M4 RF 置换重要度
rf = RandomForestRegressor(n_estimators=500, max_depth=10, random_state=42, n_jobs=-1)
rf.fit(X_phl, y_phl)
from sklearn.inspection import permutation_importance
perm = permutation_importance(rf, X_phl, y_phl, n_repeats=10, random_state=42, n_jobs=-1)
for i, f in enumerate(SET_C):
    m4_phl[f] = perm.importances_mean[i]

# M5 偏相关 (各特征与 y 控制其他特征后的偏相关系数)
# 用 OLS 回归计算  再转偏相关
import statsmodels.api as sm
X_const = sm.add_constant(Xs)
ols = sm.OLS(y_phl, X_const).fit()
# partial r = t / sqrt(t^2 + df)
tvals = ols.tvalues[1:] # 去掉 const
dfo = ols.df_resid
for i, f in enumerate(SET_C):
    m5_phl[f] = abs(tvals[i] / np.sqrt(tvals[i]**2 + dfo))

# Borda 聚合
borda_phl = borda_rank([m1_phl, m2_phl, m3_phl, m4_phl, m5_phl], SET_C, higher_better=True)
df_phl = pd.DataFrame({
    'feature': SET_C,
    'Spearman|r|': [m1_phl[f] for f in SET_C],
    'MI': [m2_phl[f] for f in SET_C],
    'ENet| |': [m3_phl[f] for f in SET_C],
    'RF_perm': [m4_phl[f] for f in SET_C],
    'Partial_r': [m5_phl[f] for f in SET_C],
    'Borda_pts': [borda_phl[f] for f in SET_C],
}).sort_values('Borda_pts', ascending=False)
df_phl['Borda_排名'] = range(1, len(SET_C)+1)

print("表征痰湿严重程度的特征排序 (Borda 聚合):")
print(df_phl.round(4).to_string(index=False))
df_phl.to_csv(table_path('Q1_phlegm_ranking.csv'), encoding='utf-8-sig', index=False)

# ===== 4.2 预警高血脂发病风险 —— 以二分类标签为目标 =====
print("\n--- 4.2 预警高血脂发病风险(二分类标签为目标) ---")
print("说明: 风险预警筛选已排除 TC/TG/LDL-C/HDL-C 四项直接血脂诊断指标, 以避免标签泄露。")
y_risk = df['高血脂症二分类标签'].values
m1_r, m2_r, m3_r, m4_r, m5_r = {}, {}, {}, {}, {}
X_risk = df[RISK_SET].values
Xs_risk = StandardScaler().fit_transform(X_risk)

# M1 Spearman (二分类也可用)
for f in RISK_SET:
    r, _ = stats.spearmanr(df[f].values, y_risk)
    m1_r[f] = abs(r)

# M2 MI (classif)
mi = mutual_info_classif(X_risk, y_risk, random_state=42)
for i, f in enumerate(RISK_SET):
    m2_r[f] = mi[i]

# M3 弹性网络 Logistic
logit_Cs = np.logspace(-2, 2, 20)
logit_l1_ratios = [0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99]
lr_enet = LogisticRegressionCV(
    cv=5,
    penalty='elasticnet',
    solver='saga',
    Cs=logit_Cs,
    l1_ratios=logit_l1_ratios,
    scoring='roc_auc',
    max_iter=20000,
    random_state=42,
```

```python
    n_jobs=-1,
)
lr_enet.fit(Xs_risk, y_risk)
print(f"M3 分类最优参数: best_C={lr_enet.C_[0]:.6f}, best_l1_ratio={lr_enet.l1_ratio_[0]:.2f}")
for i, f in enumerate(RISK_SET):
    m3_r[f] = abs(lr_enet.coef_[0][i])

# M4 RF 置换
rfc = RandomForestClassifier(n_estimators=500, max_depth=10,
                             random_state=42, class_weight='balanced', n_jobs=-1)
rfc.fit(X_risk, y_risk)
perm_r = permutation_importance(rfc, X_risk, y_risk, n_repeats=10, random_state=42, n_jobs=-1)
for i, f in enumerate(RISK_SET):
    m4_r[f] = perm_r.importances_mean[i]

# M5 偏相关 (Logistic 回归 Wald 检验)
X_const_risk = sm.add_constant(Xs_risk)
lr_full = sm.Logit(y_risk, X_const_risk).fit(disp=0, maxiter=2000)
zvals = lr_full.tvalues[1:]
for i, f in enumerate(RISK_SET):
    m5_r[f] = abs(zvals[i]) / (abs(zvals[i]) + 1) # 标准化到 (0,1)

borda_r = borda_rank([m1_r, m2_r, m3_r, m4_r, m5_r], RISK_SET, higher_better=True)
df_risk = pd.DataFrame({
    'feature': RISK_SET,
    'Spearman|r|': [m1_r[f] for f in RISK_SET],
    'MI': [m2_r[f] for f in RISK_SET],
    'ENet| |': [m3_r[f] for f in RISK_SET],
    'RF_perm': [m4_r[f] for f in RISK_SET],
    'Wald': [m5_r[f] for f in RISK_SET],
    'Borda_pts': [borda_r[f] for f in RISK_SET],
}).sort_values('Borda_pts', ascending=False)
df_risk['Borda_排名'] = range(1, len(RISK_SET)+1)

print("预警高血脂发病风险的特征排序 (Borda 聚合):")
print(df_risk.round(4).to_string(index=False))
df_risk.to_csv(table_path('Q1_risk_ranking.csv'), encoding='utf-8-sig', index=False)

# ====================================================================
# Step 5: 九种体质对发病风险的贡献度
# ====================================================================
print("\n"+"="*60); print("STEP 5: 九种体质对发病风险的贡献度"); print("="*60)

# 方法1: 标签为定类变量的贡献度 -> 独热编码后各体质的Logistic beta
# 方法2: 体质积分的多元Logistic beta
# 方法3: 各体质发病率比 vs 整体
# 方法4: 卡方检验（体质标签 vs 诊断）
# 方法5: Cohen's h (效应量)

y = df['高血脂症二分类标签'].values
p_all = y.mean() # 整体发病率

# Method A: 独热编码体质标签 + Logistic回归
X_oh = pd.get_dummies(df['体质标签'], prefix='体质').values
lr_oh = LogisticRegression(max_iter=2000, random_state=42)
lr_oh.fit(X_oh, y)
beta_oh = lr_oh.coef_[0]

# Method B: 体质积分多元 Logistic
Xtz = StandardScaler().fit_transform(df[tizhi_cols].values)
lr_tz = LogisticRegression(max_iter=2000, random_state=42)
lr_tz.fit(Xtz, y)
beta_score = lr_tz.coef_[0]

# Method C: 各体质人群发病率、相对风险RR、绝对风险差ARR
```

```python
rows = []
for i, t in enumerate(tizhi_cols, 1):
    sub = df[df['体质标签'] == i]
    n = len(sub); p = sub['高血脂症二分类标签'].mean()
    rr = p / p_all
    arr = p - p_all
    # Cohen's h
    h = 2*np.arcsin(np.sqrt(p)) - 2*np.arcsin(np.sqrt(p_all))
    # 卡方 (该体质 vs 其它)
    others = df[df['体质标签'] != i]['高血脂症二分类标签']
    tab = pd.crosstab(
        pd.Series([1]*len(sub) + [0]*len(others)),
        pd.concat([sub['高血脂症二分类标签'], others]))
    chi2, pval, _, _ = stats.chi2_contingency(tab)
    rows.append({'编号': i, '体质': t, '样本量': n, '发病率': p,
                 'RR': rr, 'ARR': arr, 'Cohen_h': h, 'chi2_p': pval,
                 '独热 ': beta_oh[i-1] if i-1 < len(beta_oh) else np.nan,
                 '积分多元 ': beta_score[i-1]})
df_tz = pd.DataFrame(rows)

# 贡献度综合 (Borda 基于 |Cohen_h|, |RR-1|, |ARR|, |独热 |, |积分多元 |)
contrib_scores = [
    {r['体质']: abs(r['Cohen_h']) for _, r in df_tz.iterrows()},
    {r['体质']: abs(r['RR']-1) for _, r in df_tz.iterrows()},
    {r['体质']: abs(r['ARR']) for _, r in df_tz.iterrows()},
    {r['体质']: abs(r['独热 ']) for _, r in df_tz.iterrows()},
    {r['体质']: abs(r['积分多元 ']) for _, r in df_tz.iterrows()},
]
borda_tz = borda_rank(contrib_scores, tizhi_cols, higher_better=True)
df_tz['Borda_pts'] = [borda_tz[t] for t in df_tz['体质']]
df_tz = df_tz.sort_values('Borda_pts', ascending=False).reset_index(drop=True)
df_tz['贡献排名'] = range(1, 10)

print("九种体质对发病风险的贡献度 (发病率降序):")
print(df_tz.round(4).to_string(index=False))
df_tz.to_csv(table_path('Q1_tizhi_contribution.csv'), encoding='utf-8-sig', index=False)

# ==================================================================
# Step 6: 可视化汇总
# ==================================================================
print("\n"+"="*60); print("STEP 6: 生成汇总图"); print("="*60)

fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# (1) 痰湿严重程度 top-8
ax = axes[0, 0]
top8 = df_phl.head(8)
ax.barh(range(8)[::-1], top8['Borda_pts'].values, color='steelblue')
ax.set_yticks(range(8)[::-1])
ax.set_yticklabels([labels_en.get(f, f[:10]) for f in top8['feature']])
ax.set_xlabel('Borda points')
ax.set_title('Top-8 Indicators for Phlegm-damp Severity')

# (2) 发病风险 top-8
ax = axes[0, 1]
top8r = df_risk.head(8)
ax.barh(range(8)[::-1], top8r['Borda_pts'].values, color='firebrick')
ax.set_yticks(range(8)[::-1])
ax.set_yticklabels([labels_en.get(f, f[:10]) for f in top8r['feature']])
ax.set_xlabel('Borda points')
ax.set_title('Top-8 Indicators for Hyperlipidemia Risk')

# (3) 九种体质 Borda 贡献度
ax = axes[1, 0]
color_list = ['darkorange' if t=='痰湿质' else 'steelblue' for t in df_tz['体质']]
bars = ax.bar(range(9), df_tz['Borda_pts'], color=color_list)
name_en = {'平和质':'Peace','气虚质':'Qi-def','阳虚质':'Yang-def','阴虚质':'Yin-def',
```

```
        '痰湿质':'Phlegm','湿热质':'Damp-heat','血瘀质':'Blood-stasis',
        '气郁质':'Qi-stag','特禀质':'Const'}
ax.set_xticks(range(9))
ax.set_xticklabels([name_en[t] for t in df_tz['体质']], rotation=30)
ax.set_ylabel('Borda points')
ax.set_title('9 Constitutions Contribution (Borda rank)')
ax.grid(axis='y', alpha=0.3)

# (4) 九种体质发病率 vs 整体
ax = axes[1, 1]
sorted_df_tz = df_tz.sort_values('编号') # 按原序1-9
color2 = ['darkorange' if t==5 else 'steelblue' for t in sorted_df_tz['编号']]
ax.bar(range(9), sorted_df_tz['发病率'], color=color2)
ax.axhline(p_all, color='k', ls='--', alpha=0.5, label=f'overall={p_all:.3f}')
ax.set_xticks(range(9))
ax.set_xticklabels([name_en[t] for t in sorted_df_tz['体质']], rotation=30)
ax.set_ylabel('Hyperlipidemia prevalence')
ax.set_title('Prevalence by Constitution Type')
ax.legend()
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig(figure_path('Q1_summary.png'), dpi=150)
plt.close()
print("图已保存: Q1_summary.png, Q1_corr_heatmap.png")
print("数据已保存: Q1_*.csv")
```

Listing 2: problem1.py

```
# -*- coding: utf-8 -*-
"""
================================================================================
  问题二: 中老年高血脂三级风险预警模型
  范式转换: 诊断重述 -> 筛查工具（剔除血脂四项，消除标签泄漏）
  方法体系:
    - WOE-Logistic 评分卡（Siddiqi 2006 经典范式）作为可解释主线
    - GradientBoostingClassifier + RandomForest 作为非线性对照
    - Youden 指数 + Bootstrap 置信区间 + 特异度约束双阈值
    - Cochran-Armitage 趋势检验 + NNT 临床锚定
    - Isotonic 校准 + HL 检验 + DeLong + 决策曲线 DCA
    - 痰湿子集三路径挖掘: Apriori + 决策树 + 置换重要度
================================================================================
依赖: pandas numpy scipy scikit-learn matplotlib openpyxl
运行: python src/pyCode/problem2.py
输出: src/outputs/figures/Q2_*.png, src/outputs/tables/Q2_results.json 与配套 CSV/TXT
================================================================================
"""

import os, json, warnings, itertools
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import chi2, norm

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.calibration import calibration_curve, CalibratedClassifierCV
from sklearn.inspection import permutation_importance
from sklearn.metrics import (
    roc_auc_score, brier_score_loss, roc_curve, f1_score,
```

```python
    confusion_matrix, cohen_kappa_score,
)

from common import configure_plotting, figure_path, load_data, set_random_seed, table_path

RNG = 42
configure_plotting()
set_random_seed(RNG)

# ============================================================================
# 【配置】
# ============================================================================
DATA_PATH = os.environ.get("Q2_DATA_PATH")

COL_MAP = {
    "label":    "高血脂症二分类标签",
    "phlegm":   "痰湿质",
    "fpg":      "空腹血糖",
    "ua":       "血尿酸",
    "bmi":      "BMI",
    "act":      "活动量表总分（ADL总分+IADL总分）",
    "age_group": "年龄组",
    "gender":   "性别",
    "smoking":  "吸烟史",
    "drinking": "饮酒史",
    "tizhi":    "体质标签",
    "tc":       "TC（总胆固醇）",
    "tg":       "TG（甘油三酯）",
    "ldl":      "LDL-C（低密度脂蛋白）",
    "hdl":      "HDL-C（高密度脂蛋白）",
}
FEATURES_CONT = ["phlegm", "fpg", "ua", "bmi", "act"]
FEATURES_ORD = ["age_group"]
FEATURES_CAT = ["gender", "smoking", "drinking"]
FEATURES_ALL = FEATURES_CONT + FEATURES_ORD + FEATURES_CAT
LIPID_BLACKLIST = ["tc", "tg", "ldl", "hdl"]

PDO = 20
BASE_POINTS = 100
BASE_ODDS = 1.0


# ============================================================================
# 模块 A — 数据加载 + 泄漏自查 + 预处理
# ============================================================================
def module_A_preprocess(path=DATA_PATH):
    df = pd.read_excel(path) if path else load_data()
    inv = {v: k for k, v in COL_MAP.items() if v in df.columns}
    missing = [v for v in COL_MAP.values() if v not in df.columns]
    if missing:
        raise KeyError(f"Excel 中缺失以下列: {missing}")
    df = df.rename(columns=inv)

    abn = ((df["tc"] > 6.2) | (df["tg"] > 1.7) |
           (df["ldl"] > 3.1) | (df["hdl"] < 1.04)).astype(int)
    consist = float((abn == df["label"]).mean())
    pos_rate = float(df["label"].mean())
    n_phlegm = int((df["tizhi"] == 5).sum())
    print(f"[A] n={len(df)}, 阳性率={pos_rate:.4f}, "
          f"泄漏一致率={consist:.4f}, 痰湿体质 n={n_phlegm}")
    print(f"[A] Layer-1 血脂异常人数={int(abn.sum())} "
          f"(覆盖率 {abn.sum()/len(df):.2%})")

    X = df[FEATURES_ALL].copy()
    y = df["label"].astype(int).values
    tizhi = df["tizhi"].astype(int).values
    abn_full = abn.values.astype(int)
```

```python

    X[FEATURES_CONT] = SimpleImputer(strategy="median") \
        .fit_transform(X[FEATURES_CONT])
    X[FEATURES_ORD + FEATURES_CAT] = SimpleImputer(strategy="most_frequent") \
        .fit_transform(X[FEATURES_ORD + FEATURES_CAT])
    X = X.astype(float)

    Xtr, Xte, ytr, yte, ttr, tte, abn_tr, abn_te = train_test_split(
        X, y, tizhi, abn_full,
        test_size=0.30, stratify=y, random_state=RNG)

    return dict(X_raw=X, y=y, tizhi=tizhi, abn_full=abn_full,
                Xtr_raw=Xtr, Xte_raw=Xte,
                ytr=ytr, yte=yte, ttr=ttr, tte=tte,
                abn_tr=abn_tr, abn_te=abn_te,
                leak_rate=consist, pos_rate=pos_rate, n_phlegm=n_phlegm)


# ==============================================================================
# 模块 B — WOE / IV 编码
# ==============================================================================
def _make_bins_cont(x, n_bins=5):
    edges = np.unique(np.quantile(x, np.linspace(0, 1, n_bins + 1)))
    if len(edges) < 3:
        return np.array([x.min() - 1, x.max() + 1])
    return edges


def woe_encode_train(Xtr, ytr, cont=FEATURES_CONT):
    y = np.asarray(ytr)
    n1_total = int(y.sum()); n0_total = len(y) - n1_total
    eps = 0.5
    woe_map, edges_map, iv_summary = {}, {}, {}

    for f in Xtr.columns:
        x = Xtr[f].values
        if f in cont:
            edges = _make_bins_cont(x, n_bins=5)
            bins = np.digitize(x, edges[1:-1])
            labels = [(float(edges[i]), float(edges[i + 1]))
                      for i in range(len(edges) - 1)]
        else:
            uniq = sorted(np.unique(x).tolist())
            labels = [(float(v), float(v)) for v in uniq]
            mapping = {v: i for i, v in enumerate(uniq)}
            bins = np.array([mapping[v] for v in x])
            edges = uniq

        rows = []
        for b_id, (lo, hi) in enumerate(labels):
            mask = (bins == b_id)
            n = int(mask.sum())
            n1 = int(y[mask].sum())
            n0 = n - n1
            p1 = (n1 + eps) / (n1_total + eps * len(labels))
            p0 = (n0 + eps) / (n0_total + eps * len(labels))
            woe = float(np.log(p1 / p0))
            iv_i = float((p1 - p0) * woe)
            rows.append([b_id, lo, hi, n, n1, n0, woe, iv_i])
        tbl = pd.DataFrame(rows, columns=["bin_id", "lo", "hi",
                                          "n", "n1", "n0", "woe", "iv_i"])
        woe_map[f] = tbl
        edges_map[f] = edges
        iv_summary[f] = float(tbl["iv_i"].sum())
    return woe_map, edges_map, iv_summary


```

```python
def woe_transform(X, woe_map, edges_map, cont=FEATURES_CONT):
    out = pd.DataFrame(index=X.index)
    for f in X.columns:
        x = X[f].values
        if f in cont:
            edges = edges_map[f]
            bins = np.digitize(x, edges[1:-1])
        else:
            uniq = edges_map[f]
            mapping = {v: i for i, v in enumerate(uniq)}
            bins = np.array([mapping.get(v, 0) for v in x])
        woe_vals = woe_map[f].set_index("bin_id")["woe"].to_dict()
        default = list(woe_vals.values())[0]
        out[f] = np.array([woe_vals.get(int(b), default) for b in bins])
    return out


# ==============================================================================
# 模块 C — 三模型 5 折 CV
# ==============================================================================
def bootstrap_ci(y, p, metric_fn, B=1000, alpha=0.05, seed=RNG):
    rng = np.random.default_rng(seed)
    n = len(y); scores = []
    for _ in range(B):
        idx = rng.integers(0, n, n)
        if len(np.unique(y[idx])) < 2:
            continue
        scores.append(metric_fn(y[idx], p[idx]))
    lo, hi = np.percentile(scores, [100 * alpha / 2, 100 * (1 - alpha / 2)])
    return (round(float(lo), 4), round(float(hi), 4),
            round(float(np.mean(scores)), 4))


def module_C_cv_train(d, woe_map, edges_map):
    Xtr_woe = woe_transform(d["Xtr_raw"], woe_map, edges_map)
    Xte_woe = woe_transform(d["Xte_raw"], woe_map, edges_map)
    ytr = d["ytr"]

    models = {
        "LR_WOE": (LogisticRegression(
            C=1.0, solver="liblinear", class_weight="balanced",
            max_iter=1000, random_state=RNG), Xtr_woe, Xte_woe),
        "GBDT":  (GradientBoostingClassifier(
            n_estimators=200, max_depth=3, learning_rate=0.05,
            subsample=0.8, min_samples_leaf=20, random_state=RNG),
            d["Xtr_raw"], d["Xte_raw"]),
        "RF":    (RandomForestClassifier(
            n_estimators=300, max_depth=6, min_samples_leaf=10,
            class_weight="balanced", n_jobs=-1, random_state=RNG),
            d["Xtr_raw"], d["Xte_raw"]),
    }
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RNG)
    oof = {k: np.zeros(len(ytr)) for k in models}
    fold_aucs = {k: [] for k in models}

    for tr, va in skf.split(np.zeros(len(ytr)), ytr):
        for k, (m, Xt, _) in models.items():
            m.fit(Xt.iloc[tr], ytr[tr])
            oof[k][va] = m.predict_proba(Xt.iloc[va])[:, 1]
            fold_aucs[k].append(roc_auc_score(ytr[va], oof[k][va]))

    cv_summary = {}
    for k, (m, Xt, Xev) in models.items():
        mu, sd = np.mean(fold_aucs[k]), np.std(fold_aucs[k])
        lo, hi, _ = bootstrap_ci(ytr, oof[k], roc_auc_score)
        cv_summary[k] = dict(
            auc_mean=round(float(mu), 4),
```

```python
            auc_std=round(float(sd), 4),
            auc_95ci=(lo, hi),
            brier=round(float(brier_score_loss(ytr, oof[k])), 4),
            fold_aucs=[round(float(a), 4) for a in fold_aucs[k]],
        )
        print(f"[C] {k}: AUC={mu:.4f}±{sd:.4f} 95%CI=({lo},{hi}) "
              f"Brier={cv_summary[k]['brier']}")
        m.fit(Xt, ytr)
    return models, oof, cv_summary, Xtr_woe, Xte_woe


def delong_test(y, p1, p2):
    y = np.asarray(y); p1 = np.asarray(p1); p2 = np.asarray(p2)
    auc1, auc2 = roc_auc_score(y, p1), roc_auc_score(y, p2)
    pos, neg = np.where(y == 1)[0], np.where(y == 0)[0]
    m, n = len(pos), len(neg)
    def v_struct(p):
        V10 = np.zeros(m); V01 = np.zeros(n)
        for i, pi in enumerate(pos):
            V10[i] = np.mean((p[pi] > p[neg]) + 0.5 * (p[pi] == p[neg]))
        for j, nj in enumerate(neg):
            V01[j] = np.mean((p[pos] > p[nj]) + 0.5 * (p[pos] == p[nj]))
        return V10, V01
    V10_1, V01_1 = v_struct(p1)
    V10_2, V01_2 = v_struct(p2)
    S10 = np.cov(np.vstack([V10_1, V10_2]))
    S01 = np.cov(np.vstack([V01_1, V01_2]))
    S = S10 / m + S01 / n
    var_diff = S[0, 0] + S[1, 1] - 2 * S[0, 1]
    if var_diff <= 0:
        return auc1, auc2, 0.0, 1.0
    z = (auc1 - auc2) / np.sqrt(var_diff)
    pval = 2 * (1 - norm.cdf(abs(z)))
    return float(auc1), float(auc2), float(z), float(pval)


# ==============================================================================
# 模块 D — WOE → 评分卡
# ==============================================================================
def module_D_scorecard(Xtr_woe, ytr, woe_map, edges_map,
                       PDO=PDO, base_points=BASE_POINTS,
                       base_odds=BASE_ODDS):
    """
    约定: 医学惯例 "分数越高 = 风险越高"
        Points_bin = round( + factor *  _j * WOE_bin )
    """
    lr = LogisticRegression(C=1.0, solver="liblinear",
                            class_weight=None, max_iter=1000,
                            random_state=RNG)
    lr.fit(Xtr_woe, ytr)
    intercept = float(lr.intercept_[0])
    coefs = {f: float(b) for f, b in zip(Xtr_woe.columns, lr.coef_[0])}

    factor = PDO / np.log(2)
    offset = base_points - factor * np.log(base_odds)
    base_adjust = -factor * intercept

    scorecard = {}
    for f in Xtr_woe.columns:
        tbl = woe_map[f].copy()
        tbl["points"] = (factor * coefs[f] * tbl["woe"]).round().astype(int)
        tbl["beta"] = coefs[f]
        scorecard[f] = tbl

    print(f"[D] PDO={PDO}, Base={base_points}, Offset={offset:.2f}, "
          f"Intercept adjust={base_adjust:.2f}")
    for f, b in coefs.items():
```

```
        print(f"    [{f}] = {b:+.4f}")
    return dict(card=scorecard, intercept=intercept, coefs=coefs,
                factor=float(factor), offset=float(offset),
                base_adjust=float(base_adjust),
                base_points=base_points, PDO=PDO)


def apply_scorecard(X_raw, sc, woe_map, edges_map):
    total = np.full(len(X_raw), sc["offset"] + sc["base_adjust"], dtype=float)
    for f in X_raw.columns:
        tbl = sc["card"][f]
        b2p = tbl.set_index("bin_id")["points"].to_dict()
        x = X_raw[f].values
        if f in FEATURES_CONT:
            edges = edges_map[f]
            bins = np.digitize(x, edges[1:-1])
        else:
            uniq = edges_map[f]
            mapping = {v: i for i, v in enumerate(uniq)}
            bins = np.array([mapping.get(v, 0) for v in x])
        total += np.array([b2p.get(int(b), 0) for b in bins])
    return total


def score_to_prob(score, sc):
    logit = (score - sc["offset"]) / sc["factor"]
    return 1 / (1 + np.exp(-logit))


# ==============================================================================
# 模块 E — 多准则阈值
# ==============================================================================
def module_E_thresholds(y, p, B=1000, seed=RNG, label="prob", save=True,
                        sp_target=0.70):
    fpr, tpr, thr = roc_curve(y, p)
    # sklearn thr[0] 为 inf 虚拟阈值，剔除
    finite = np.isfinite(thr)
    fpr, tpr, thr = fpr[finite], tpr[finite], thr[finite]
    j_idx = np.argmax(tpr - fpr)
    t_youden = float(thr[j_idx])
    J_max = float(tpr[j_idx] - fpr[j_idx])

    d_ = np.sqrt(fpr ** 2 + (1 - tpr) ** 2)
    t_near01 = float(thr[np.argmin(d_)])
    t_eer = float(thr[np.argmin(np.abs(fpr - (1 - tpr)))])

    lo_p, hi_p = float(np.quantile(p, 0.01)), float(np.quantile(p, 0.99))
    grid = np.linspace(lo_p, hi_p, 500)
    f1s = [f1_score(y, (p >= g).astype(int), zero_division=0) for g in grid]
    t_f1 = float(grid[int(np.argmax(f1s))])

    # Sp sp_target 下取最小阈值（最宽松的仍满足 Sp 约束的阈值）
    # 即 "最小 c 使得 Sp(c)  sp_target", 作为低危组上界 cL
    mask = (1 - fpr) >= sp_target
    if mask.any():
        t_sp_tgt = float(thr[mask].min())
    else:
        t_sp_tgt = float(np.quantile(p, 0.25))

    rng = np.random.default_rng(seed); bs = []
    for _ in range(B):
        idx = rng.integers(0, len(y), len(y))
        if len(np.unique(y[idx])) < 2:
            continue
        f_, t_, th_ = roc_curve(y[idx], p[idx])
        bs.append(float(th_[np.argmax(t_ - f_)]))
    ci_lo, ci_hi = np.percentile(bs, [2.5, 97.5])
```

```python

    out = dict(
        Youden=round(t_youden, 4), J_max=round(J_max, 4),
        Youden_CI=(round(float(ci_lo), 4), round(float(ci_hi), 4)),
        Near01=round(t_near01, 4), EER=round(t_eer, 4),
        F1=round(t_f1, 4), Sp70=round(t_sp_tgt, 4),
        sp_target=sp_target,
    )
    print(f"[E] 阈值 ({label}): {out}")

    if save:
        plt.figure(figsize=(7, 5))
        order = np.argsort(thr)
        thr_s, tpr_s, fpr_s = thr[order], tpr[order], fpr[order]
        plt.plot(thr_s, tpr_s, label="Sensitivity (TPR)", lw=1.8)
        plt.plot(thr_s, 1 - fpr_s, label="Specificity (TNR)", lw=1.8)
        plt.plot(thr_s, tpr_s - fpr_s, "--", label="Youden J", lw=1.8)
        for name, t in [("Youden", t_youden),
                        ("Sp>=0.70", t_sp_tgt), ("F1", t_f1)]:
            plt.axvline(t, ls=":", alpha=0.5)
        plt.xlabel(f"Threshold ({label})")
        plt.ylabel("Metric value"); plt.legend(loc="best")
        plt.title(f"Multi-criterion threshold search ({label})")
        plt.tight_layout()
        plt.savefig(figure_path("Q2_threshold_search.png"), dpi=150)
        plt.close()
    return out


def cost_sensitive_threshold(y, p, C_FN=5, C_FP=1):
    lo, hi = float(np.quantile(p, 0.01)), float(np.quantile(p, 0.99))
    grid = np.linspace(lo, hi, 300)
    costs = []
    for t in grid:
        pr = (p >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(y, pr, labels=[0, 1]).ravel()
        costs.append(C_FN * fn + C_FP * fp)
    i_opt = int(np.argmin(costs))
    return dict(t_opt=round(float(grid[i_opt]), 4),
                min_cost=int(min(costs)), C_FN=C_FN, C_FP=C_FP)


# ============================================================================
# 模块 F — 三级分层 + Cochran-Armitage
# ============================================================================
def cochran_armitage(y, level_int):
    """2×k 线性趋势检验，已修正方差公式 V = n1·n0/N² · Σ(s_j − s̄)²"""
    y = np.asarray(y, dtype=float)
    s = np.asarray(level_int, dtype=float)
    N = len(y); n1 = float(y.sum()); n0 = N - n1
    if n0 == 0 or n1 == 0:
        return 0.0, 1.0
    s_bar = s.mean()
    T = float(np.sum(y * (s - s_bar)))
    V = (n1 * n0 / N**2) * float(np.sum((s - s_bar) ** 2))
    if V <= 0:
        return 0.0, 1.0
    Z = T / np.sqrt(V)
    pval = 2 * (1 - norm.cdf(abs(Z)))
    return float(Z), float(pval)


def module_F_stratify(y, p, cH, scheme="A", sp_target=0.70):
    """
    scheme='A': cH=Youden, cL = max{c: Sp(c) >= sp_target}
      默认 sp_target=0.70 而非 0.95。原因：本数据阳性率 79.3%，
      阴性样本仅 20.7%，Sp=0.95 要求对应的阈值在分布右尾，
```

```
        与 Youden 阈值几乎重合，导致中层样本为 0。放宽至 0.70
        可保证三层均有足够样本，同时维持 "低危组假阳性  30%"
        的可接受假阳率水平（与 FRS/ASCVD 的低危定义相容）。
    """
    p = np.asarray(p); y = np.asarray(y)
    if scheme == "A":
        fpr, tpr, thr = roc_curve(y, p)
        # sklearn 将 thr[0] 设为 inf (虚拟阈值)，需剔除
        finite = np.isfinite(thr)
        fpr, tpr, thr = fpr[finite], tpr[finite], thr[finite]
        mask = (1 - fpr) >= sp_target
        if mask.any():
            cL = float(thr[mask].min())
        else:
            cL = float(np.quantile(p, 1/4))
        cL = min(cL, cH - 1e-3)
    elif scheme == "B":
        fpr, tpr, thr = roc_curve(y, p)
        cH = float(thr[np.argmax(tpr - fpr)])
        m_lo = p < cH
        if m_lo.sum() > 10 and len(np.unique(y[m_lo])) > 1:
            fpr2, tpr2, thr2 = roc_curve(y[m_lo], p[m_lo])
            cL = float(thr2[np.argmax(tpr2 - fpr2)])
            cL = min(cL, cH - 1e-3)
        else:
            cL = cH - 0.05
    elif scheme == "C":
        cL, cH = np.quantile(p, [1/3, 2/3])
    elif scheme == "D":
        cL, cH = 1/16, 1/3
    else:
        raise ValueError(scheme)

    level = np.where(p < cL, 0, np.where(p < cH, 1, 2))
    strata = []
    for k, name in enumerate(["低", "中", "高"]):
        mk = (level == k)
        pr = float(y[mk].mean()) if mk.sum() else None
        strata.append(dict(
            level=name, n=int(mk.sum()),
            pos_rate=round(pr, 4) if pr is not None else None,
            NNT=round(1.0 / pr, 2) if (pr and pr > 0) else None,
        ))
    Z, pv = cochran_armitage(y, level)
    return dict(scheme=scheme,
                cL=round(float(cL), 4), cH=round(float(cH), 4),
                strata=strata, CA_Z=round(Z, 4), CA_p=pv, level=level)


def module_F_twolayer(y, score, abn, cL, cH):
    """
    两层融合分层:
      Layer 1 硬规则: abn=1 (血脂四项任一异常) → 直接判定为 高风险
      Layer 2 评分卡: abn=0 残余样本按 score 与阈值 {cL,cH} 分层 低/中/高
      融合: 低/中 来自 Layer 2; 高 = Layer 1 全部  Layer 2 高
    """
    y = np.asarray(y); score = np.asarray(score); abn = np.asarray(abn).astype(int)
    # Layer 2 评分分层 (仅 abn=0 时起效)
    level_l2 = np.where(score < cL, 0, np.where(score < cH, 1, 2))
    # 融合: abn=1 全部 → 高 (level=2)
    final_level = np.where(abn == 1, 2, level_l2)
    strata = []
    for k, name in enumerate(["低", "中", "高"]):
        mk = (final_level == k)
        n_ = int(mk.sum())
        pr = float(y[mk].mean()) if n_ > 0 else None
        strata.append(dict(
```

```
            level=name, n=n_,
            pos_rate=round(pr, 4) if pr is not None else None,
            NNT=round(1.0 / pr, 2) if (pr and pr > 0) else None,
        ))
    layer1_n = int((abn == 1).sum())
    layer2_counts = {name: int(((abn == 0) & (level_l2 == k)).sum())
                     for k, name in enumerate(["低", "中", "高"])}
    Z, pv = cochran_armitage(y, final_level)
    return dict(cL=round(float(cL), 4), cH=round(float(cH), 4),
                strata=strata,
                layer1_n=layer1_n,
                layer2_counts=layer2_counts,
                CA_Z=round(Z, 4), CA_p=pv, level=final_level)


# ==============================================================================
# 模块 G — 校准 + HL
# ==============================================================================
def hosmer_lemeshow(y, p, g=10):
    df = pd.DataFrame({"y": y, "p": p})
    df["decile"] = pd.qcut(df["p"], q=g, duplicates="drop", labels=False)
    o1 = df.groupby("decile")["y"].sum().values
    e1 = df.groupby("decile")["p"].sum().values
    n = df.groupby("decile")["y"].count().values
    o0 = n - o1; e0 = n - e1
    e1s = np.where(e1 == 0, 1e-9, e1); e0s = np.where(e0 == 0, 1e-9, e0)
    chi_sq = float(np.sum((o1 - e1) ** 2 / e1s) +
                   np.sum((o0 - e0) ** 2 / e0s))
    dfree = max(int(df["decile"].nunique()) - 2, 1)
    pval = float(1 - chi2.cdf(chi_sq, dfree))
    return chi_sq, pval, dfree


def calibration_intercept_slope(y, p):
    eps = 1e-6
    logit = np.log(np.clip(p, eps, 1 - eps) / np.clip(1 - p, eps, 1 - eps))
    X = np.column_stack([np.ones_like(logit), logit])
    lr = LogisticRegression(fit_intercept=False, C=1e6, solver="lbfgs",
                            max_iter=1000)
    lr.fit(X, y)
    return float(lr.coef_[0][0]), float(lr.coef_[0][1])


def module_G_calibrate(model_factory, Xtr, ytr, Xte, yte, name="GBDT"):
    model = model_factory()
    try:
        cal = CalibratedClassifierCV(model, method="isotonic", cv=5)
        cal.fit(Xtr, ytr)
        p_cal = cal.predict_proba(Xte)[:, 1]
    except Exception as e:
        print(f"[G] Isotonic 失败 ({e}), 回退原概率。")
        model.fit(Xtr, ytr); p_cal = model.predict_proba(Xte)[:, 1]

    pt, pp = calibration_curve(yte, p_cal, n_bins=10, strategy="quantile")
    plt.figure(figsize=(5.5, 5))
    plt.plot(pp, pt, "o-", lw=2, label=f"{name} (isotonic)")
    plt.plot([0, 1], [0, 1], "k--", alpha=0.5, label="Perfect")
    plt.xlabel("Predicted probability"); plt.ylabel("Observed positive rate")
    plt.legend(); plt.title(f"Calibration curve ({name})")
    plt.tight_layout()
    plt.savefig(figure_path("Q2_calibration.png"), dpi=150); plt.close()

    chi_sq, hl_p, dfree = hosmer_lemeshow(yte, p_cal)
    a, b = calibration_intercept_slope(yte, p_cal)
    brier = float(brier_score_loss(yte, p_cal))
    pi = float(yte.mean()); bss = 1 - brier / max(pi * (1 - pi), 1e-9)
    stats_ = dict(Brier=round(brier, 4), BSS=round(bss, 4),
```

```
                HL_chi2=round(chi_sq, 3), HL_df=dfree,
                HL_p=round(hl_p, 4),
                calib_intercept=round(a, 4),
                calib_slope=round(b, 4))
    print(f"[G] 校准 ({name}): {stats_}")
    return p_cal, stats_


# ==============================================================================
# 模块 H — ROC + DeLong
# ==============================================================================
def module_H_roc_delong(y, probs_dict):
    plt.figure(figsize=(6, 5)); aucs = {}
    for k, p in probs_dict.items():
        fpr, tpr, _ = roc_curve(y, p)
        auc = roc_auc_score(y, p); aucs[k] = auc
        plt.plot(fpr, tpr, lw=1.8, label=f"{k} AUC={auc:.3f}")
    plt.plot([0, 1], [0, 1], "k--", alpha=0.5)
    plt.xlabel("FPR (1 - Specificity)"); plt.ylabel("TPR (Sensitivity)")
    plt.title("Three-model ROC (test set)")
    plt.legend(loc="lower right"); plt.tight_layout()
    plt.savefig(figure_path("Q2_roc_curves.png"), dpi=150); plt.close()

    keys = list(probs_dict.keys()); mat = {}
    for k1, k2 in itertools.combinations(keys, 2):
        a1, a2, z, pv = delong_test(y, probs_dict[k1], probs_dict[k2])
        mat[f"{k1}_vs_{k2}"] = dict(auc1=round(a1, 4), auc2=round(a2, 4),
                                    z=round(z, 3), p=round(pv, 6))
    print("[H] DeLong 两两比较:")
    for k, v in mat.items():
        print(f"  {k}: {v}")
    return mat, aucs


# ==============================================================================
# 模块 I — 决策曲线 DCA + 成本敏感
# ==============================================================================
def module_I_decision_curve(y, p, C_FN=5, C_FP=1):
    thresholds = np.arange(0.01, 0.99, 0.01); N = len(y)
    nb_model, nb_all = [], []; pi = y.mean()
    for pt in thresholds:
        pr = (p >= pt).astype(int)
        tn, fp, fn, tp = confusion_matrix(y, pr, labels=[0, 1]).ravel()
        nb_model.append(tp / N - fp / N * (pt / (1 - pt)))
        nb_all.append(pi - (1 - pi) * (pt / (1 - pt)))
    plt.figure(figsize=(6.5, 4.5))
    plt.plot(thresholds, nb_model, lw=2, label="Model")
    plt.plot(thresholds, nb_all, "--", label="Treat all")
    plt.axhline(0, color="k", ls=":", label="Treat none")
    plt.xlabel("Threshold probability $p_t$"); plt.ylabel("Net benefit")
    plt.title("Decision Curve Analysis"); plt.ylim(-0.05, 0.5)
    plt.legend(); plt.tight_layout()
    plt.savefig(figure_path("Q2_dca.png"), dpi=150); plt.close()
    cs = cost_sensitive_threshold(y, p, C_FN=C_FN, C_FP=C_FP)
    print(f"[I] Cost-sensitive: {cs}")
    return cs


# ==============================================================================
# 模块 J — 痰湿子集规则挖掘
# ==============================================================================
def manual_apriori(bin_df, target_col="high_risk",
                   min_support=0.10, min_conf=0.60, min_lift=1.20,
                   max_ant_len=3, top_k=10):
    items = [c for c in bin_df.columns if c != target_col]
    N = len(bin_df); target_sup = bin_df[target_col].mean()
    sup1 = {frozenset([it]): bin_df[it].mean() for it in items}
```

```python
    sup1 = {k: v for k, v in sup1.items() if v >= min_support}
    all_itemsets = {**sup1}
    current = list(sup1.keys())
    for k in range(2, max_ant_len + 1):
        new_sets = set()
        for a, b in itertools.combinations(current, 2):
            u = a | b
            if len(u) == k:
                new_sets.add(u)
        next_level = []
        for s in new_sets:
            cols = list(s)
            sup = bin_df[cols].all(axis=1).mean()
            if sup >= min_support:
                all_itemsets[s] = sup
                next_level.append(s)
        current = next_level
        if not current:
            break
    rules = []
    for ant, sup_a in all_itemsets.items():
        if target_col in ant:
            continue
        cols = list(ant)
        mask = bin_df[cols].all(axis=1)
        if mask.sum() == 0:
            continue
        sup_ab = float((mask & bin_df[target_col].astype(bool)).mean())
        conf = sup_ab / sup_a if sup_a > 0 else 0
        lift = conf / target_sup if target_sup > 0 else 0
        if conf >= min_conf and lift >= min_lift:
            rules.append([sorted(list(ant)), float(sup_a), sup_ab,
                          float(conf), float(lift), int(mask.sum())])
    df = pd.DataFrame(rules, columns=["antecedents", "support",
                                      "sup_ab", "confidence",
                                      "lift", "n_match"])
    df = df.sort_values("lift", ascending=False).head(top_k)
    return df, float(target_sup)


def module_J_phlegm_rules(X_raw, y, tizhi):
    mask = (tizhi == 5)
    Xs, ys = X_raw[mask].copy(), y[mask]
    if len(Xs) < 30:
        print("[J] 痰湿子集样本不足"); return None
    print(f"[J] 痰湿子集 n={len(Xs)}, 阳性率={ys.mean():.4f}")

    bin_df = pd.DataFrame(index=Xs.index)
    for c in FEATURES_CONT:
        if c in Xs.columns:
            med = float(Xs[c].median())
            bin_df[f"{c}_hi"] = (Xs[c] > med).astype(int)
    if "age_group" in Xs.columns:
        bin_df["age_ge3"] = (Xs["age_group"] >= 3).astype(int)
    for c in FEATURES_CAT:
        if c in Xs.columns:
            bin_df[c] = Xs[c].astype(int)
    bin_df["high_risk"] = ys
    bin_df = bin_df.astype(bool)

    rules_df, base_sup = None, float(ys.mean())
    try:
        rules_df, base_sup = manual_apriori(
            bin_df, target_col="high_risk",
            min_support=0.10, min_conf=0.60, min_lift=1.10,
            max_ant_len=3, top_k=10)
        print(f"[J] Apriori base={base_sup:.4f}, Top-{len(rules_df)}:")
```

```
        for _, r in rules_df.iterrows():
            print(f"  {{{', '.join(r['antecedents'])}}} -> high_risk "
                  f"sup={r['support']:.3f} conf={r['confidence']:.3f} "
                  f"lift={r['lift']:.3f} (n={r['n_match']})")
    except Exception as e:
        print(f"[J] Apriori 失败: {e}")

    dt = DecisionTreeClassifier(max_depth=4, min_samples_leaf=10,
                                class_weight="balanced",
                                random_state=RNG).fit(Xs, ys)
    tree_txt = export_text(dt, feature_names=list(Xs.columns), max_depth=4)
    print(f"[J] 决策树:\n{tree_txt}")

    imp = None
    try:
        gb = GradientBoostingClassifier(n_estimators=100, max_depth=3,
                                        learning_rate=0.05,
                                        random_state=RNG).fit(Xs, ys)
        pi = permutation_importance(gb, Xs, ys, n_repeats=30,
                                    random_state=RNG, scoring="roc_auc")
        imp = [(c, round(float(pi.importances_mean[i]), 4),
                round(float(pi.importances_std[i]), 4))
               for i, c in enumerate(Xs.columns)]
        imp.sort(key=lambda t: -t[1])
        print("[J] 痰湿子集置换重要度 Top-5:")
        for c, m, s in imp[:5]:
            print(f"  {c:14s} ΔAUC={m:+.4f} ± {s:.4f}")
    except Exception as e:
        print(f"[J] 置换重要度失败: {e}")

    return dict(apriori=rules_df, tree_text=tree_txt,
                permutation=imp, base_sup=float(ys.mean()))


# ============================================================================
# 模块 K — 稳健性
# ============================================================================
def module_K_robustness(models, d, probs_te, score_total, cH_score):
    yte = d["yte"]
    gb_model, _, Xte_gb = models["GBDT"]
    pi = permutation_importance(gb_model, Xte_gb, yte, n_repeats=30,
                                random_state=RNG, scoring="roc_auc")
    perm = [(c, round(float(pi.importances_mean[i]), 4),
             round(float(pi.importances_std[i]), 4))
            for i, c in enumerate(Xte_gb.columns)]
    perm.sort(key=lambda t: -t[1])
    print("[K] GBDT 置换重要度:")
    for c, m, s in perm:
        print(f"  {c:14s} ΔAUC={m:+.4f} ± {s:.4f}")

    perturb = []
    for dH in [-10, -5, 0, 5, 10]:
        cH_p = cH_score + dH
        r = module_F_stratify(yte, score_total, cH_p, scheme="A")
        perturb.append({
            "cH": round(float(cH_p), 3), "cL": r["cL"],
            "n": [s["n"] for s in r["strata"]],
            "pos": [s["pos_rate"] for s in r["strata"]],
            "CA_Z": r["CA_Z"], "CA_p": r["CA_p"],
        })
    print("[K] 阈值扰动:")
    for p in perturb:
        print(f"  cH={p['cH']} n={p['n']} pos={p['pos']} "
              f"CA_p={p['CA_p']:.2e}")

    level_dict = {}
    for k, p in probs_te.items():
```

```python
        from sklearn.metrics import roc_curve as _rc
        fpr, tpr, thr = _rc(yte, p)
        cH_k = float(thr[np.argmax(tpr - fpr)])
        r = module_F_stratify(yte, p, cH_k, scheme="A")
        level_dict[k] = r["level"]
    kappa = {}
    for k1, k2 in itertools.combinations(level_dict.keys(), 2):
        kappa[f"{k1}_vs_{k2}"] = round(float(cohen_kappa_score(
            level_dict[k1], level_dict[k2], weights="linear")), 4)
    print(f"[K] 分层一致性 : {kappa}")
    return dict(perm_importance=perm, threshold_perturb=perturb, kappa=kappa)


# ==============================================================================
# 模块 L — 六面板汇总图
# ==============================================================================
def module_L_summary_plot(yte, probs_dict, p_cal, strata_main,
                          thr_result, dca_cost, rules, score_total):
    fig = plt.figure(figsize=(17, 10))
    ax1 = plt.subplot(2, 3, 1)
    for k, p in probs_dict.items():
        fpr, tpr, _ = roc_curve(yte, p)
        auc = roc_auc_score(yte, p)
        ax1.plot(fpr, tpr, lw=1.8, label=f"{k} AUC={auc:.3f}")
    ax1.plot([0, 1], [0, 1], "k--", alpha=0.4)
    ax1.set_xlabel("FPR"); ax1.set_ylabel("TPR")
    ax1.legend(fontsize=8); ax1.set_title("(a) Three-model ROC")

    ax2 = plt.subplot(2, 3, 2)
    pt, pp = calibration_curve(yte, p_cal, n_bins=10, strategy="quantile")
    ax2.plot(pp, pt, "o-", lw=2)
    ax2.plot([0, 1], [0, 1], "k--", alpha=0.4)
    ax2.set_xlabel("Predicted"); ax2.set_ylabel("Observed")
    ax2.set_title("(b) Calibration (GBDT+Isotonic)")

    ax3 = plt.subplot(2, 3, 3)
    ax3.hist(score_total[yte == 0], bins=30, alpha=0.6,
             label=f"Negative (n={int((yte==0).sum())})", color="#2ca02c")
    ax3.hist(score_total[yte == 1], bins=30, alpha=0.6,
             label=f"Positive (n={int((yte==1).sum())})", color="#d62728")
    ax3.axvline(strata_main["cL"], color="blue", ls="--",
                label=f"cL={strata_main['cL']:.1f}")
    ax3.axvline(strata_main["cH"], color="red", ls="--",
                label=f"cH={strata_main['cH']:.1f}")
    ax3.set_xlabel("Scorecard total"); ax3.set_ylabel("Frequency")
    ax3.set_title("(c) Score distribution by diagnosis"); ax3.legend(fontsize=8)

    ax4 = plt.subplot(2, 3, 4)
    names = [s["level"] for s in strata_main["strata"]]
    ns = [s["n"] for s in strata_main["strata"]]
    prs = [s["pos_rate"] if s["pos_rate"] is not None else 0
           for s in strata_main["strata"]]
    bars = ax4.bar(names, ns, color=["#2ca02c", "#ff7f0e", "#d62728"])
    ax4b = ax4.twinx()
    ax4b.plot(names, prs, "ko-", lw=2, ms=9); ax4b.set_ylim(0, 1.05)
    for b, n, pr in zip(bars, ns, prs):
        ax4.text(b.get_x() + b.get_width()/2, b.get_height() + 1,
                 f"n={n}", ha="center", va="bottom", fontsize=9)
        ax4b.text(b.get_x() + b.get_width()/2, pr + 0.03,
                  f"{pr*100:.1f}%", ha="center", fontsize=9)
    ax4.set_ylabel("N"); ax4b.set_ylabel("Positive rate")
    ax4.set_title(f"(d) Three-tier stratification CA Z={strata_main['CA_Z']:.2f}, "
                  f"p={strata_main['CA_p']:.1e}")

    ax5 = plt.subplot(2, 3, 5)
    N = len(yte); tg = np.arange(0.01, 0.991, 0.01); nb = []; pi = yte.mean()
    for pt_ in tg:
```

```python
        pr = (p_cal >= pt_).astype(int)
        tn, fp, fn, tp = confusion_matrix(yte, pr, labels=[0, 1]).ravel()
        nb.append(tp / N - fp / N * (pt_ / (1 - pt_)))
    ax5.plot(tg, nb, lw=2, label="Model")
    ax5.plot(tg, pi - (1 - pi) * (tg / (1 - tg)), "--", label="Treat all")
    ax5.axhline(0, color="k", ls=":", label="Treat none")
    ax5.set_xlabel("Threshold probability"); ax5.set_ylabel("Net benefit")
    ax5.set_ylim(-0.05, 0.6); ax5.legend(fontsize=8)
    ax5.set_title(f"(e) DCA (cost-optimal {dca_cost['t_opt']:.3f})")

    ax6 = plt.subplot(2, 3, 6); ax6.axis("off")
    ax6.set_title("(f) Phlegm-damp subset: Top-5 Apriori rules")
    if rules and rules.get("apriori") is not None and len(rules["apriori"]):
        df = rules["apriori"].head(5).copy()
        txt = ""
        for _, r in df.iterrows():
            ant = ", ".join(r["antecedents"])
            txt += (f"* {{{ant}}}\n"
                    f"   sup={r['support']:.3f}, "
                    f"conf={r['confidence']:.3f}, lift={r['lift']:.3f}\n\n")
        ax6.text(0.02, 0.98, txt, va="top", fontsize=9, family="monospace")
    else:
        ax6.text(0.5, 0.5, "No rules meet thresholds",
                 ha="center", va="center")

    plt.tight_layout()
    plt.savefig(figure_path("Q2_summary.png"), dpi=150, bbox_inches="tight")
    plt.close()



def _risk_level_name(level_code):
    return ["低", "中", "高"][int(level_code)]


def _rules_frame(rules):
    if not rules or rules.get("apriori") is None:
        return pd.DataFrame(columns=["antecedents", "support", "sup_ab", "confidence", "lift",
            "n_match"])
    df = rules["apriori"].copy()
    if len(df):
        df["antecedents"] = df["antecedents"].apply(
            lambda items: " & ".join(items) if isinstance(items, (list, tuple)) else str(items)
        )
    return df


def save_table_outputs(d, cv_summary, robust, strata_score_A, strata_variants,
                       score_te, p_cal, probs_te, thr_on_score, thr_on_prob,
                       rules, calib_stats, dca_cost, aucs, delong_mat,
                       score_auc, iv_summary):
    """写出主仓库沿用的 Q2 表格接口，文件名保持无版本号。"""
    yte = np.asarray(d["yte"])
    level = np.asarray(strata_score_A["level"], dtype=int)

    summary_rows = []
    for idx, item in enumerate(strata_score_A["strata"]):
        summary_rows.append({
            "risk_level_code": idx + 1,
            "risk_level": item["level"],
            "n": item["n"],
            "positive_rate": item["pos_rate"],
            "NNT": item["NNT"],
            "cL": strata_score_A["cL"],
            "cH": strata_score_A["cH"],
            "CA_Z": strata_score_A["CA_Z"],
            "CA_p": strata_score_A["CA_p"],
```

```
        })
    risk_summary = pd.DataFrame(summary_rows)
    risk_summary.to_csv(table_path("Q2_risk_stat.csv"), index=False, encoding="utf-8-sig")
    risk_summary.to_csv(table_path("Q2_stratification_summary.csv"), index=False, encoding="utf-8-sig")

    risk_full = d["Xte_raw"].copy()
    risk_full.insert(0, "sample_index", risk_full.index)
    risk_full["y_true"] = yte
    risk_full["is_phlegm"] = (np.asarray(d["tte"]) == 5).astype(int)
    risk_full["score"] = np.round(score_te, 4)
    risk_full["p_calibrated"] = np.round(p_cal, 6)
    for name, prob in probs_te.items():
        risk_full[f"prob_{name}"] = np.round(prob, 6)
    risk_full["risk_level_code"] = level + 1
    risk_full["risk_level"] = [_risk_level_name(v) for v in level]
    risk_full.to_csv(table_path("Q2_risk_full.csv"), index=False, encoding="utf-8-sig")
    risk_full.to_csv(table_path("Q2_stratification_full.csv"), index=False, encoding="utf-8-sig")

    calib_df = pd.DataFrame({"y": yte, "p": p_cal})
    calib_df["decile"] = pd.qcut(calib_df["p"], q=10, duplicates="drop", labels=False)
    calibration = calib_df.groupby("decile", observed=True).agg(
        n=("y", "size"),
        observed_positive=("y", "sum"),
        predicted_positive=("p", "sum"),
        observed_rate=("y", "mean"),
        predicted_mean=("p", "mean"),
    ).reset_index()
    calibration.to_csv(table_path("Q2_calibration.csv"), index=False, encoding="utf-8-sig")

    cv_rows = []
    for model_name, stat in cv_summary.items():
        ci = stat.get("auc_95ci", (None, None))
        cv_rows.append({
            "model": model_name,
            "cv_auc_mean": stat.get("auc_mean"),
            "cv_auc_std": stat.get("auc_std"),
            "cv_auc_ci_low": ci[0],
            "cv_auc_ci_high": ci[1],
            "cv_brier": stat.get("brier"),
            "test_auc": round(float(aucs.get(model_name, np.nan)), 4),
            "fold_aucs": ";".join(str(v) for v in stat.get("fold_aucs", [])),
        })
    pd.DataFrame(cv_rows).to_csv(table_path("Q2_cv_results.csv"), index=False, encoding="utf-8-sig")

    method_rows = []
    for row in cv_rows:
        method_rows.append({
            "method": row["model"],
            "cv_auc_mean": row["cv_auc_mean"],
            "test_auc": row["test_auc"],
            "cv_brier": row["cv_brier"],
            "role": "主筛查模型" if row["model"] == "LR_WOE" else "非线性对照模型",
        })
    method_rows.append({
        "method": "Scorecard",
        "cv_auc_mean": None,
        "test_auc": round(float(score_auc), 4),
        "cv_brier": None,
        "role": "可解释整数评分卡",
    })
    pd.DataFrame(method_rows).to_csv(table_path("Q2_method_compare.csv"), index=False, encoding="utf-8-sig")

    imp_rows = []
```

```
    for rank, (feature, mean_delta, std_delta) in enumerate(robust.get("perm_importance", []), start=1):
        imp_rows.append({
            "rank": rank,
            "feature": feature,
            "delta_auc_mean": mean_delta,
            "delta_auc_std": std_delta,
            "IV": round(float(iv_summary.get(feature, np.nan)), 4),
        })
    pd.DataFrame(imp_rows).to_csv(table_path("Q2_feature_importance.csv"), index=False, encoding="utf-8-sig")

    threshold_rows = []
    for source, values in [("scorecard", thr_on_score), ("probability", thr_on_prob)]:
        for key, value in values.items():
            if key == "Youden_CI":
                threshold_rows.append({
                    "source": source,
                    "criterion": "Youden_CI_low",
                    "threshold": value[0],
                    "note": "bootstrap percentile interval",
                })
                threshold_rows.append({
                    "source": source,
                    "criterion": "Youden_CI_high",
                    "threshold": value[1],
                    "note": "bootstrap percentile interval",
                })
            elif key != "sp_target":
                threshold_rows.append({
                    "source": source,
                    "criterion": key,
                    "threshold": value,
                    "note": f"sp_target={values.get('sp_target')}" if key.startswith("Sp") else "",
                })
    threshold_rows.append({
        "source": "probability",
        "criterion": "cost_sensitive",
        "threshold": dca_cost.get("t_opt"),
        "note": f"C_FN={dca_cost.get('C_FN')}, C_FP={dca_cost.get('C_FP')}, min_cost={dca_cost.get('min_cost')}",
    })
    pd.DataFrame(threshold_rows).to_csv(table_path("Q2_thresholds.csv"), index=False, encoding="utf-8-sig")

    boot_rows = [
        {"metric": "scorecard_youden", "point": thr_on_score.get("Youden"), "ci_low": thr_on_score.get("Youden_CI", (None, None))[0], "ci_high": thr_on_score.get("Youden_CI", (None, None))[1]},
        {"metric": "probability_youden", "point": thr_on_prob.get("Youden"), "ci_low": thr_on_prob.get("Youden_CI", (None, None))[0], "ci_high": thr_on_prob.get("Youden_CI", (None, None))[1]},
    ]
    for model_name, stat in cv_summary.items():
        ci = stat.get("auc_95ci", (None, None))
        boot_rows.append({"metric": f"{model_name}_cv_auc", "point": stat.get("auc_mean"), "ci_low": ci[0], "ci_high": ci[1]})
    pd.DataFrame(boot_rows).to_csv(table_path("Q2_bootstrap_CI.csv"), index=False, encoding="utf-8-sig")

    sensitivity_rows = []
    for item in robust.get("threshold_perturb", []):
        row = {"cH": item["cH"], "cL": item["cL"], "CA_Z": item["CA_Z"], "CA_p": item["CA_p"]}
        for prefix, values in [("n", item.get("n", [])), ("pos", item.get("pos", []))]:
            for label, value in zip(["low", "medium", "high"], values):
                row[f"{prefix}_{label}"] = value
```

```python
        sensitivity_rows.append(row)
    pd.DataFrame(sensitivity_rows).to_csv(table_path("Q2_sensitivity.csv"), index=False, encoding="utf-8-sig")

    rules_df = _rules_frame(rules)
    rules_df.to_csv(table_path("Q2_apriori_rules_full.csv"), index=False, encoding="utf-8-sig")
    rules_df.head(10).to_csv(table_path("Q2_apriori_rules.csv"), index=False, encoding="utf-8-sig")
    core_combo = rules_df.head(5).rename(columns={"antecedents": "core_combo"})
    core_combo.to_csv(table_path("Q2_core_combo.csv"), index=False, encoding="utf-8-sig")
    with open(table_path("Q2_tree_rules.txt"), "w", encoding="utf-8") as fp:
        fp.write(rules.get("tree_text", "") if rules else "")

    summary_metrics = [
        {"metric": "n_total", "value": int(len(d["X_raw"]))},
        {"metric": "positive_rate", "value": round(float(d["pos_rate"]), 4)},
        {"metric": "leakage_consistency", "value": round(float(d["leak_rate"]), 4)},
        {"metric": "GBDT_test_auc", "value": round(float(aucs.get("GBDT", np.nan)), 4)},
        {"metric": "scorecard_test_auc", "value": round(float(score_auc), 4)},
        {"metric": "Brier", "value": calib_stats.get("Brier")},
        {"metric": "HL_p", "value": calib_stats.get("HL_p")},
        {"metric": "cL_score", "value": strata_score_A.get("cL")},
        {"metric": "cH_score", "value": strata_score_A.get("cH")},
        {"metric": "CA_Z", "value": strata_score_A.get("CA_Z")},
        {"metric": "CA_p", "value": strata_score_A.get("CA_p")},
        {"metric": "best_IV_feature", "value": max(iv_summary.items(), key=lambda kv: kv[1])[0]},
    ]
    pd.DataFrame(summary_metrics).to_csv(table_path("Q2_summary_metrics.csv"), index=False, encoding="utf-8-sig")

    variant_rows = []
    for name, result in strata_variants.items():
        row = {"scheme": name, "cL": result["cL"], "cH": result["cH"], "CA_Z": result["CA_Z"], "CA_p": result["CA_p"]}
        for prefix, values in [("n", [s["n"] for s in result["strata"]]), ("pos", [s["pos_rate"] for s in result["strata"]])]:
            for label, value in zip(["low", "medium", "high"], values):
                row[f"{prefix}_{label}"] = value
        variant_rows.append(row)
    pd.DataFrame(variant_rows).to_csv(table_path("Q2_threshold_scheme_compare.csv"), index=False, encoding="utf-8-sig")

    delong_rows = []
    for pair, values in delong_mat.items():
        row = {"comparison": pair}
        row.update(values)
        delong_rows.append(row)
    pd.DataFrame(delong_rows).to_csv(table_path("Q2_delong_compare.csv"), index=False, encoding="utf-8-sig")

# ==============================================================================
# 主流程
# ==============================================================================
def main():
    print("=" * 80)
    print(" 问题二: 纯数据驱动三级风险预警 (WOE-Logistic + GBDT + RF)")
    print("=" * 80)

    d = module_A_preprocess()
    woe_map, edges_map, iv_summary = woe_encode_train(d["Xtr_raw"], d["ytr"])

    print("\n[B] 信息值 IV 表:")
    for f, iv in sorted(iv_summary.items(), key=lambda t: -t[1]):
        strength = ("very strong" if iv > 0.5 else "strong" if iv > 0.3
                    else "medium" if iv > 0.1 else "weak" if iv > 0.02
```

```
                 else "none")
        print(f"  {f:14s} IV={iv:.4f} [{strength}]")

    models, oof, cv_summary, Xtr_woe, Xte_woe = module_C_cv_train(
        d, woe_map, edges_map)

    sc = module_D_scorecard(Xtr_woe, d["ytr"], woe_map, edges_map)
    score_tr = apply_scorecard(d["Xtr_raw"], sc, woe_map, edges_map)
    score_te = apply_scorecard(d["Xte_raw"], sc, woe_map, edges_map)
    print(f"\n[D] 测试集评分: min={score_te.min():.1f}, max={score_te.max():.1f}, "
          f"mean={score_te.mean():.1f}")
    print(f"  阳性组均分 {score_te[d['yte']==1].mean():.2f}")
    print(f"  阴性组均分 {score_te[d['yte']==0].mean():.2f}")
    score_auc = float(roc_auc_score(d["yte"], score_te))
    print(f"  评分卡 AUC = {score_auc:.4f}")

    def gbdt_factory():
        return GradientBoostingClassifier(n_estimators=200, max_depth=3,
                                          learning_rate=0.05, subsample=0.8,
                                          min_samples_leaf=20,
                                          random_state=RNG)
    p_cal, calib_stats = module_G_calibrate(
        gbdt_factory, d["Xtr_raw"], d["ytr"], d["Xte_raw"], d["yte"],
        name="GBDT")

    probs_te = {
        "LR_WOE":  models["LR_WOE"][0].predict_proba(Xte_woe)[:, 1],
        "GBDT":    models["GBDT"][0].predict_proba(d["Xte_raw"])[:, 1],
        "RF":      models["RF"][0].predict_proba(d["Xte_raw"])[:, 1],
        "Scorecard": score_to_prob(score_te, sc),
    }
    delong_mat, aucs = module_H_roc_delong(d["yte"], probs_te)

    thr_on_score = module_E_thresholds(d["yte"], score_te, B=1000,
                                       label="scorecard")
    thr_on_prob = module_E_thresholds(d["yte"], p_cal, B=1000,
                                      label="prob_cal", save=False)

    print("\n[F] 三级分层四方案对比 (基于 GBDT 校准概率):")
    strata_variants = {}
    for scm in ["A", "B", "C", "D"]:
        r = module_F_stratify(d["yte"], p_cal,
                              thr_on_prob["Youden"], scheme=scm)
        strata_variants[scm] = r
        print(f"  方案{scm}: cL={r['cL']} cH={r['cH']} "
              f"n={[s['n'] for s in r['strata']]} "
              f"pos={[s['pos_rate'] for s in r['strata']]} "
              f"CA Z={r['CA_Z']} p={r['CA_p']:.2e}")

    strata_score_A = module_F_stratify(d["yte"], score_te,
                                       thr_on_score["Youden"], scheme="A")
    print(f"\n[F] 单层方案 A (基于评分卡分数, 仅测试集):")
    print(f"  cL={strata_score_A['cL']} cH={strata_score_A['cH']} "
          f"n={[s['n'] for s in strata_score_A['strata']]} "
          f"pos={[s['pos_rate'] for s in strata_score_A['strata']]}")
    print(f"  CA Z={strata_score_A['CA_Z']} p={strata_score_A['CA_p']:.2e}")

    # ============================================================
    # 两层融合 (全数据集 1000 样本)
    #   Layer 1: 血脂四项异常 → 高风险 (硬规则)
    #   Layer 2: 残余样本按评分卡分层 (复用主方案阈值)
    # ============================================================
    cL_fuse = strata_score_A["cL"]
    cH_fuse = strata_score_A["cH"]
    score_full = apply_scorecard(d["X_raw"], sc, woe_map, edges_map)
    strata_twolayer = module_F_twolayer(
        d["y"], score_full, d["abn_full"], cL_fuse, cH_fuse)
```

```python
    print(f"\n[F2] 两层融合 (全数据集 n={len(d['y'])}):")
    print(f" Layer1 硬规则捕获: {strata_twolayer['layer1_n']} 人")
    print(f" Layer2 评分分层: {strata_twolayer['layer2_counts']}")
    print(f" 融合分层 n={[s['n'] for s in strata_twolayer['strata']]}, "
         f"阳性率={[s['pos_rate'] for s in strata_twolayer['strata']]}")
    print(f" CA Z={strata_twolayer['CA_Z']} p={strata_twolayer['CA_p']:.2e}")

    dca_cost = module_I_decision_curve(d["yte"], p_cal, C_FN=5, C_FP=1)

    rules = module_J_phlegm_rules(d["X_raw"], d["y"], d["tizhi"])

    robust = module_K_robustness(models, d, probs_te,
                                score_te, strata_score_A["cH"])

    module_L_summary_plot(d["yte"], probs_te, p_cal, strata_score_A,
                        thr_on_score, dca_cost, rules, score_te)

    save_table_outputs(d, cv_summary, robust, strata_score_A, strata_variants,
                     score_te, p_cal, probs_te, thr_on_score, thr_on_prob,
                     rules, calib_stats, dca_cost, aucs, delong_mat,
                     score_auc, iv_summary)

    dump = {
        "data_profile": dict(
            n=int(len(d["X_raw"])), pos_rate=float(d["pos_rate"]),
            leak_rate=float(d["leak_rate"]),
            n_phlegm=int(d["n_phlegm"]),
            n_train=int(len(d["ytr"])), n_test=int(len(d["yte"])),
        ),
        "features_in_model": FEATURES_ALL,
        "features_excluded": LIPID_BLACKLIST,
        "IV_summary": {f: round(v, 4) for f, v in iv_summary.items()},
        "scorecard_config": dict(
            PDO=sc["PDO"], base_points=sc["base_points"],
            offset=round(sc["offset"], 3),
            base_adjust=round(sc["base_adjust"], 3),
            intercept=round(sc["intercept"], 4),
            coefs={f: round(b, 4) for f, b in sc["coefs"].items()},
        ),
        "scorecard_bins": {
            f: tbl[["bin_id", "lo", "hi", "n", "n1",
                    "woe", "iv_i", "points"]].round(4)
               .to_dict(orient="records")
            for f, tbl in sc["card"].items()
        },
        "cv_summary": cv_summary,
        "test_aucs": {k: round(float(a), 4) for k, a in aucs.items()},
        "score_auc_test": round(score_auc, 4),
        "delong": delong_mat,
        "calibration": calib_stats,
        "thresholds_on_score": thr_on_score,
        "thresholds_on_prob": thr_on_prob,
        "cost_sensitive": dca_cost,
        "stratification_main_A_on_score": {
            "cL": strata_score_A["cL"], "cH": strata_score_A["cH"],
            "strata": strata_score_A["strata"],
            "CA_Z": strata_score_A["CA_Z"],
            "CA_p": strata_score_A["CA_p"]},
        "two_layer_fusion_full_dataset": {
            "cL": strata_twolayer["cL"], "cH": strata_twolayer["cH"],
            "layer1_n": strata_twolayer["layer1_n"],
            "layer2_counts": strata_twolayer["layer2_counts"],
            "strata": strata_twolayer["strata"],
            "CA_Z": strata_twolayer["CA_Z"],
            "CA_p": strata_twolayer["CA_p"]},
        "stratification_variants_on_prob": {
            k: {"cL": v["cL"], "cH": v["cH"],
```

```
                "n": [s["n"] for s in v["strata"]],
                "pos": [s["pos_rate"] for s in v["strata"]],
                "CA_Z": v["CA_Z"], "CA_p": v["CA_p"]}
            for k, v in strata_variants.items()},
        "robustness": {
            "permutation_importance": robust["perm_importance"],
            "threshold_perturb": robust["threshold_perturb"],
            "kappa": robust["kappa"],
        },
        "phlegm_rules": {
            "apriori_top": (rules["apriori"].to_dict(orient="records")
                            if rules and rules.get("apriori") is not None
                            else []),
            "tree_text": rules.get("tree_text") if rules else None,
            "permutation_top5":
                (rules.get("permutation")[:5] if rules and
                 rules.get("permutation") is not None else None),
            "phlegm_pos_rate":
                (rules.get("base_sup") if rules else None),
        },
    }
    with open(table_path("Q2_results.json"), "w", encoding="utf-8") as fp:
        json.dump(dump, fp, indent=2, ensure_ascii=False, default=str)
    print("\n[OK] JSON 写入 src/outputs/tables/Q2_results.json")
    print("[OK] 图表写入 src/outputs/figures/Q2_*.png")
    print("=" * 80)
    return dump


if __name__ == "__main__":
    main()
```

Listing 3: problem2.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
problem3.py — 2026 MathorCup C 题 · 问题三
6 个月痰湿体质个性化干预方案 — v2 统一 NMB 架构

在原严格按题意的加性动力学之上，引入频次序列的月间渐进性约束，
避免最优解出现“五月恒定 + 一月异常”的剧烈跳变处方。
"""

from __future__ import annotations

import argparse
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Tuple

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import FIGURE_DIR, TABLE_DIR, ensure_output_dirs, find_sample_data_path


mpl.rcParams["font.sans-serif"] = [
    "Noto Sans CJK JP",
    "Noto Sans CJK SC",
    "WenQuanYi Zen Hei",
    "SimHei",
    "DejaVu Sans",
```

```python
]
mpl.rcParams["axes.unicode_minus"] = False


# ========================================================================
# Part 0 • 参数常量
# ========================================================================
ACT_COST = {1: 3, 2: 5, 3: 8}
ACT_MIN = {1: 10, 2: 20, 3: 30}
TCM_COST = {1: 30, 2: 80, 3: 130}

RULE_K = 0.03
RULE_F = 0.01
F_ANCHOR = 5
MONTHS = 6
WEEKS_PER_MONTH = 4
BUDGET_SOFT = 2000
BUDGET_HARD = 2400

LAMBDA_LIST = [30, 50, 100, 200]
LAMBDA_BASE = 30
LAMBDA_GRID = np.linspace(0, 260, 53)

F_EFFECTIVE_SET = [5, 6, 7, 8, 9, 10]

TAU_GRID = [1.0, 1.5, 2.0, 2.5, math.inf]
TAU_BASE = 1.5


# ========================================================================
# Part 1 • 动力学
# ========================================================================
def tcm_tier(score: float) -> int:
    if score >= 62:
        return 3
    if score >= 59:
        return 2
    return 1


def r_exercise(
    k: int,
    f: int,
    rule_k: float = RULE_K,
    rule_f: float = RULE_F,
) -> float:
    if f < F_ANCHOR:
        return 0.0
    return rule_k * k + rule_f * (f - F_ANCHOR)


def step_month(
    k: int,
    f: int,
    score_prev: float,
    **dyn_kw,
) -> Tuple[float, int, float, float, float]:
    tier = tcm_tier(score_prev)
    rate = r_exercise(k, f, **dyn_kw)
    score_new = score_prev * (1 - rate)
    act_cost = WEEKS_PER_MONTH * f * ACT_COST[k]
    tcm_cost = TCM_COST[tier]
    return score_new, tier, rate, act_cost, tcm_cost


def tv_l2_rms(f_seq: Iterable[int]) -> float:
    arr = np.asarray(list(f_seq), dtype=float)
```

```python
    if arr.size < 2:
        return 0.0
    diff = np.diff(arr)
    return float(np.sqrt(np.mean(diff**2)))


# ========================================================================
# Part 2 • 月度 Pareto 前沿推进
# ========================================================================
@dataclass
class Plan:
    k_seq: Tuple[int, ...]
    f_seq: Tuple[int, ...]
    S_traj: Tuple[float, ...]
    L_seq: Tuple[int, ...]
    r_seq: Tuple[float, ...]
    cost_month: Tuple[float, ...]
    C_total: float
    S6: float
    S0: float

    @property
    def E(self) -> float:
        return self.S0 - self.S6

    @property
    def rms_df(self) -> float:
        return tv_l2_rms(self.f_seq)


def enumerate_pareto(
    S0: float,
    K_i: List[int],
    f_values: List[int] = F_EFFECTIVE_SET,
    budget: float = BUDGET_HARD,
    tau_max: float = math.inf,
    **dyn_kw,
) -> List[Plan]:
    frontier: List[tuple] = [(S0, 0.0, (), (), (S0,), (), (), ())]
    for t in range(MONTHS):
        new_states = []
        for (S_t, C_t, ks, fs, Straj, Ls, rs, cms) in frontier:
            for k in K_i:
                for f in f_values:
                    if fs:
                        partial = fs + (f,)
                        partial_rms = float(
                            np.sqrt(np.sum(np.diff(np.asarray(partial, dtype=float)) ** 2) / (
                                MONTHS - 1))
                        )
                        if partial_rms > tau_max + 1e-9:
                            continue
                    S_new, L, r, c_act, c_tcm = step_month(k, f, S_t, **dyn_kw)
                    dC = c_act + c_tcm
                    C_new = C_t + dC
                    if C_new > budget:
                        continue
                    new_states.append(
                        (
                            S_new,
                            C_new,
                            ks + (k,),
                            fs + (f,),
                            Straj + (S_new,),
                            Ls + (L,),
                            rs + (r,),
                            cms + (dC,),
```

```
                    )
                )
        if t < MONTHS - 1:
            frontier = _pareto_prune(new_states)
        else:
            frontier = _pareto_prune_global(new_states)

    plans = []
    for (S_t, C_t, ks, fs, Straj, Ls, rs, cms) in frontier:
        if tv_l2_rms(fs) > tau_max + 1e-9:
            continue
        plans.append(
            Plan(
                k_seq=ks,
                f_seq=fs,
                S_traj=Straj,
                L_seq=Ls,
                r_seq=rs,
                cost_month=cms,
                C_total=C_t,
                S6=S_t,
                S0=S0,
            )
        )
    plans.sort(key=lambda p: (p.E, p.C_total))
    return plans


def _pareto_prune(states: List[tuple]) -> List[tuple]:
    if not states:
        return []
    from collections import defaultdict

    groups = defaultdict(list)
    for s in states:
        groups[tcm_tier(s[0])].append(s)

    kept = []
    for items in groups.values():
        items = sorted(items, key=lambda s: (s[1], s[0]))
        min_s = float("inf")
        for s in items:
            if s[0] < min_s - 1e-9:
                kept.append(s)
                min_s = s[0]
    return kept


def _pareto_prune_global(states: List[tuple]) -> List[tuple]:
    if not states:
        return []
    states = sorted(states, key=lambda s: (s[1], s[0]))
    kept = []
    min_s = float("inf")
    for s in states:
        if s[0] < min_s - 1e-9:
            kept.append(s)
            min_s = s[0]
    return kept


# ========================================================================
# Part 3 • Extended Dominance
# ========================================================================
def extended_dominance(plans: List[Plan]) -> List[Plan]:
    P = sorted(plans, key=lambda p: (p.E, p.C_total))
    dedup: Dict[float, Plan] = {}
```

```python
    for p in P:
        if p.E not in dedup or p.C_total < dedup[p.E].C_total:
            dedup[p.E] = p
    P = sorted(dedup.values(), key=lambda p: p.E)
    changed = True
    while changed and len(P) >= 3:
        changed = False
        for j in range(1, len(P) - 1):
            dE1 = P[j].E - P[j - 1].E
            dE2 = P[j + 1].E - P[j].E
            if dE1 <= 1e-9 or dE2 <= 1e-9:
                continue
            icer_prev = (P[j].C_total - P[j - 1].C_total) / dE1
            icer_next = (P[j + 1].C_total - P[j].C_total) / dE2
            if icer_prev > icer_next + 1e-9:
                P.pop(j)
                changed = True
                break
    return P


# ========================================================================
# Part 4 • 选点
# ========================================================================
def nmb_argmax(plans: List[Plan], lam: float) -> Tuple[int, Plan, float]:
    nmbs = [lam * p.E - p.C_total for p in plans]
    idx = int(np.argmax(nmbs))
    return idx, plans[idx], nmbs[idx]


def icer_frontier(plans: List[Plan]) -> List[float]:
    P = sorted(plans, key=lambda p: p.E)
    icers = []
    for i in range(1, len(P)):
        dE = P[i].E - P[i - 1].E
        dC = P[i].C_total - P[i - 1].C_total
        icers.append(dC / dE if dE > 1e-9 else np.inf)
    return icers


def topsis_entropy(plans: List[Plan]) -> Tuple[int, Plan]:
    X = np.array([[p.S6, p.C_total] for p in plans], dtype=float)
    norm = np.sqrt((X**2).sum(axis=0))
    norm[norm == 0] = 1
    R = X / norm
    P = R / R.sum(axis=0)
    P = np.clip(P, 1e-12, 1.0)
    E = -1 / np.log(len(R)) * (P * np.log(P)).sum(axis=0)
    w = (1 - E) / (1 - E).sum()
    V = R * w
    A_plus = V.min(axis=0)
    A_minus = V.max(axis=0)
    d_plus = np.sqrt(((V - A_plus) ** 2).sum(axis=1))
    d_minus = np.sqrt(((V - A_minus) ** 2).sum(axis=1))
    c_rel = d_minus / (d_plus + d_minus + 1e-12)
    idx = int(np.argmax(c_rel))
    return idx, plans[idx]


def kneedle(plans: List[Plan]) -> Tuple[int, Plan]:
    if len(plans) <= 2:
        return 0, plans[0]
    P = sorted(plans, key=lambda p: p.C_total)
    x = np.array([p.C_total for p in P])
    y = np.array([p.S6 for p in P])
    x_n = (x - x.min()) / (np.ptp(x) + 1e-12)
    y_n = (y - y.min()) / (np.ptp(y) + 1e-12)
```

```python
    x0, y0 = x_n[0], y_n[0]
    x1, y1 = x_n[-1], y_n[-1]
    d = np.abs((x1 - x0) * (y0 - y_n) - (x0 - x_n) * (y1 - y0))
    idx = int(np.argmax(d))
    return idx, P[idx]


# ========================================================================
# Part 5 · 数据读取与可行集
# ========================================================================
FEATURE_RENAME = {
    "样本ID": "id",
    "样本 ID": "id",
    "ID": "id",
    "年龄组": "age",
    "年龄": "age",
    "性别": "gender",
    "体质标签": "type",
    "中医体质": "type",
    "痰湿质": "S0",
    "痰湿积分": "S0",
    "痰湿质积分": "S0",
    "ADL总分": "adl",
    "ADL 总分": "adl",
    "IADL总分": "iadl",
    "IADL 总分": "iadl",
    "活动量表总分（ADL总分+IADL总分）": "act_score",
    "活动量表总分": "act_score",
    "活动量表": "act_score",
}


def load_patients(path: str) -> pd.DataFrame:
    ext = Path(path).suffix.lower()
    if ext in (".xlsx", ".xls"):
        df = pd.read_excel(path)
    elif ext == ".csv":
        df = pd.read_csv(path)
    else:
        raise ValueError(f"不支持的文件格式: {ext}")
    df = df.rename(columns={c: FEATURE_RENAME.get(c.strip(), c) for c in df.columns})
    if "S0" not in df.columns:
        for c in df.columns:
            if str(c) == "痰湿质" or ("痰湿" in str(c) and "积分" in str(c)):
                df = df.rename(columns={c: "S0"})
                break
    if "act_score" not in df.columns and "adl" in df.columns and "iadl" in df.columns:
        df["act_score"] = df["adl"] + df["iadl"]
    return df


def resolve_output_dirs(out_root: Optional[str] = None) -> Tuple[Path, Path]:
    if out_root:
        root = Path(out_root)
        fig_dir = root / "figures"
        table_dir = root / "tables"
        fig_dir.mkdir(parents=True, exist_ok=True)
        table_dir.mkdir(parents=True, exist_ok=True)
        return fig_dir, table_dir
    ensure_output_dirs()
    return FIGURE_DIR, TABLE_DIR


def feasible_K(age_group: int, act_score: float) -> List[int]:
    K_age = {1: [1, 2, 3], 2: [1, 2, 3], 3: [1, 2], 4: [1, 2], 5: [1]}
    if act_score < 40:
        K_score = [1]
```

```python
    elif act_score < 60:
        K_score = [1, 2]
    else:
        K_score = [1, 2, 3]
    return sorted(set(K_age[int(age_group)]) & set(K_score))


# ========================================================================
# Part 6 • 推荐对象
# ========================================================================
@dataclass
class Recommendation:
    pid: int
    S0: float
    K_i: List[int]
    age_group: int
    pareto_full: List[Plan]
    frontier_ed: List[Plan]
    rec_nmb: Plan
    nmb_values: Dict[float, float]
    rec_by_lam: Dict[float, Plan]
    rec_topsis: Plan
    rec_knee: Plan
    icer_series: List[float]


def recommend_patient(
    pid: int,
    S0: float,
    age: int,
    act: float,
    tau_max: float = math.inf,
    lam_list: List[float] = LAMBDA_LIST,
    lam_base: float = LAMBDA_BASE,
    **dyn_kw,
) -> Recommendation:
    K_i = feasible_K(age, act)
    if not K_i:
        K_i = [1]
    plans_all = enumerate_pareto(S0, K_i, tau_max=tau_max, **dyn_kw)
    plans_ed = extended_dominance(plans_all)
    rec_by_lam = {lam: nmb_argmax(plans_ed, lam)[1] for lam in lam_list}
    rec_nmb = rec_by_lam[lam_base]
    nmb_values = {lam: nmb_argmax(plans_ed, lam)[2] for lam in lam_list}
    _, rec_topsis = topsis_entropy(plans_ed)
    _, rec_knee = kneedle(plans_ed)
    icers = icer_frontier(plans_ed)
    return Recommendation(
        pid=pid,
        S0=S0,
        K_i=K_i,
        age_group=age,
        pareto_full=plans_all,
        frontier_ed=plans_ed,
        rec_nmb=rec_nmb,
        nmb_values=nmb_values,
        rec_by_lam=rec_by_lam,
        rec_topsis=rec_topsis,
        rec_knee=rec_knee,
        icer_series=icers,
    )


# ========================================================================
# Part 7 • 表格
# ========================================================================
def tabulate_prescription(plan: Plan) -> pd.DataFrame:
```

```python
    rows = []
    for t in range(MONTHS):
        k, f, L = plan.k_seq[t], plan.f_seq[t], plan.L_seq[t]
        rate = plan.r_seq[t]
        S_start = plan.S_traj[t]
        S_end = plan.S_traj[t + 1]
        c_act = WEEKS_PER_MONTH * f * ACT_COST[k]
        c_tcm = TCM_COST[L]
        rows.append(
            {
                "月": t + 1,
                "k": k,
                "f": f,
                "L": L,
                "月初S": round(S_start, 2),
                "r(%)": round(rate * 100, 2),
                "月末S": round(S_end, 2),
                "活动费": c_act,
                "中医费": c_tcm,
                "小计": c_act + c_tcm,
            }
        )
    total_row = pd.DataFrame(
        [
            {
                "月": "合计",
                "k": "",
                "f": "",
                "L": "",
                "月初S": "",
                "r(%)": "",
                "月末S": round(plan.S6, 2),
                "活动费": int(sum(plan.cost_month) - sum(TCM_COST[L] for L in plan.L_seq)),
                "中医费": int(sum(TCM_COST[L] for L in plan.L_seq)),
                "小计": round(plan.C_total, 2),
            }
        ]
    )
    return pd.concat([pd.DataFrame(rows), total_row], ignore_index=True)


def expand_prescription_v2(plan: Plan, S0: float) -> pd.DataFrame:
    rows = []
    S = S0
    partial_f = []
    for month, (k, f) in enumerate(zip(plan.k_seq, plan.f_seq), start=1):
        partial_f.append(f)
        L = tcm_tier(S)
        r = r_exercise(k, f)
        S_end = S * (1 - r)
        act_cost = WEEKS_PER_MONTH * f * ACT_COST[k]
        tcm_cost = TCM_COST[L]
        rows.append(
            {
                "月": month,
                "k": k,
                "f": f,
                "L": L,
                "月初S": round(S, 2),
                "r(%)": round(r * 100, 2),
                "月末S": round(S_end, 2),
                "活动费": act_cost,
                "中医费": tcm_cost,
                "小计": act_cost + tcm_cost,
                "RMS(Δf)": round(tv_l2_rms(partial_f), 2),
            }
        )
```

```python
        S = S_end
    return pd.DataFrame(rows)


# ========================================================================
# Part 8 · 可视化
# ========================================================================
def plot_pareto_three(recs: List[Recommendation], out: str, lam: float = LAMBDA_BASE) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, r in zip(axes, recs):
        C = [p.C_total for p in r.pareto_full]
        S = [p.S6 for p in r.pareto_full]
        Ce = [p.C_total for p in r.frontier_ed]
        Se = [p.S6 for p in r.frontier_ed]
        ax.scatter(C, S, s=18, color="#a0a0a0", alpha=0.4, label="原始 Pareto")
        ax.plot(Ce, Se, "-o", color="steelblue", ms=5, label="ED 后效率前沿")
        ax.scatter(
            r.rec_nmb.C_total,
            r.rec_nmb.S6,
            marker="*",
            s=260,
            c="red",
            zorder=10,
            edgecolor="k",
            linewidths=0.8,
            label=f"NMB 推荐 ( ={int(lam)})",
        )
        ax.scatter(
            r.rec_knee.C_total,
            r.rec_knee.S6,
            marker="^",
            s=95,
            facecolors="none",
            edgecolors="darkorange",
            linewidths=1.6,
            label="Kneedle 对照",
        )
        ax.scatter(
            r.rec_topsis.C_total,
            r.rec_topsis.S6,
            marker="s",
            s=75,
            facecolors="none",
            edgecolors="green",
            linewidths=1.6,
            label="TOPSIS 对照",
        )
        ax.axvline(BUDGET_SOFT, ls="--", color="gray", alpha=0.6)
        ax.set_xlabel("总成本 $C_{\\mathrm{total}}$ (元)")
        ax.set_ylabel("6 月末积分 $S_6$")
        ax.set_title(f"样本 {r.pid} $S_0={r.S0:.0f}$, $K_i=\\{{ {',.'.join(map(str, r.K_i))} \\}}$")
        ax.grid(alpha=0.3)
        if r.pid == recs[0].pid:
            ax.legend(fontsize=8, loc="upper right")
    plt.tight_layout()
    plt.savefig(out, dpi=180, bbox_inches="tight")
    plt.close()


def plot_trajectory_three(recs: List[Recommendation], out: str) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, r in zip(axes, recs):
        traj = list(r.rec_nmb.S_traj)
        ax.plot(range(len(traj)), traj, "-o", color="crimson", lw=2, ms=7)
        for t, (k, f, L) in enumerate(zip(r.rec_nmb.k_seq, r.rec_nmb.f_seq, r.rec_nmb.L_seq)):
```

```python
            ax.annotate(
                f"$L{L}$\n$({k},{f})$",
                xy=(t + 1, traj[t + 1]),
                xytext=(6, 6),
                textcoords="offset points",
                fontsize=8,
            )
        ax.axhline(62, ls="--", color="crimson", alpha=0.5, label="$L=3$ 阈值")
        ax.axhline(59, ls="--", color="darkorange", alpha=0.5, label="$L=2$ 阈值")
        ax.set_xlabel("月份")
        ax.set_ylabel("痰湿积分 $S_t$")
        drop = r.rec_nmb.E / r.S0 * 100 if r.S0 > 0 else 0
        ax.set_title(
            f"样本 {r.pid} — NMB 推荐 (总成本 {r.rec_nmb.C_total:.0f} 元, "
            f"$S_6={r.rec_nmb.S6:.2f}$, 降 {drop:.1f}%)"
        )
        ax.grid(alpha=0.3)
        if r.pid == recs[0].pid:
            ax.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(out, dpi=180, bbox_inches="tight")
    plt.close()


def plot_lambda_sweep(recs: List[Recommendation], out: str) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    lam_grid = np.arange(0, 260, 2)
    for ax, r in zip(axes, recs):
        Cs, S6s = [], []
        for lam in lam_grid:
            _, p, _ = nmb_argmax(r.frontier_ed, lam)
            Cs.append(p.C_total)
            S6s.append(p.S6)
        Ce = [p.C_total for p in r.frontier_ed]
        Se = [p.S6 for p in r.frontier_ed]
        ax.plot(Ce, Se, "-o", color="lightblue", ms=4, lw=1, alpha=0.6, label="ED 效率前沿", zorder=2)
        sc = ax.scatter(Cs, S6s, c=lam_grid, cmap="plasma", s=35, zorder=5)
        for lam, mk, col in [(30, "*", "red"), (50, "D", "orange"), (100, "s", "green"), (200, "^", "purple")]:
            _, p, _ = nmb_argmax(r.frontier_ed, lam)
            ax.scatter(
                p.C_total,
                p.S6,
                marker=mk,
                s=160,
                c=col,
                edgecolor="k",
                linewidths=0.8,
                zorder=10,
                label=f" ={lam}",
            )
        ax.set_xlabel("总成本 $C_{\\mathrm{total}}$ (元)")
        ax.set_ylabel("6 月末积分 $S_6$")
        ax.set_title(f"样本 {r.pid} $S_0={r.S0:.0f}$, $|K|={len(r.K_i)}$")
        ax.grid(alpha=0.3)
        if r.pid == recs[0].pid:
            ax.legend(fontsize=7, loc="upper right")
        fig.colorbar(sc, ax=ax, label=" (元/分)", shrink=0.8)
    plt.tight_layout()
    plt.savefig(out, dpi=180, bbox_inches="tight")
    plt.close()


def plot_icer_ladder(recs: List[Recommendation], out: str) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, r in zip(axes, recs):
```

```python
        if len(r.frontier_ed) < 2:
            ax.text(0.5, 0.5, "前沿点数不足", transform=ax.transAxes, ha="center")
            ax.set_title(f"样本 {r.pid}")
            continue
        P = sorted(r.frontier_ed, key=lambda p: p.E)
        Es = [p.E for p in P]
        icers = [0] + r.icer_series
        for j in range(len(icers) - 1):
            ax.plot([Es[j], Es[j + 1]], [icers[j + 1], icers[j + 1]], "-", color="steelblue", lw=2)
            if j < len(icers) - 2:
                ax.plot([Es[j + 1], Es[j + 1]], [icers[j + 1], icers[j + 2]], ":", color="steelblue", lw=1, alpha=0.7)
        for lam, col, lbl in [(30, "red", " =30"), (50, "orange", " =50"), (100, "green", " =100"), (200, "purple", " =200")]:
            ax.axhline(lam, ls="--", color=col, alpha=0.6, lw=1, label=lbl)
        ax.set_xlabel("效果 $E = S_0 - S_6$ (积分下降)")
        ax.set_ylabel("相邻 ICER $\\Delta C / \\Delta E$ (元/分)")
        ax.set_title(f"样本 {r.pid}: ED 后 ICER 阶梯")
        ax.grid(alpha=0.3)
        if r.pid == recs[0].pid:
            ax.legend(fontsize=8, loc="upper left")
        ax.set_ylim(0, max(260, max(icers) * 1.05) if icers else 260)
    plt.tight_layout()
    plt.savefig(out, dpi=180, bbox_inches="tight")
    plt.close()


def plot_batch_summary(df_batch: pd.DataFrame, out: str) -> None:
    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    (ax1, ax2, ax3), (ax4, ax5, ax6) = axes

    sc = ax1.scatter(df_batch["S0"], df_batch["S6_nmb"], c=df_batch["C_nmb"], cmap="viridis", s=28)
    ax1.plot(
        [df_batch["S0"].min(), df_batch["S0"].max()],
        [df_batch["S0"].min(), df_batch["S0"].max()],
        "--",
        color="gray",
        alpha=0.5,
    )
    ax1.set_xlabel("$S_0$")
    ax1.set_ylabel("$S_6$ (NMB@ =30)")
    ax1.set_title("(a) $S_0$-$S_6$ 散点（色标=总成本）")
    fig.colorbar(sc, ax=ax1, label="$C_\\mathrm{total}$")
    ax1.grid(alpha=0.3)

    ax2.hist(df_batch["C_nmb"], bins=40, color="steelblue", alpha=0.8)
    ax2.axvline(BUDGET_SOFT, ls="--", color="red", label="预算建议 2000")
    ax2.set_xlabel("总成本 (元)")
    ax2.set_ylabel("人数")
    ax2.set_title("(b) 总成本分布")
    ax2.legend()
    ax2.grid(alpha=0.3)

    ax3.hist(df_batch["drop_pct"], bins=40, color="salmon", alpha=0.8)
    ax3.set_xlabel("降幅 (%)")
    ax3.set_ylabel("人数")
    ax3.set_title("(c) 降幅分布")
    ax3.grid(alpha=0.3)

    groups = [df_batch[df_batch["K_size"] == k]["drop_pct"].values for k in [1, 2, 3]]
    ax4.boxplot(groups, tick_labels=["$|K|=1$", "$|K|=2$", "$|K|=3$"])
    ax4.set_ylabel("降幅 (%)")
    ax4.set_title("(d) 按可行集 $|K_i|$ 分组降幅")
    ax4.grid(alpha=0.3)

```

```python
    age_grps = [df_batch[df_batch["age"] == a]["C_nmb"].values for a in [1, 2, 3, 4, 5]]
    ax5.boxplot(age_grps, tick_labels=["40-49", "50-59", "60-69", "70-79", "80-89"])
    ax5.set_ylabel("总成本 (元)")
    ax5.set_title("(e) 按年龄组总成本")
    ax5.grid(alpha=0.3)

    cer_by_tier = df_batch.groupby("S0_tier")["CER"].mean()
    ax6.bar(["<=58", "59-61", ">=62"], [cer_by_tier.get(t, 0) for t in ["低", "中", "高"]],
        color=["#7fbf7b", "#ffcc66", "#d73027"])
    ax6.set_ylabel("平均 CER (元/分)")
    ax6.set_title("(f) 按 $S_0$ 档位的 CER")
    ax6.grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig(out, dpi=180, bbox_inches="tight")
    plt.close()


def plot_frontier_comparison(records: List[dict], out: str) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, rec in zip(axes, records):
        ed_free = rec["unconstrained"].frontier_ed
        ed_base = rec["baseline"].frontier_ed
        best_free = rec["unconstrained"].rec_nmb
        best_base = rec["baseline"].rec_nmb
        C_free = np.array([p.C_total for p in ed_free])
        S_free = np.array([p.S6 for p in ed_free])
        C_base = np.array([p.C_total for p in ed_base])
        S_base = np.array([p.S6 for p in ed_base])
        ax.plot(C_free, S_free, "o-", color="gray", alpha=0.5, label="无约束 ED")
        ax.plot(C_base, S_base, "s-", color="tab:blue", alpha=0.85, label="基准约束 ED")
        ax.scatter([best_free.C_total], [best_free.S6], marker="*", s=250, c="gray", edgecolor
            ="black", label=f"无约束 NMB@30 ({best_free.C_total:.0f}, {best_free.S6:.2f})")
        ax.scatter([best_base.C_total], [best_base.S6], marker="*", s=250, c="tab:red",
            edgecolor="black", label=f"基准 NMB@30 ({best_base.C_total:.0f}, {best_base.S6:.2f
            })")
        ax.axvline(2000, ls="--", color="red", alpha=0.5)
        ax.set_xlabel("总成本 $C_{total}$ (元)")
        ax.set_ylabel("终末积分 $S_6$")
        ax.set_title(f"样本 {rec['sid']} $S_0$={rec['S0']:.0f}, $K$={rec['K']}")
        ax.legend(fontsize=7, loc="upper right")
        ax.grid(alpha=0.3)
    plt.suptitle("无平滑约束 vs 基准平滑约束的效率前沿与 NMB@=30 推荐对比", fontsize=12)
    plt.tight_layout()
    plt.savefig(out, dpi=130, bbox_inches="tight")
    plt.close()


def plot_trajectory_comparison(records: List[dict], out: str) -> None:
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
    for ax, rec in zip(axes, records):
        free = rec["unconstrained"].rec_nmb
        base = rec["baseline"].rec_nmb
        traj_free = list(free.S_traj)
        traj_base = list(base.S_traj)
        t = np.arange(0, MONTHS + 1)
        ax.plot(t, traj_free, "o--", color="gray", alpha=0.7, label=f"无约束: f={free.f_seq}")
        ax.plot(t, traj_base, "s-", color="tab:red", lw=2, label=f"基准: f={base.f_seq}")
        ax.axhline(62, ls=":", color="red", alpha=0.5, label="$L=3$ 阈")
        ax.axhline(59, ls=":", color="orange", alpha=0.5, label="$L=2$ 阈")
        ax.set_xlabel("月份")
        ax.set_ylabel("痰湿积分 $S$")
        ax.set_title(f"样本 {rec['sid']} 轨迹对比")
        ax.legend(fontsize=7, loc="upper right")
        ax.grid(alpha=0.3)
    plt.suptitle("NMB@=30 推荐方案下的 6 月积分轨迹: 无约束 vs 基准约束", fontsize=12)
    plt.tight_layout()
```

```python
    plt.savefig(out, dpi=130, bbox_inches="tight")
    plt.close()


def plot_3d_pareto(records: List[dict], out: str) -> None:
    fig = plt.figure(figsize=(15, 4.5))
    for i, rec in enumerate(records, start=1):
        pts = np.array(
            [[p.C_total, p.S6, p.rms_df] for p in rec["unconstrained"].pareto_full + rec["baseline"].pareto_full]
        )
        pts = np.unique(np.round(pts, 10), axis=0)
        ax = fig.add_subplot(1, 3, i, projection="3d")
        sc = ax.scatter(pts[:, 0], pts[:, 1], pts[:, 2], c=pts[:, 2], cmap="viridis", s=18, alpha=0.7)
        best_base = rec["baseline"].rec_nmb
        best_free = rec["unconstrained"].rec_nmb
        ax.scatter([best_base.C_total], [best_base.S6], [best_base.rms_df], marker="*", s=200, c="red", edgecolor="black", label=f"基准 ({best_base.C_total:.0f},{best_base.S6:.2f})")
        ax.scatter([best_free.C_total], [best_free.S6], [best_free.rms_df], marker="P", s=170, c="gray", edgecolor="black", label=f"无约束 ({best_free.C_total:.0f},{best_free.S6:.2f})")
        ax.set_xlabel("成本 $C$")
        ax.set_ylabel("$S_6$")
        ax.set_zlabel("RMS(Δf)")
        ax.set_title(f"样本 {rec['sid']} $S_0$={rec['S0']:.0f}, K={rec['K']}")
        ax.legend(fontsize=7, loc="upper left")
        plt.colorbar(sc, ax=ax, shrink=0.6, label="RMS(Δf)")
    plt.suptitle("三目标 Pareto 散点图: 降分效果 vs 成本 vs 月间频次波动", fontsize=12)
    plt.tight_layout()
    plt.savefig(out, dpi=130, bbox_inches="tight")
    plt.close()


def plot_batch_comparison(df: pd.DataFrame, out: str) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    axes[0, 0].hist(df["unconstrained_rms"], bins=20, color="gray", alpha=0.7, label="无约束")
    axes[0, 0].hist(df["baseline_rms"], bins=20, color="tab:red", alpha=0.7, label="基准")
    axes[0, 0].axvline(1.5, ls="--", color="red")
    axes[0, 0].set_xlabel("RMS(Δf)")
    axes[0, 0].set_ylabel("人数")
    axes[0, 0].set_title("频次跳变强度分布: 无约束 vs 基准")
    axes[0, 0].legend()

    axes[0, 1].scatter(df["unconstrained_C"], df["baseline_C"], alpha=0.4, s=20)
    lim = [df["unconstrained_C"].min() - 50, df["unconstrained_C"].max() + 50]
    axes[0, 1].plot(lim, lim, "r--", alpha=0.5, label="y=x")
    axes[0, 1].set_xlabel("无约束成本 (元)")
    axes[0, 1].set_ylabel("基准成本 (元)")
    axes[0, 1].set_title("每位患者总成本: 无约束 vs 基准")
    axes[0, 1].legend()

    axes[1, 0].scatter(df["unconstrained_S6"], df["baseline_S6"], alpha=0.4, s=20)
    lim = [df["unconstrained_S6"].min() - 1, df["unconstrained_S6"].max() + 1]
    axes[1, 0].plot(lim, lim, "r--", alpha=0.5, label="y=x")
    axes[1, 0].set_xlabel("无约束 $S_6$")
    axes[1, 0].set_ylabel("基准 $S_6$")
    axes[1, 0].set_title("每位患者终末积分: 无约束 vs 基准")
    axes[1, 0].legend()

    axes[1, 1].hist(df["delta_S6"], bins=25, color="steelblue", alpha=0.8)
    axes[1, 1].axvline(0, color="red", ls="--")
    axes[1, 1].set_xlabel("ΔS6 = 基准 - 无约束 (积分)")
    axes[1, 1].set_ylabel("人数")
    axes[1, 1].set_title(f"基准相对无约束的临床代价分布\n(均值 {df['delta_S6'].mean():+.3f})")
    plt.tight_layout()
```

```python
    plt.savefig(out, dpi=130, bbox_inches="tight")
    plt.close()


# ========================================================================
# Part 9 · 敏感性分析
# ========================================================================
def sensitivity_lambda(recs: List[Recommendation], lam_grid: np.ndarray = LAMBDA_GRID) -> pd.DataFrame:
    rows = []
    for r in recs:
        for lam in lam_grid:
            _, p, nmb = nmb_argmax(r.frontier_ed, lam)
            rows.append(
                {
                    "样本": r.pid,
                    " ": lam,
                    "S6": round(p.S6, 2),
                    "C_total": round(p.C_total, 0),
                    "E": round(p.E, 2),
                    "NMB": round(nmb, 1),
                    "k_seq": str(p.k_seq),
                    "f_seq": str(p.f_seq),
                }
            )
    return pd.DataFrame(rows)


def sensitivity_rule_coefs(recs: List[Recommendation], factors=(0.8, 1.0, 1.2), tau_max: float = math.inf) -> pd.DataFrame:
    rows = []
    for r in recs:
        for s_k in factors:
            for s_f in factors:
                rk = RULE_K * s_k
                rf = RULE_F * s_f
                plans = enumerate_pareto(r.S0, r.K_i, rule_k=rk, rule_f=rf, tau_max=tau_max)
                plans = extended_dominance(plans)
                _, p, _ = nmb_argmax(plans, LAMBDA_BASE)
                rows.append(
                    {
                        "样本": r.pid,
                        "rule_B 缩放": s_k,
                        "rule_C 缩放": s_f,
                        "S6": round(p.S6, 2),
                        "C_total": round(p.C_total, 0),
                        "k_seq": str(p.k_seq),
                        "f_seq": str(p.f_seq),
                    }
                )
    return pd.DataFrame(rows)


def sensitivity_tau(samples: List[dict]) -> Tuple[Dict[int, dict], pd.DataFrame]:
    results = {}
    rows = []
    for sample in samples:
        sid = sample["pid"]
        S0 = sample["S0"]
        age = sample["age"]
        act = sample["act"]
        K = feasible_K(age, act)
        tau_sweep = {}
        for tau in TAU_GRID:
            rec = recommend_patient(sid, S0, age, act, tau_max=tau)
            tau_sweep[tau] = {
                "n_front": len(rec.pareto_full),
```

```
            "n_ed": len(rec.frontier_ed),
            "S6": rec.rec_nmb.S6,
            "C": rec.rec_nmb.C_total,
            "kseq": rec.rec_nmb.k_seq,
            "fseq": rec.rec_nmb.f_seq,
            "tv_rms": rec.rec_nmb.rms_df,
        }
        rows.append(
            {
                "样本": sid,
                "tau": "∞" if math.isinf(tau) else f"{tau:.1f}",
                "n_Pareto": len(rec.pareto_full),
                "n_ED": len(rec.frontier_ed),
                "S6": f"{rec.rec_nmb.S6:.2f}",
                "总成本": int(round(rec.rec_nmb.C_total)),
                "降幅": f"{100 * rec.rec_nmb.E / S0:.1f}%",
                "RMS": f"{rec.rec_nmb.rms_df:.2f}",
                "f序列": str(rec.rec_nmb.f_seq),
            }
        )
    results[sid] = {
        "S0": S0,
        "K": K,
        "age": age,
        "score": act,
        "tau_sweep": tau_sweep,
    }
return results, pd.DataFrame(rows)


# ========================================================================
# Part 10 · 批量
# ========================================================================
def load_target_patients(data_path: str) -> pd.DataFrame:
    df = load_patients(data_path)
    if "type" in df.columns:
        return df[df["type"] == 5].reset_index(drop=True).copy()
    return df.copy()


def _write_batch_group_tables(df_out: pd.DataFrame, table_dir: Path) -> None:
    by_k = df_out.groupby("K_size").agg(
        人数=("id", "size"),
        平均S6=("S6_nmb", "mean"),
        平均总成本=("C_nmb", "mean"),
        平均降幅=("drop_pct", "mean"),
    ).round(2).reset_index()
    by_k.to_csv(table_dir / "Q3_batch_by_K.csv", index=False, encoding="utf-8-sig")

    by_s0 = (
        df_out.groupby("S0_tier")
        .agg(
            人数=("id", "size"),
            平均S6=("S6_nmb", "mean"),
            平均总成本=("C_nmb", "mean"),
            平均降幅=("drop_pct", "mean"),
            平均CER=("CER", "mean"),
        )
        .reindex(["低", "中", "高"])
        .round(2)
        .reset_index()
    )
    by_s0.to_csv(table_dir / "Q3_batch_by_S0_tier.csv", index=False, encoding="utf-8-sig")

    by_age = df_out.groupby("age").agg(
        人数=("id", "size"),
        平均S6=("S6_nmb", "mean"),
```

```
        平均总成本=("C_nmb", "mean"),
        平均降幅=("drop_pct", "mean"),
        平均CER=("CER", "mean"),
    ).round(2).reset_index()
    by_age.to_csv(table_dir / "Q3_batch_by_age.csv", index=False, encoding="utf-8-sig")

    df_cost = df_out.copy()
    df_cost["cost_bin"] = pd.cut(df_cost["C_nmb"], bins=[-np.inf, 500, 1000, np.inf], labels=["低档", "中档", "高档"])
    by_cost = df_cost.groupby("cost_bin", observed=True).agg(
        人数=("id", "size"),
        平均S0=("S0", "mean"),
        平均S6=("S6_nmb", "mean"),
        平均成本=("C_nmb", "mean"),
        平均降幅=("drop_pct", "mean"),
        平均CER=("CER", "mean"),
    ).round(2).reset_index()
    by_cost.to_csv(table_dir / "Q3_batch_by_cost.csv", index=False, encoding="utf-8-sig")


def _write_batch_overall_table(df_out: pd.DataFrame, table_dir: Path, tau_max: float) -> None:
    rows = [
        {"metric": "n_patients", "value": int(len(df_out))},
        {"metric": "tau_max", "value": "inf" if math.isinf(tau_max) else round(float(tau_max), 4)},
        {"metric": "mean_S6", "value": round(float(df_out["S6_nmb"].mean()), 6)},
        {"metric": "mean_total_cost", "value": round(float(df_out["C_nmb"].mean()), 6)},
        {"metric": "mean_drop_pct", "value": round(float(df_out["drop_pct"].mean()), 6)},
        {"metric": "mean_rms_df", "value": round(float(df_out["rms_df"].mean()), 6)},
        {"metric": "share_rms_le_tau", "value": round(float((df_out["rms_df"] <= tau_max + 1e-9).mean()), 6)},
        {"metric": "share_cost_le_budget_soft", "value": round(float((df_out["C_nmb"] <= BUDGET_SOFT + 1e-9).mean()), 6)},
    ]
    pd.DataFrame(rows).to_csv(table_dir / "Q3_batch_overall.csv", index=False, encoding="utf-8-sig")


def run_batch(data_path: str, fig_dir: Path, table_dir: Path, tau_max: float = TAU_BASE) -> pd.DataFrame:
    df_p = load_target_patients(data_path)
    print(f"共 {len(df_p)} 名痰湿体质患者进入批量优化")
    rows = []
    for idx, row in df_p.iterrows():
        rec = recommend_patient(
            int(row.get("id", idx + 1)),
            float(row["S0"]),
            int(row["age"]),
            float(row.get("act_score", row.get("adl", 0) + row.get("iadl", 0))),
            tau_max=tau_max,
        )
        rows.append(
            {
                "id": rec.pid,
                "age": rec.age_group,
                "S0": rec.S0,
                "K_size": len(rec.K_i),
                "S0_tier": ("低" if rec.S0 <= 58 else "中" if rec.S0 < 62 else "高"),
                "S6_nmb": rec.rec_nmb.S6,
                "C_nmb": rec.rec_nmb.C_total,
                "E_nmb": rec.rec_nmb.E,
                "drop_pct": rec.rec_nmb.E / rec.S0 * 100 if rec.S0 > 0 else 0,
                "rms_df": rec.rec_nmb.rms_df,
                "CER": (rec.rec_nmb.C_total / rec.rec_nmb.E) if rec.rec_nmb.E > 1e-6 else np.inf,
                "k_seq": str(rec.rec_nmb.k_seq),
```

```
            "f_seq": str(rec.rec_nmb.f_seq),
            "agree_nmb_topsis": int(rec.rec_nmb.k_seq == rec.rec_topsis.k_seq and rec.rec_nmb.f_seq == rec.rec_topsis.f_seq),
            "agree_nmb_knee": int(rec.rec_nmb.k_seq == rec.rec_knee.k_seq and rec.rec_nmb.f_seq == rec.rec_knee.f_seq),
        }
    )
df_out = pd.DataFrame(rows)
df_out.to_csv(table_dir / "Q3_batch_results.csv", index=False, encoding="utf-8-sig")
_write_batch_group_tables(df_out, table_dir)
_write_batch_overall_table(df_out, table_dir, tau_max)
print("\n按 |K_i| 分组: ")
print(df_out.groupby("K_size")[["S6_nmb", "C_nmb", "drop_pct"]].mean().round(2))
print("\n按 S0 档位分组: ")
print(df_out.groupby("S0_tier")[["S6_nmb", "C_nmb", "drop_pct", "CER"]].mean().round(2))
print("\n按年龄组分组: ")
print(df_out.groupby("age")[["S6_nmb", "C_nmb", "drop_pct"]].mean().round(2))
print(f"\nNMB vs TOPSIS 一致率: {df_out['agree_nmb_topsis'].mean() * 100:.1f}%")
print(f"NMB vs Kneedle 一致率: {df_out['agree_nmb_knee'].mean() * 100:.1f}%")
plot_batch_summary(df_out, str(fig_dir / "Q3_summary.png"))
return df_out


def run_comparison_bundle(data_path: str, fig_dir: Path, table_dir: Path) -> None:
    df_p = load_target_patients(data_path)

    samples = []
    for sid in [1, 2, 3]:
        row = df_p[df_p["id"] == sid].iloc[0]
        samples.append(
            {
                "pid": sid,
                "S0": float(row["S0"]),
                "age": int(row["age"]),
                "act": float(row.get("act_score", row.get("adl", 0) + row.get("iadl", 0))),
            }
        )

    tau_results, tau_df = sensitivity_tau(samples)
    with open(table_dir / "Q3_sample123_tau_sweep.json", "w", encoding="utf-8") as f:
        json.dump(
            {
                str(k): {
                    kk: vv if kk != "tau_sweep" else {
                        ("inf" if math.isinf(tau) else str(tau)): {
                            inner_k: (list(inner_v) if isinstance(inner_v, tuple) else inner_v)
                            for inner_k, inner_v in vals.items()
                        }
                        for tau, vals in v["tau_sweep"].items()
                    }
                    for kk, vv in v.items()
                }
                for k, v in tau_results.items()
            },
            f,
            ensure_ascii=False,
            indent=2,
        )
    tau_df.to_csv(table_dir / "Q3_tau_sensitivity.csv", index=False, encoding="utf-8-sig")

    records = []
    sample_rows = []
    for sample in samples:
        free = recommend_patient(sample["pid"], sample["S0"], sample["age"], sample["act"], tau_max=math.inf)
        base = recommend_patient(sample["pid"], sample["S0"], sample["age"], sample["act"], tau_max=TAU_BASE)
```

```python
        records.append({"sid": sample["pid"], "S0": sample["S0"], "K": feasible_K(sample["age"], sample["act"]), "unconstrained": free, "baseline": base})
        tabulate_prescription(base.rec_nmb).to_csv(table_dir / f"sample{sample['pid']}_prescription.csv", index=False, encoding="utf-8-sig")
        note = {1: "平滑约束非紧", 2: "后两月平滑回落", 3: "逐步升频后保持"}[sample["pid"]]
        sample_rows.append(
            {
                "sample": sample["pid"],
                "S0": sample["S0"],
                "K": str(feasible_K(sample["age"], sample["act"])),
                "k_seq": str(base.rec_nmb.k_seq),
                "f_seq": str(base.rec_nmb.f_seq),
                "S6": round(base.rec_nmb.S6, 2),
                "C_total": round(base.rec_nmb.C_total, 2),
                "rms_df": round(base.rec_nmb.rms_df, 2),
                "note": note,
            }
        )
    pd.DataFrame(sample_rows).to_csv(table_dir / "Q3_sample_recommendations.csv", index=False, encoding="utf-8-sig")

    plot_frontier_comparison(records, str(fig_dir / "Q3_frontier_comparison.png"))
    plot_trajectory_comparison(records, str(fig_dir / "Q3_trajectory_comparison.png"))
    plot_3d_pareto(records, str(fig_dir / "Q3_3d_pareto.png"))

    rows = []
    for _, row in df_p.iterrows():
        sid = int(row["id"])
        S0 = float(row["S0"])
        age = int(row["age"])
        act = float(row.get("act_score", row.get("adl", 0) + row.get("iadl", 0)))
        free = recommend_patient(sid, S0, age, act, tau_max=math.inf)
        base = recommend_patient(sid, S0, age, act, tau_max=TAU_BASE)
        rows.append(
            {
                "id": sid,
                "S0": S0,
                "K": str(feasible_K(age, act)),
                "unconstrained_S6": free.rec_nmb.S6,
                "unconstrained_C": free.rec_nmb.C_total,
                "unconstrained_rms": free.rec_nmb.rms_df,
                "baseline_S6": base.rec_nmb.S6,
                "baseline_C": base.rec_nmb.C_total,
                "baseline_rms": base.rec_nmb.rms_df,
                "delta_S6": base.rec_nmb.S6 - free.rec_nmb.S6,
                "delta_C": base.rec_nmb.C_total - free.rec_nmb.C_total,
                "unconstrained_fseq": str(free.rec_nmb.f_seq),
                "baseline_fseq": str(base.rec_nmb.f_seq),
            }
        )
    batch_cmp = pd.DataFrame(rows)
    batch_cmp.to_csv(table_dir / "Q3_compare_unconstrained_vs_baseline.csv", index=False, encoding="utf-8-sig")
    plot_batch_comparison(batch_cmp, str(fig_dir / "Q3_batch_comparison.png"))


# ========================================================================
# Part 11 · 主程序
# ========================================================================
DEMO_SAMPLES = [
    dict(pid=1, gender="女", age=2, adl=20, iadl=18, S0=64, act=38),
    dict(pid=2, gender="男", age=1, adl=24, iadl=16, S0=58, act=40),
    dict(pid=3, gender="女", age=1, adl=27, iadl=36, S0=59, act=63),
]


def run_demo(fig_dir: Path, table_dir: Path, tau_max: float = TAU_BASE) -> List[
```

```
 Recommendation]:
    recs: List[Recommendation] = []
    for sample in DEMO_SAMPLES:
        print(f"\n=== 样本 {sample['pid']} S0={sample['S0']}, K_i={feasible_K(sample['age'], sample['act'])} ===")
        rec = recommend_patient(sample["pid"], sample["S0"], sample["age"], sample["act"], tau_max=tau_max)
        print(f"原始 Pareto {len(rec.pareto_full)} 点; ED 后 {len(rec.frontier_ed)} 点")
        print(f"ICER 序列: {[round(x, 2) for x in rec.icer_series]}")
        for lam in LAMBDA_LIST:
            p = rec.rec_by_lam[lam]
            drop = p.E / rec.S0 * 100
            print(f"[ ={lam:3d}] S6={p.S6:6.2f}, C={p.C_total:5.0f}, 降={drop:5.1f}%, k={p.k_seq}, f={p.f_seq}")
        print(f"TOPSIS → S6={rec.rec_topsis.S6:.2f}, C={rec.rec_topsis.C_total:.0f}")
        print(f"Kneedle → S6={rec.rec_knee.S6:.2f}, C={rec.rec_knee.C_total:.0f}")
        tab = tabulate_prescription(rec.rec_nmb)
        tab.to_csv(table_dir / f"sample{sample['pid']}_prescription.csv", index=False, encoding="utf-8-sig")
        print("NMB 推荐处方:")
        print(tab.to_string(index=False))
        recs.append(rec)

    plot_pareto_three(recs, str(fig_dir / "Q3_pareto.png"))
    plot_trajectory_three(recs, str(fig_dir / "Q3_trajectory.png"))
    plot_lambda_sweep(recs, str(fig_dir / "Q3_lambda_sweep.png"))
    plot_icer_ladder(recs, str(fig_dir / "Q3_icer_ladder.png"))

    df_lam = sensitivity_lambda(recs, np.arange(0, 305, 5))
    df_lam.to_csv(table_dir / "Q3_lambda_sweep.csv", index=False, encoding="utf-8-sig")
    df_rule = sensitivity_rule_coefs(recs, tau_max=tau_max)
    df_rule.to_csv(table_dir / "Q3_rule_sens.csv", index=False, encoding="utf-8-sig")
    return recs


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Problem 3 solver with baseline smoothness constraint and comparison outputs")
    parser.add_argument("--data", default=None, help="附件 1 路径（xlsx/csv），省略则自动使用 src/Data 中的附件 1")
    parser.add_argument("--out", default=None, help="输出根目录；省略则写入 src/outputs/figures 与 src/outputs/tables")
    parser.add_argument("--mode", choices=["demo", "batch", "all"], default="all")
    args = parser.parse_args()

    fig_dir, table_dir = resolve_output_dirs(args.out)
    data_path = args.data
    if data_path is None and args.mode in ("batch", "all"):
        data_path = str(find_sample_data_path())

    if args.mode in ("demo", "all"):
        print("=" * 70)
        print(f" Part A · 三位样本基准情景（ _max={TAU_BASE}）")
        print("=" * 70)
        recs_demo = run_demo(fig_dir, table_dir, tau_max=TAU_BASE)

    if args.mode in ("batch", "all") and data_path:
        print("\n" + "=" * 70)
        print(f" Part B · 附件 1 · 278 人基准批量（ _max={TAU_BASE}）")
        print("=" * 70)
        df_batch = run_batch(data_path, fig_dir, table_dir, tau_max=TAU_BASE)
        run_comparison_bundle(data_path, fig_dir, table_dir)

    print(f"\n完成。图写入 {fig_dir}; 表写入 {table_dir}")
```

Listing 4: problem3.py